EXAM HEIST



EXAM HEIST

Premium exam material

Get certification quickly with the Exam Heist Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates

First attempt guaranteed success.

https://www.ExamHeist.com

Anthropic

(Claude Certified Architect - Foundations)

Claude Certified Architect – Foundations (CCA-F) Certification

Total: **175 Questions**

Link: https://examheist.com/papers/anthropic/claude-certified-architect

Question: 1 **Exam Heist**

After the web search agent and document analysis agent complete their tasks, the coordinator invokes the synthesis agent. However, the synthesis agent responds that it cannot complete the task because no research findings were provided. What is the most likely cause of this issue?

A. The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories.

B. The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents.

C. The subagents need to share a single API connection to enable automatic context sharing between invocations.

D. The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

Answer: D

Explanation:

A. The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories. Incorrect.

Agents do not require direct access to each other's histories. Proper orchestration passes outputs explicitly via prompts.

B. The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents. Incorrect.

If this were the issue, the agent would receive truncated data, not no data at all. The error indicates missing inputs entirely.

C. The subagents need to share a single API connection to enable automatic context sharing between invocations. Incorrect.

Agent communication does not depend on shared API connections. Context must be explicitly passed by the coordinator.

D. The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt. Correct.

The synthesis agent can only act on the information provided in its prompt. If prior outputs are not passed, it will report missing research findings.

Question: 2 **Exam Heist**

When researching "renewable energy adoption," the web search agent returns recent statistics (2024: 35% adoption) while the document analysis agent extracts data from internal reports (2022: 18% adoption). The synthesis agent incorrectly flags these as contradictory sources rather than recognizing the data shows growth over time. What change would best enable the synthesis agent to correctly interpret such temporal differences?

A. Require subagents to include publication or data collection dates in their structured outputs.

B. Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix.

C. Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric.

D. Configure the web search agent to only return results from the past 6 months

Answer: A

Explanation:

A. Require subagents to include publication or data collection dates in their structured outputs. Correct.

Providing timestamps allows the synthesis agent to understand that the figures refer to different points in time, enabling it to interpret the data as a trend (growth) rather than a contradiction.

B. Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix. Incorrect.

This approach hides useful context and does not help the agent understand relationships between data points over time.

C. Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric. Incorrect.

Discarding older data removes valuable historical insight and prevents trend analysis.

D. Configure the web search agent to only return results from the past 6 months. Incorrect.

Limiting recency reduces context and does not address the core issue of interpreting time-based differences.

Question: 3 **Exam Heist**

Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document analysis agent frequently identifies gaps — for instance, noting "the retrieved sources discuss API authentication but lack details on token refresh patterns" — but under the current strict pipeline, this insight isn't actionable since search has already completed. What is the most effective architectural change?

A. Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.

B. Add a research planning agent before the search phase that decomposes topics into specific sub-questions.

C. Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review.

D. Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected.

Answer: A

Explanation:

A. Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.

This introduces a dynamic, agentic loop (or reflection pattern) into the workflow. Instead of a rigid, linear pipeline where steps cannot be retraced, the system can now adapt based on what it discovers.

The Analysis Agent is the Expert: The document analysis agent is the one actively reading the text and identifying exactly what is missing (e.g., "missing token refresh patterns").

The Coordinator Manages the Flow: By reporting these specific gaps back to the coordinator, the coordinator can intelligently route the workflow back to the search agent with a highly targeted query, then pass the new findings back to the analysis agent to close the loop.

Question: 4 **Exam Heist**

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially complete and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach be Information fidelity with context efficiency when restoring agent state?

A. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.

B. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.

C. Have each agent maintain its own persistent state file and reload it independently at the start of each session.

D. Index all agent outputs in a shared vector store. When resuming each agent queries the store using semantic search to retrieve relevant prior findings.

Answer: A

Explanation:

A. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts. Correct.

This provides **high information fidelity** (structured, complete outputs) while maintaining **context efficiency** (only relevant pieces are re-injected into prompts). The coordinator remains in control of what each agent needs, avoiding unnecessary bloat and duplication.

B. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming. Incorrect.

Conversation logs are often verbose and unstructured, leading to **context overload** and inefficient prompt usage without guaranteed clarity.

C. Have each agent maintain its own persistent state file and reload it independently at the start of each session. Incorrect.

This decentralizes control and can lead to **inconsistencies and coordination issues**, especially when agents need shared or aligned context.

D. Index all agent outputs in a shared vector store. When resuming each agent queries the store using semantic search to retrieve relevant prior findings. Incorrect.

Vector stores are useful for retrieval, but they introduce **probabilistic recall** and may miss or distort critical structured state, reducing fidelity during recovery.

Question: 5 **Exam Heist**

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

A. Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before Invoking synthesis again.

B. Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.

C. Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.

D. Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

Answer: A

Explanation:

A. Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again. Correct.

This introduces an **iterative feedback loop**, where identified gaps are actively addressed. The coordinator maintains control and ensures completeness before final report generation.

B. Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information. Incorrect.

Broader queries may help coverage but are inefficient and still won't guarantee that **specific gaps discovered later** are filled.

C. Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output. Incorrect.

This improves transparency but does **not solve the completeness problem**.

D. Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator. Incorrect.

This breaks **separation of concerns** and reduces system control. The coordinator should manage task delegation, not the synthesis agent.

Question: 6 **Exam Heist**

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely. What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

A. Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations

B. Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents

C. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reading single-precedent granularity

D. Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis

Answer: D

Explanation:

A. Enable the document analysis subagent to spawn its own specialized subagents dynamically when it

encounters cases with many citations. Incorrect.

This decentralizes orchestration and makes the system harder to monitor and debug. The coordinator loses visibility into dynamically spawned agents.

B. Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents. Incorrect.

While this improves scalability, it introduces infrastructure complexity and reduces transparency for debugging at the coordinator level.

C. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity. Incorrect.

This further complicates the architecture and makes tracing execution paths difficult, reducing observability and control.

D. Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis. Correct.

This enables **parallel processing** to reduce latency while keeping orchestration centralized. The coordinator retains full visibility, making monitoring and debugging easier.

Question: 7 **Exam Heist**

Introduction monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, waits for its response, then invokes the document analysis subagent and waits again. These tasks are independent - neither requires the other's output. How should you modify the system to run these subagents concurrently?

A. Switch both subagents to use a Haiku tier model instead of to reduce their individual execution time.

B. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator subagent pair, then aggregates results.

C. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same

D. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns.

Answer: D

Explanation:

A. Switch both subagents to use a Haiku tier model instead to reduce their individual execution time. Incorrect.

This may reduce latency per task, but it does **not address the core issue** of sequential execution vs. parallelism.

B. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator subagent pair, then aggregates results. Incorrect.

This overcomplicates the architecture and duplicates coordinators unnecessarily instead of fixing concurrency within the existing flow.

C. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same time. Incorrect.

Instructions alone are not reliable for enforcing concurrency. Execution behavior depends on how tool calls are structured, not just prompt wording.

D. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns. Correct.

Issuing both tool calls in one response enables **true parallel execution**, since the system can run them concurrently instead of waiting for one to finish before starting the other.

Question: 8 **Exam Heist**

Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing other times reports over hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "Industry analysts estimate $50B market size (methodolo the document analysis agent returns "peer-reviewed study estimates $358 (1578, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the ma 6358-6508 depending on factors." What systematic approach best addresses this?

A. Configure subagents to only report findings meeting a high confidence threshold, filtering uncertain information before it reaches the coordinator.

B. Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.

C. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterization and methodological context.

D. Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calculated reliability scores to produce a statistically grounded synthesis.

Answer: C

Explanation:

A. Configure subagents to only report findings meeting a high confidence threshold, filtering uncertain information before it reaches the coordinator. Incorrect.

This suppresses potentially valuable but uncertain insights and introduces bias by hiding ambiguity rather than managing it.

B. Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources. Incorrect.

While useful for validation, this approach still filters out uncertainty instead of representing it, and may discard novel or emerging insights.

C. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterization and methodological context. Correct.

This directly addresses inconsistent handling of uncertainty by **making it explicit and structured**, allowing users to understand both consensus and disagreement without losing context.

D. Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0–1.0), then weight-average findings by their calculated reliability scores to produce a statistically grounded synthesis. Incorrect.

This introduces artificial precision and may oversimplify complex, qualitative uncertainty, potentially

misleading users.

Question: 9 **Exam Heist**

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summari narrative flow. What change would most improve briefing quality?

A. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose, and technical lists as structured points.

B. Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation (such as Markdown) to facilitate more flexible rendering.

C. Standardize all subagent outputs to JSON with fields for every data type to ensure programmatic consistency across the pipeline.

D. Standardize all subagent outputs to prose summaries with a uniform character to maintain a consistent executive voice regardless of the source material.

Answer: A

Explanation:

A. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose, and technical lists as structured points. Correct.

This preserves the **natural structure and strengths of each data type**, improving clarity, readability, and usefulness of the final briefing.

B. Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation (such as Markdown) to facilitate more flexible rendering. Incorrect.

While helpful for consistency, this does not guarantee **appropriate presentation** of different content types and may still lead to generic formatting.

C. Standardize all subagent outputs to JSON with fields for every data type to ensure programmatic consistency across the pipeline. Incorrect.

This improves structure but shifts complexity to the synthesis stage and does not inherently improve **human-readable output quality**.

D. Standardize all subagent outputs to prose summaries with a uniform character to maintain a consistent executive voice regardless of the source material. Incorrect.

This sacrifices important structure (like tables and lists), reducing clarity and effectiveness for data-heavy content.

Question: 10 **Exam Heist**

The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate — it generates messages like "I'll ask the web search agent to find sources on this topic" — but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with

incomplete information. Logs show no errors. What is the most likely cause?

A. Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in Claude AgentOptions.

B. The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.

C. The coordinator's allowed Tools configuration doesn't include "Task", so while it can reason about delegation, cannot invoke the tool required to spawn subagents.

D. The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked.

Answer: C

Explanation:

A. Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in Claude AgentOptions. Incorrect.

Even with context isolation, subagents would still be invoked — the issue here is that **no invocation happens at all**, not that context is missing.

B. The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified. Incorrect.

Token limits might truncate responses, but this would typically produce malformed outputs or errors — not silent absence of any tool calls.

C. The coordinator's allowed Tools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents. Correct.

The coordinator can **plan and describe delegation**, but without the **Task tool enabled**, it cannot actually execute subagent calls — resulting in no errors but no execution.

D. The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked. Incorrect.

While listing agents can help, the model already demonstrates awareness ("I'll ask the web search agent..."). The problem is **execution capability**, not awareness.

Question: 11 **Exam Heist**

In production, final reports frequently contain claims without proper source attribution. Investigation shows that while the web search and document analysis agents correctly attach citations to their outputs, the synthesis agent loses track of which sources support which conclusions when combining findings. What's the most effective architectural change?

A. Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents.

B. Have the coordinator inject source identifier prefixes into text before each handoff, then parse these prefixes at report generation to reconstruct citations.

C. Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation.

D. Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources.

Answer: D

Explanation:

A. Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents. Incorrect.

This relies on **post-hoc inference**, which is error-prone and can misattribute claims due to semantic ambiguity.

B. Have the coordinator inject source identifier prefixes into text before each handoff, then parse these prefixes at report generation to reconstruct citations. Incorrect.

This is a fragile, text-based workaround that can break during transformations and doesn't scale well.

C. Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation. Incorrect.

This adds unnecessary complexity and still depends on indirect reconstruction rather than preserving attribution explicitly.

D. Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources. Correct.

This ensures **end-to-end attribution fidelity** by keeping claim-to-source relationships explicit and structured throughout the pipeline, preventing loss during synthesis.

Question: 12 **Exam Heist**

After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings. What is the correct approach for providing the synthesis subagent with the information it needs?

A. Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks

B. Include the complete findings from both subagents directly in the synthesis subagent's prompt

C. Pass reference Identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results

D. Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator

Answer: C

Explanation:

A. Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks. Incorrect.

This introduces unnecessary coupling and complexity. Subagents shouldn't need to actively fetch data from others.

B. Include the complete findings from both subagents directly in the synthesis subagent's prompt. Incorrect.

While simple, this approach does **not scale well** for large outputs and can exceed context limits, reducing efficiency.

C. Pass reference identifiers and configure the subagent with read access to a shared memory store where

other subagents deposited their results. Correct.

This is the **most scalable and production-ready approach**. It preserves **information fidelity** while avoiding context bloat, allowing the synthesis agent to retrieve exactly what it needs.

D. Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator. Incorrect.

There is no automatic context inheritance — without explicit data access, the synthesis agent cannot function properly.

Question: 13 **Exam Heist**

The web search agent has gathered several relevant sources for a research topic. The document analysis agent now needs to examine these sources. How does information flow between these two specialized subagents?

A. "The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent.

B. The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events.

C. The web search agent directly invokes the document analysis agent, using the discovered sources as parameters.

D. Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them.

Answer: A

Explanation:

A. The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent. Correct.

This follows the standard orchestration pattern where the **coordinator manages all data flow**, explicitly passing outputs between subagents.

B. The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events. Incorrect.

This introduces unnecessary infrastructure complexity and is not the typical agent orchestration model.

C. The web search agent directly invokes the document analysis agent, using the discovered sources as parameters. Incorrect.

Subagents should not invoke each other directly; this breaks **centralized control and observability**.

D. Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them. Incorrect.

While possible in advanced systems, this is not the standard or simplest approach; it adds complexity without clear necessity in typical pipelines.

Question: 14 **Exam Heist**

After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts

key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations. What context-passing strategy provides the best balance of completeness and efficiency?

A. Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.

B. Pass the full accumulated context from all prior agents.

C. Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and ant Irant excerpts.

D. Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.

Answer: C

Explanation:

A. Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated. Incorrect.

This relies on **post-hoc reconstruction**, which is error-prone and can lead to incorrect or missing citations.

B. Pass the full accumulated context from all prior agents. Incorrect.

This ensures completeness but is highly inefficient (120K+ tokens) and risks exceeding context limits.

C. Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts. Correct.

This provides the **best balance of completeness and efficiency** — retaining precise attribution while keeping context size manageable.

D. Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only. Incorrect.

This loses granularity and makes precise citation mapping difficult, reducing attribution fidelity.

Question: 15 **Exam Heist**

Your search products tool queries an external catalog API that returns paginated results (50 items per request). Production logs show queries frequently match 200+ products, and the design that auto-fetches all pages causes 15-20 second delays. How should you redesign the pagination handling?

A. Create separate search products and fetch more results tools for pagination.

B. Implement server-side relevance ranking and return only the top 50 most relevant items.

C. Add a max pages parameter (default: 2) that controls how many pages are fetched internally.

D. Return the first page with total match count and cursor for additional pages.

Answer: D

Explanation:

A. Create separate search products and fetch more results tools for pagination. Incorrect.

This exposes pagination mechanics to the agent, increasing complexity and coupling tool usage with control flow.

B. Implement server-side relevance ranking and return only the top 50 most relevant items. Incorrect.

While this reduces latency, it **removes access to the full result set**, limiting flexibility when more results are actually needed.

C. Add a max pages parameter (default: 2) that controls how many pages are fetched internally. Incorrect.

This is an improvement over fetching everything, but it still **hides pagination control inside the tool** and may fetch unnecessary data.

D. Return the first page with total match count and cursor for additional pages. Correct.

This enables **lazy loading and explicit control**, allowing the agent to fetch more results only when needed—balancing performance and completeness.

Question: 16 **Exam Heist**

Your search Flights tool calls an external airline API that occasionally returns a 503 Service Unavailable error. What is the most effective way to handle this error in your tool implementation?

A. Return an empty flight list as if the search succeeded but found no matching flights.

B. Log the error internally and return an empty response, letting the model continue without the flight data.

C. Return an error message in the tool result explaining the service is temporarily unavailable.

D. Automatically retry the request up to five times with exponential backoff before returning results to the agent.

Answer: D

Explanation:

A. Return an empty flight list as if the search succeeded but found no matching flights. Incorrect.

This hides the failure and misleads the system into thinking no flights exist, which can lead to incorrect conclusions.

B. Log the error internally and return an empty response, letting the model continue without the flight data. Incorrect.

This still suppresses the failure signal, preventing the agent from taking corrective action.

C. Return an error message in the tool result explaining the service is temporarily unavailable. Incorrect.

While transparent, this alone doesn't attempt recovery and may degrade user experience unnecessarily.

D. Automatically retry the request up to five times with exponential backoff before returning results to the agent. Correct.

This is the most effective approach—handles **transient failures gracefully**, improves reliability, and only surfaces errors if retries fail.

Question: 17 **Exam Heist**

Your MCP server implements a check_availability tool that queries an external calendar API. During testing, you encounter three error conditions:

(1) the tool is called with a malformed request, missing the required user_email parameter
(2) the calendar API returns a 404 because the specified user doesn't exist in the calendar system
(3) the calendar API returns a 503 because the service is temporarily unavailable.
How should each error be reported according to MCP's error handling design?

A. Report all three as tool results with isError: true
B. Report errors 1 and 2 as JSON-RPC protocol errors, report error 3 as a tool result with isError: true
C. Report error 1 as a JSON-RPC protocol error, report errors 2 and 3 as tool results with isError: true
D. Report all three as JSON-RPC protocol errors.

Answer: C

Explanation:

A. Report all three as tool results with isError: true Incorrect.

Malformed requests (error 1) are **protocol-level issues**, not tool execution results, so they should not be reported this way.

B. Report errors 1 and 2 as JSON-RPC protocol errors, report error 3 as a tool result with isError: true Incorrect.

A 404 (error 2) is a **valid tool execution outcome** (the user doesn't exist), not a protocol error.

C. Report error 1 as a JSON-RPC protocol error, report errors 2 and 3 as tool results with isError: true Correct.

Error 1 (malformed request) → **JSON-RPC protocol error** (invalid input)
Error 2 (user not found) → **Tool result with isError: true** (valid execution, meaningful failure)
Error 3 (service unavailable) → **Tool result with isError: true** (transient external failure)

D. Report all three as JSON-RPC protocol errors. Incorrect.

Only malformed requests should be protocol errors; external API responses are **tool-level outcomes**, not protocol failures.

Question: 18 **Exam Heist**

Your documents (query) tool returns results as "Found 3 documents: Q2 Budget Proposal, Q2 Budget Forecast, Annual Review". You want the agent to document (4, multi) and doc (24, multi). What return format would best enable these multi-step workflows?

A. URLs that users can click to open the document in their browser.
B. Structured data containing document IDs and metadata for each result.
C. A JSON array of document titles extracted from the search results.
D. More detailed human-readable descriptions including the size and authors.

Answer: B

Explanation:

A. URLs that users can click to open the document in their browser. Incorrect.

URLs are useful for users, but not ideal for agents performing multi-step workflows that require **reliable**

referencing and further operations.

B. Structured data containing document IDs and metadata for each result. Correct.

This enables the agent to **programmatically reference specific documents** (via IDs) across multiple steps, making workflows like follow-up queries or document retrieval precise and reliable.

C. A JSON array of document titles extracted from the search results. Incorrect.

Titles alone are ambiguous and not stable identifiers, making it difficult for agents to reliably act on specific documents.

D. More detailed human-readable descriptions including the size and authors. Incorrect.

Helpful for users, but still **unstructured** and not suitable for precise multi-step agent operations.

Question: 19 **Exam Heist**

Your agent has access to 50+ specialized API connectors for different external services. As the connector library grew, tool selection accuracy dropped to 58%. You design a search_connectors(description) tool that finds matching connectors, but in testing agents frequently skip searching and call connectors directly (often incorrectly), or search select wrong connectors from the filtered results.
How should you design the tool composition pattern to address both issues?

A. Design connectors with built-in compatibility validation that return descriptive errors for mismatched requests.

B. Design a find_and_execute(description, params) composite tool that searches and immediately executes the best matching connector.

C. Enhance all connector descriptions with detailed usage samples, edge cases, and input requirements. Add few-shot examples showing the correct search-then-use workflow.

D. Design search_connectors to dynamically add matched connectors to the agent's available tools. Connectors start unavailable and persist once discovered.

Answer: D

Explanation:

A. Design connectors with built-in compatibility validation that return descriptive errors for mismatched requests. Incorrect.

This helps with error handling after a wrong choice is made, but does **not improve initial tool selection accuracy**.

B. Design a find_and_execute(description, params) composite tool that searches and immediately executes the best matching connector. Incorrect.

This removes transparency and control, making debugging harder and preventing the agent from reasoning about tool choice.

C. Enhance all connector descriptions with detailed usage samples, edge cases, and input requirements. Add few-shot examples showing the correct search-then-use workflow. Incorrect.

While helpful, this still relies on the agent to follow instructions and does not enforce correct behavior, especially at scale with 50+ tools.

D. Design search_connectors to dynamically add matched connectors to the agent's available tools.

Connectors start unavailable and persist once discovered. Correct.

This enforces the **search-first pattern** by limiting available tools initially and **reducing the decision space**, improving both discovery and correct selection.

Question: 20 **Exam Heist**

Your publish article tool calls an external CMS API that occasionally returns transient errors (network timeouts, 503s) and non-transient errors (403 permission denied, 422 validation failure).
Currently, every error is returned directly to the agent, which leads to the agent retrying non-transient errors and wasting turns on failures that will never succeed. How should you partition error-handling responsibility between the tool implementation and the agent?

A. Handle all errors inside the tool: Implement retries with exponential backoff for every error type, and only surface a failure to the agent after a fixed number of retry attempts have been exhausted.

B. Handle transient errors (timeouts, 503s) with automatic retries inside the tool implementation, and surface non-transient errors (permission denied, validation fallures) to the agent with descriptive messages so it can take corrective action.

C. Surface all errors to the agent immediately with detailed context, and let the agent decide which errors to retry and how many times-keeping the tool implementation stateless and simple.

D. Implement a universal error handler that catches all exceptions and returns a generic "tool unavailable-try again later" message, shielding the agent from error complexity.

Answer: B

Explanation:

A. Handle all errors inside the tool: Implement retries with exponential backoff for every error type, and only surface a failure to the agent after a fixed number of retry attempts have been exhausted.
Incorrect.

This wastes time retrying **non-transient errors** (e.g., 403, 422) that will never succeed and hides useful feedback from the agent.

B. Handle transient errors (timeouts, 503s) with automatic retries inside the tool implementation, and surface non-transient errors (permission denied, validation failures) to the agent with descriptive messages so it can take corrective action. Correct.

This cleanly separates responsibility:

Tool handles **recoverable/transient issues** automatically
Agent receives **actionable errors** it can fix (permissions, input validation)

C. Surface all errors to the agent immediately with detailed context, and let the agent decide which errors to retry and how many times—keeping the tool implementation stateless and simple. Incorrect.

This pushes retry logic to the agent, leading to **inefficient behavior and wasted turns**.

D. Implement a universal error handler that catches all exceptions and returns a generic "tool unavailable—try again later" message, shielding the agent from error complexity. Incorrect.

This removes critical detail, preventing the agent from taking **corrective actions** when possible.

Question: 21 **Exam Heist**

Your remove_team_member tool uses a dry_run: boolean parameter for previewing impacts before execution. Production monitoring shows the agent bypasses the preview step in 15% of calls by calling with dry_run=false directly. You need to ensure every removal is preceded by a preview that the user explicitly confirms. What is the most reliable approach?

A. Add server-side validation that permits dry_run=false only when a dry_run=true call with identical parameters occurred within the past 60 seconds.

B. Replace with two tools: preview_remove_member returns impact details and a single-use confirmation token; execute_remove_member requires that token, binding execution to the specific previewed action.

C. Annotate the tool as requiring confirmation and configure the orchestration layer to prompt the user for approval before forwarding any calls to annotated tools.

D. Add detailed instructions and few-shot examples to the tool description requiring the agent to always call with dry_run=true first and wait for user confirmation before calling with dry_run=false.

Answer: B

Explanation:

A. Add server-side validation that permits dry_run=false only when a dry_run=true call with identical parameters occurred within the past 60 seconds. Incorrect.

This approach is brittle because it depends on timing and does not guarantee that the user actually reviewed or confirmed the preview.

B. Replace with two tools: preview_remove_member returns impact details and a single-use confirmation token; execute_remove_member requires that token, binding execution to the specific previewed action. Correct.

This enforces the correct workflow at the system level by requiring a valid preview step and tying execution to an explicit confirmation, making bypass impossible.

C. Annotate the tool as requiring confirmation and configure the orchestration layer to prompt the user for approval before forwarding any calls to annotated tools. Incorrect.

This depends on orchestration behavior and is not strictly enforced, so it can still be bypassed or misconfigured.

D. Add detailed instructions and few-shot examples to the tool description requiring the agent to always call with dry_run=true first and wait for user confirmation before calling with dry_run=false. Incorrect.

Instruction-based approaches are not reliable for enforcement, as demonstrated by the existing bypass rate.

Question: 22 **Exam Heist**

Your expense reimbursement agent processes employee requests using a process reimbursement tool. Company policy requires that reimbursements above $500 must be approved before funds are disbursed. The agent handles hundreds of requests daily, and you need the threshold enforcement to be tamper-proof regardless of how the agent is prompted ensures the $500 approval threshold cannot be bypassed?

A. The process reimbursement tool accepts an approved by manager parameter. The system prompt instructs the agent to only set this to true after confirming that a manager approved the request. A nightly audit script reviews all reimbursements where approved by manager was set to true.

B. Provide two tools: auto reimburse (hard-coded limit of $500) and manager approval. Include detailed system prompt instructions telling the agent to check the amount and use the appropriate tool. Add a Post ToolUse

hook that logs which tool was called for auditing.

C. The process reimbursement tool accepts amount and details, and internally enforces the threshold; amounts <$500 are auto-disbursed and the tool returns a success confirmation. Amounts >$500 cause the tool to create a pending approval request and return a status indicating manager review is pending.

D. Implement the threshold check in a PreToolUse hook that inspects the amount parameter before process reimbursement executes. If the amount exceeds $500, the hook modifies the context to add a requires approval: true flag, which the tool checks before disbursing.

Answer: C

Explanation:

A. The process reimbursement tool accepts an approved by manager parameter. The system prompt instructs the agent to only set this to true after confirming that a manager approved the request. A nightly audit script reviews all reimbursements where approved by manager was set to true. Incorrect.

This relies on the agent following instructions and post-hoc auditing, which is not tamper-proof and allows bypass at execution time.

B. Provide two tools: auto reimburse (hard-coded limit of $500) and manager approval. Include detailed system prompt instructions telling the agent to check the amount and use the appropriate tool. Add a Post ToolUse hook that logs which tool was called for auditing. Incorrect.

Again depends on agent behavior and correct tool selection. Logging helps auditing but does not prevent misuse.

C. The process reimbursement tool accepts amount and details, and internally enforces the threshold; amounts <$500 are auto-disbursed and the tool returns a success confirmation. Amounts >$500 cause the tool to create a pending approval request and return a status indicating manager review is pending. Correct.

This enforces the rule **inside the tool itself**, making it impossible to bypass regardless of how the agent is prompted.

D. Implement the threshold check in a PreToolUse hook that inspects the amount parameter before process reimbursement executes. If the amount exceeds $500, the hook modifies the context to add a requires approval: true flag, which the tool checks before disbursing. Incorrect.

PreToolUse hooks can be bypassed or misconfigured and still rely on downstream logic. Enforcement should reside directly within the tool for full reliability.

Question: 23 **Exam Heist**

Your order management system requires tools for three distinct operations: issuing refunds (requires amount and reason), canceling orders (requires reason), and res (requires shipping address). Each operation shares an order id parameter but has different additional requirements. You notice during testing that with your current frequently omits required parameters or includes irrelevant ones. What design change will most effectively improve parameter accuracy?

A. Split into three separate tools (each defining only the parameters required for that specific operation.

B. Keep one unified tool with all parameters marked optional, but add few-shot examples in the system prompt showing correct parameter combinations for each operation.

C. Keep one unified tool but add JSON Schema if-then-else conditionals to enforce that parameters like amount are required only when the operation type is "refund".

D. Keep one unified tool with a nested operation object parameter whose internal structure varies by operation

type, documented in the tool description.

Answer: A

Explanation:

A. Split into three separate tools (e.g., issue_refund, cancel_order, reship_order), each defining only the parameters required for that specific operation. Correct.

This reduces ambiguity and ensures the agent only sees **relevant parameters per operation**, leading to much higher accuracy.

B. Keep one unified tool with all parameters marked optional, but add few-shot examples in the system prompt showing correct parameter combinations for each operation. Incorrect.

Examples help, but the schema remains ambiguous, so errors will still occur.

C. Keep one unified tool but add JSON Schema if-then-else conditionals to enforce that parameters like amount are required only when the operation type is "refund". Incorrect.

While technically valid, this increases complexity and is less reliable than simply separating tools.

D. Keep one unified tool with a nested operation object parameter whose internal structure varies by operation type, documented in the tool description. Incorrect.

This adds complexity and cognitive load, making it harder for the agent to consistently provide correct parameters.

Question: 24 **Exam Heist**

Your portfolio value tool returns the total value of a user's investment portfolio. You're deciding between returning a structured JSON object with explicit fields versus returning information as a formatted text string. What is the primary advantage of using structured output with defined fields?

A. Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs.

B. The agent can reliably extract specific values without parsing free form text, reducing errors in subsequent operations.

C. Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values.

D. JSON schemas automatically validate that the underlying API returned correct data before the agent processes it.

Answer: B

Explanation:

A. Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs. Incorrect.

Token usage depends on the content; JSON is not inherently more compact than text and may sometimes use more tokens.

B. The agent can reliably extract specific values without parsing free form text, reducing errors in subsequent operations. Correct.

Structured output provides **clear, predictable fields**, making it easy for the agent to use the data accurately in downstream steps.

C. Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values. Incorrect.

The model is still probabilistic; JSON improves structure, but not deterministic processing.

D. JSON schemas automatically validate that the underlying API returned correct data before the agent processes it. Incorrect.

Schemas define structure, but they do not guarantee correctness of the actual data returned by the API.

Question: 25 **Exam Heist**

Your scheduling agent uses get_available_slots(date, provider_id) to retrieve open appointment times, then book_appointment(provider_id, slot_time, patient_id) to reserve a slot. tickets show that 15% of booking attempts fall with "slot no longer available" because another user booked the slot between the availability check and the booking call. How should you r these tools?

A. Modify book_appointment to return detailed failure information including currently available alternative slots when the requested slot is unavailable, enabling the agent to retry with a di time.

B. Keep both tools but add retry logic to the agent's system prompt, instructing it to call get_available_slots again and select a different time if booking fails.

C. Add a hold_slot(provider_id, slot_time) tool that creates a 60 second temporary reservation, requiring the agent to call it between checking availability and booking.

D. Combine both tools into a single find_and_book_appointment that atomically checks availability and books, returning either the confirmed booking or available alternatives.

Answer: D

Explanation:

A. Modify book_appointment to return detailed failure information including currently available alternative slots when the requested slot is unavailable, enabling the agent to retry with a different time. Incorrect.

This improves recovery but does not fix the **race condition** between availability check and booking.

B. Keep both tools but add retry logic to the agent's system prompt, instructing it to call get_available_slots again and select a different time if booking fails. Incorrect.

This still suffers from the same race condition and relies on agent behavior rather than fixing the underlying issue.

C. Add a hold_slot(provider_id, slot_time) tool that creates a 60 second temporary reservation, requiring the agent to call it between checking availability and booking. Incorrect.

This reduces the issue but introduces additional complexity and still requires multiple steps that can fail.

D. Combine both tools into a single find_and_book_appointment that atomically checks availability and books, returning either the confirmed booking or available alternatives. Correct.

This eliminates the race condition by making the operation **atomic**, ensuring consistency and reliability.

Question: 26 **Exam Heist**

Your agent has a log_workout tool that accepts exercise_type (string), value (number), and measurement (string). Production monitoring shows the agent frequently passes mismatched combinations-using measurement: "reps" for cardio exercises like running, or measurement: "miles" for strength exercises like bench press. Your exercises naturally divide into two categories: cardio (measured in time or distance) and strength (measured in reps and sets). 23% of tool calls have invalid combinations. What approach would most effectively reduce these errors?

A. Implement server-side validation returning descriptive errors for invalid combinations, allowing the agent to retry with corrections.

B. Add enum constraints on measurement limiting values to "minutes", "miles", "reps", or "sets" to prevent arbitrary measurement strings.

C. Add explicit examples to the tool description showing valid combinations (e.g., "For running: use minutes or miles. For push-ups: use reps") with constraints for each exercise category.

D. Split into log_cardio_workout (with duration_minutes or distance_miles parameters) and log_strength_workout (with reps and sets parameters).

Answer: D

Explanation:

A. Implement server-side validation returning descriptive errors for invalid combinations, allowing the agent to retry with corrections. Incorrect.

This catches errors after they occur but does not prevent them, leading to wasted turns and inefficiency.

B. Add enum constraints on measurement limiting values to "minutes", "miles", "reps", or "sets" to prevent arbitrary measurement strings. Incorrect.

This restricts values but does not prevent **invalid combinations** (e.g., still allows "miles" for bench press).

C. Add explicit examples to the tool description showing valid combinations (e.g., "For running: use minutes or miles. For push-ups: use reps") with constraints for each exercise category. Incorrect.

Helpful guidance, but not enforceable — agents can still make mistakes.

D. Split into log_cardio_workout (with duration_minutes or distance_miles parameters) and log_strength_workout (with reps and sets parameters). Correct.

This enforces correctness at the schema level by **eliminating invalid parameter combinations entirely**, significantly reducing errors.

Question: 27 **Exam Heist**

Your MCP server includes archive_file(file_id) and delete_file(file_id) tools. Production logs show the agent calls delete_file when users ask to "remove old backups," policy requires archiving backup files. Both tools currently have minimal descriptions: "Archives a file" and "Deletes a file." Which change most directly improves tool selection?

A. Add a confirmation step that requires users to type "CONFIRM DELETE" before delete_file executes.

B. Implement server-side validation that rejects delete_file calls for files tagged as backups, returning an error message suggesting archive_file.

C. Expand tool descriptions to clarify use cases, adding guidance like "Do not use for backup files" to delete_file.

D. Add few-shot examples to the system prompt demonstrating that requests involving "backup" or "old" should use archive_file.

Answer: C

Explanation:

A. Add a confirmation step that requires users to type "CONFIRM DELETE" before delete_file executes. Incorrect.

This prevents accidental execution but does not improve the agent's **tool selection decision**.

B. Implement server-side validation that rejects delete_file calls for files tagged as backups, returning an error message suggesting archive_file. Incorrect.

This enforces policy after the wrong choice is made but does not directly improve initial selection.

C. Expand tool descriptions to clarify use cases, adding guidance like "Do not use for backup files" to delete_file. Correct.

Clear, specific descriptions directly influence the agent's **tool selection reasoning**, making it less likely to choose the wrong tool.

D. Add few-shot examples to the system prompt demonstrating that requests involving "backup" or "old" should use archive_file. Incorrect.

Helpful, but less direct and less reliable than improving the tool descriptions themselves.

Question: 28 **Exam Heist**

Your CRM agent's delete_contact tool handles requests like "delete the duplicate entry for Acme Corp." The database contains similarly named records (e.g., "Acme Corp," "Acme Corporation," "ACME Corp Inc."), and analytics show 8% of deletions are reversed within 24 hours due to misidentified records. Users have also complained that the current multi-step confirmation flow adds too much friction to routine cleanup tasks. Which approach most effectively reduces the error rate while maintaining workflow efficiency?

A. Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion.

B. Require users to supply the exact record ID from the CRM Interface rather than using natural language references to contact names.

C. Deploy automated duplicate detection that identifies and merges probable duplicates, removing the need for manual deletion requests.

D. Implement soft-delete with a 30-day recovery window so users can undo mistakes without slowing down the deletion workflow.

Answer: A

Explanation:

A. Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion. Correct.

This directly addresses ambiguity by showing **clear distinctions between similar records** while keeping the workflow fast with a lightweight confirmation step.

B. Require users to supply the exact record ID from the CRM interface rather than using natural language references to contact names. Incorrect.

This reduces errors but adds significant friction and hurts usability for routine tasks.

C. Deploy automated duplicate detection that identifies and merges probable duplicates, removing the need for manual deletion requests. Incorrect.

Helpful as a separate improvement, but it doesn't solve **incorrect deletions during manual requests**.

D. Implement soft-delete with a 30-day recovery window so users can undo mistakes without slowing down the deletion workflow. Incorrect.

This mitigates impact after errors occur but does not reduce the **error rate itself**.

Question: 29 **Exam Heist**

After Implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats — Inline citations vs. bibliographies, methodology sections vs. details embedded in Introductions. What's the most effective way to address these failures?

A. Modify your schema to make citations and methodology optional, and flag Incomplete records for manual review rather than falling validation.

B. Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model falls to extract.

C. Add few-shot examples demonstrating extractions from documents with varied structures — showing how to identify citations in different formats and locate methodology details across section types.

D. Implement retry logic that re-sends requests when validation detects empty required fields.

Answer: C

Explanation:

A. Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation. Incorrect.

This lowers data quality standards and avoids solving the extraction problem.

B. Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract. Incorrect.

Regex approaches are brittle and unreliable across varied formats, especially for complex structures like methodology.

C. Add few-shot examples demonstrating extractions from documents with varied structures — showing how to identify citations in different formats and locate methodology details across section types. Correct.

This directly improves the model's ability to **generalize across diverse document formats**, addressing the root cause of missed extractions.

D. Implement retry logic that re-sends requests when validation detects empty required fields. Incorrect.

Retries without improving guidance will likely produce the same incomplete outputs.

Question: 30 **Exam Heist**

The system processes product reviews using tool use with a defined schema: rating (integer 1-5), pros (string array), cons (string array), and overall_sentiment (enum: positive, ne
Testing reveals two issues with brief or ambiguous reviews (-20% of the dataset): (1) for reviews like "Great product!", Claude fabricates specific pros and cons rather than Indica
Information isn't explicitly stated, and (2) for sarcastic reviews like "Well that was.. interesting", Claude picks sentiment arbitrarily since there's no option for ambiguous cases. W
modification best addresses both issues?

A. Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum

B. Allow empty arrays for pros/cons as valid output, and add "unclear" ss the sentiment enum

C. Add an extraction_confidence field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.

D. Allow null values for pros/cons, and add "unclear" to the sentiment earum.

Answer: B

Explanation:

A. Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum Incorrect.

Making fields optional can lead to inconsistent outputs, and “neutral” doesn’t solve ambiguity — it’s different from “unclear”.

B. Allow empty arrays for pros/cons as valid output, and add "unclear" as the sentiment enum Correct.

This prevents fabrication by allowing **explicitly empty outputs** when no details are present, and “unclear” handles ambiguous or sarcastic sentiment appropriately.

C. Add an extraction_confidence field (0.0–1.0) for each value, and filter outputs where any confidence falls below a threshold. Incorrect.

This adds complexity but doesn’t prevent fabrication or resolve ambiguity in outputs.

D. Allow null values for pros/cons, and add "unclear" to the sentiment enum. Incorrect.

Nulls are less consistent than empty arrays for structured outputs and can complicate downstream processing.

Question: 31 **Exam Heist**

Your extraction system implements automatic retries when validation fails. On each retry, the specific validation error is appended to the prompt. This retry-with-error-feedback approach resolves most failures within 2-3 attempts. For which failure pattern would additional retries be LEAST effective?

A. The model extracts "et al." for co-authors when the full list exists only in an external document not in the input

B. The model extracts citation counts as locale-formatted strings ("1234") when the schema requires integers

C. The model extracts dates as ISO 8601 datetime strings ("2003-03-15T00:00:00Z") when the schema requires only the date portion.

D. The model extracts keywords as a nested object organized by category when the schema requires a flat array of strings

Answer: A

Explanation:

A. The model extracts "et al." for co-authors when the full list exists only in an external document not in the input Correct.

Retries won't help because the required information is **not present in the input context**. The model cannot recover missing data through repeated attempts.

B. The model extracts citation counts as locale-formatted strings ("1234") when the schema requires integers Incorrect.

This is a **formatting issue** that can be corrected through retries with validation feedback.

C. The model extracts dates as ISO 8601 datetime strings ("2003-03-15T00:00:00Z") when the schema requires only the date portion (YYYY-MM-DD) Incorrect.

Also a **format mismatch**, which retries can fix easily.

D. The model extracts keywords as a nested object organized by category when the schema requires a flat array of strings Incorrect.

This is a **structural mismatch** that can typically be corrected with retry feedback.

Question: 32 **Exam Heist**

Your invoice extraction uses tool use with strict JSON schemas. JSON syntax errors never occur, but 12% of extractions fail semantic validation--for example, line Item amounts don't extracted total, or vendor IDs don't match valid formats. These failures currently route to manual review. What's the most effective approach to reduce manual review volume while m accuracy?

A. Retry the extraction up to 3 times when validation fallis, accepting the first result that passes validation.

B. Implement post-processing logic that automatically corrects common amors, such as recalculating totais from line items when sums don't match.

C. When validation falls, make a follow-up request with the document, extraction, and validation errors for model correction.

D. Add stricter schema constraints with detailed field descriptions to prevent the model from generating invalid values initially.

Answer: C

Explanation:

A. Retry the extraction up to 3 times when validation fails, accepting the first result that passes validation. Incorrect.

Retries without targeted feedback often repeat the same mistakes and don't reliably fix semantic inconsistencies.

B. Implement post-processing logic that automatically corrects common errors, such as recalculating totals from line items when sums don't match. Incorrect.

While useful for specific cases, this is narrow and brittle, and doesn't address broader validation failures like incorrect IDs.

C. When validation fails, make a follow-up request with the document, extraction, and validation errors for model correction. Correct.

This provides **targeted feedback**, enabling the model to fix specific issues, significantly reducing manual review while maintaining accuracy.

D. Add stricter schema constraints with detailed field descriptions to prevent the model from generating invalid values initially. Incorrect.

Schema improvements help upfront, but they cannot fully prevent **semantic errors** like mismatched totals.

Question: 33 **Exam Heist**

Your team is extracting structured data from 50,000 legacy legal contracts under a two-week deadline. Initial testing with 500 sample documents shows 82% pass JSON schema first attempt, while the remaining 18% fall due to diverse issues — missing required fields, malformed dates, and incorrectly identified parties. Documents that fail typically need refinements targeting their specific failure modes before extraction succeeds. Which batch processing strategy is the most cost-efficient while still meeting the deadline?

A. Split documents into 10 sequential batches of 5,000 each, analysing results and refining prompts between batches to improve extraction quality progressively.

B. Submit all 50,000 documents via batch API, then submit failed extractions in successive batches — refining prompts between each batch — until all documents pass validation.

C. Use the real-time API for all 50,000 documents since the batch API's 24-hour processing window creates unacceptable deadline risk.

D. Process 2,000 sample documents via real time API to identify failure patterns and refine prompts, then batch process all 50,000 with the optimized prompts.

Answer: B

Explanation:

A. Split documents into 10 sequential batches of 5,000 each, analysing results and refining prompts between batches to improve extraction quality progressively. Incorrect.

This introduces unnecessary sequential delays and reduces throughput, risking the deadline.

B. Submit all 50,000 documents via batch API, then submit failed extractions in successive batches — refining prompts between each batch — until all documents pass validation. Correct.

This maximizes **throughput and parallelism upfront**, ensuring the deadline is met. Then it uses **targeted iterative refinement only on failures**, making it cost-efficient while handling diverse failure modes effectively.

C. Use the real-time API for all 50,000 documents since the batch API's 24-hour processing window creates unacceptable deadline risk. Incorrect.

This is unnecessarily expensive and not required given batch processing capabilities.

D. Process 2,000 sample documents via real-time API to identify failure patterns and refine prompts, then batch process all 50,000 with the optimized prompts. Incorrect.

While proactive, this assumes failure patterns generalize well, which the scenario suggests they don't — since failures are diverse and require **case-specific refinements**.

Question: 34 Exam Heist

Your extraction pipeline processes contracts that frequently include amendments. When a contract contains both original terms and later amendments (e.g., original clause specifies "30-day payment terms" while Amendment 1 changes this to "45 days"), the model inconsistently extracts one value or the other with no indication of which applies. What's the most effective approach to improve extraction accuracy for documents with amendments?

A. Preprocess documents with a classifier that identifies and removes superseded sections before the main extraction step.

B. Implement post-extraction validation using pattern matching to detect amendments and flag those extractions for manual review.

C. Redesign the schema so amended fields capture multiple values, each with source location and effective date.

D. Add prompt instructions to always extract the most recent amendment value and ignore superseded original terms.

Answer: C

Explanation:

A. Preprocess documents with a classifier that identifies and removes superseded sections before the main extraction step. Incorrect.

This is brittle and risky — accurately identifying and removing superseded clauses is complex and may lead to loss of important context.

B. Implement post-extraction validation using pattern matching to detect amendments and flag those extractions for manual review. Incorrect.

This increases manual review but does not improve extraction accuracy or resolve ambiguity.

C. Redesign the schema so amended fields capture multiple values, each with source location and effective date. Correct.

This preserves **both original and amended values with context**, enabling accurate interpretation and avoiding ambiguity about which value applies.

D. Add prompt instructions to always extract the most recent amendment value and ignore superseded original terms. Incorrect.

This relies on model judgment, which is inconsistent, and loses traceability of how values changed over time.

Question: 35 Exam Heist

Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream reject any malformed or non-conformant JSON. What approach provides the most reliable schema compliance?

A. Define a tool with your target schema as input parameters and have Claude call it with the extracted data.

B. Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response.

C. Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails.

D. Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON.

Answer: A

Explanation:

A. Define a tool with your target schema as input parameters and have Claude call it with the extracted data. Correct.

Tool use enforces **strict schema compliance at generation time**, ensuring valid, structured JSON that downstream systems can reliably consume.

B. Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response. Incorrect.

This is a fragile workaround and does not guarantee valid or schema-compliant JSON.

C. Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails. Incorrect.

Helpful but not reliable — models can still produce malformed or non-conformant JSON.

D. Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON. Incorrect.

Prompt-based formatting alone cannot guarantee strict compliance, especially in edge cases.

Question: 36 **Exam Heist**

Your schema includes a skills: string[] field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract skills mentioned." What's the most effective improvement?

A. Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."

B. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.

C. Enrich the schema to [scondidering] to capture extraction metadata.

D. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

Answer: D

Explanation:

A. Add constraints: "Extract 10–20 skills maximum, one skill per entry, only explicitly named skills." Incorrect.

This enforces limits but is **arbitrary** and may exclude valid skills or still leave ambiguity in how to split phrases.

B. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries. Incorrect.

Helpful downstream, but it does not fix **inconsistent extraction behavior at the source**.

C. Enrich the schema to capture extraction metadata. Incorrect.

Adds complexity but does not directly address inconsistency in skill identification and formatting.

D. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity. Correct.

Examples directly guide the model on **how to split, what to include, and the expected level of detail**, addressing all three issues effectively.

Question: 37 **Exam Heist**

Your pipeline uses a tool called extract_metadata with a JSON schema for paper details. You've also defined lookup_citations and verify_doi tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls lookup_citations first, which fails because it needs the DOI that extract_metadata would provide. What's the most effective way to ensure structured metadata extraction happens first?

A. Set tool choice to ("type": "tool", "name": "extract_metadata") and process the enrichment requests in subsequent turns after receiving the extracted metadata.

B. Set tool choice to "any" so Claude must use a tool, combined with system prompt instructions prioritizing extract_metadata.

C. Set tool choice to ("type": "tool", "name": "extract_metadata") for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.

D. Set tool choice to "auto" and reorder the tool definitions so extract_metadata appears first in the tools array, since Claude prioritizes earlier-listed tools.

Answer: A

Explanation:

A. Set tool choice to ("type": "tool", "name": "extract_metadata") and process the enrichment requests in subsequent turns after receiving the extracted metadata. Correct.

This enforces the correct **execution order**, ensuring required data (like DOI) is available before dependent tools are called.

B. Set tool choice to "any" so Claude must use a tool, combined with system prompt instructions prioritizing extract_metadata. Incorrect.

This does not guarantee ordering — Claude may still choose the wrong tool first.

C. Set tool choice to ("type": "tool", "name": "extract_metadata") for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur. Incorrect.

This is too rigid and prevents legitimate use of other tools in later steps.

D. Set tool choice to "auto" and reorder the tool definitions so extract_metadata appears first in the tools array, since Claude prioritizes earlier-listed tools. Incorrect.

Tool ordering does not reliably control selection or execution order.

Question: 38 **Exam Heist**

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence >90% have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions. Before deploying, what validation step is most critical?

A. Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.

B. Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.

C. Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.

D. Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.

Answer: B

Explanation:

A. Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data. Incorrect.

Important, but it doesn't ensure the **confidence signal is reliable across different cases** — it only checks overall acceptability.

B. Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate. Correct.

Aggregate accuracy can hide weak spots. You need to ensure **confidence >90% is trustworthy across all segments**, otherwise automation may introduce systematic errors.

C. Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors. Incorrect.

Useful for tuning, but only after confirming the confidence signal is **consistent and reliable across segments**.

D. Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports. Incorrect.

A pilot is valuable, but deploying without validating segment-level reliability first introduces avoidable risk.

Question: 39 **Exam Heist**

Your extraction system uses tool_use with a JSON schema containing 12 fields and detailed descriptions, totaling approximately 2,500 tokens for the complete tool definition. Processing documents under 150K tokens yields 98% accuracy. For documents between 175-190K tokens, accuracy drops to 71%, with information from the final third consistently missed. The model's context window is 200K tokens. What is the most likely cause?

A. Tool definitions consume input context tokens. Combined with system prompts and document content, the total approaches the context limit, degrading end-of-document processing.

B. Very long documents exceed the model's effective attention span regardless of context limits, causing accuracy degradation for content farther from the prompt instructions.

C. The model distributes attention proportionally across input length, causing fields mentioned only once near the document's end to receive insufficient processing focus.

D. Schemas exceeding 8-10 fields increase decision complexity during parameter generation, reducing extraction accuracy independent of document length.

Answer: A

Explanation:

A. Tool definitions consume input context tokens. Combined with system prompts and document content,

the total approaches the context limit, degrading end-of-document processing. Correct.

The tool schema (~2,500 tokens) plus system prompts and large documents push total input close to the **200K context limit**, causing truncation or reduced attention to the final portion — hence missed information in the last third.

B. Very long documents exceed the model's effective attention span regardless of context limits, causing accuracy degradation for content farther from the prompt instructions. Incorrect.

While attention can vary, the sharp drop near the context boundary strongly indicates a **context limit issue**, not general attention decay.

C. The model distributes attention proportionally across input length, causing fields mentioned only once near the document's end to receive insufficient processing focus. Incorrect.

This is a weaker effect and does not explain the **consistent failure in the final third tied to document size thresholds**.

D. Schemas exceeding 8–10 fields increase decision complexity during parameter generation, reducing extraction accuracy independent of document length. Incorrect.

Schema size is constant across cases; it does not explain why accuracy drops only for longer documents.

Question: 40 **Exam Heist**

Your extraction pipeline processes invoices and extracts line items, subtotals, tax amounts, and grand totals. During evaluation, you discover that in 18% of extractions, the sum of extracted line item amounts doesn't match the extracted grand total — sometimes due to OCR errors in the source document, sometimes due to extraction mistakes by the model. Downstream accounting systems reject records with mismatched totals. What's the most effective approach to improve extraction reliability?

A. Add a "calculated total" field where the model sums extracted line items alongside a "stated_total" field. Flag records for human review when values differ.

B. Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining which extracted values are most likely correct.

C. Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions.

D. Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total.

Answer: A

Explanation:

A. Add a "calculated total" field where the model sums extracted line items alongside a "stated_total" field. Flag records for human review when values differ. Correct.

This preserves both sources of truth and enables **reliable validation**. Discrepancies can be flagged explicitly, improving accuracy without silently altering financial data.

B. Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining which extracted values are most likely correct. Incorrect.

This adds complexity and uncertainty — "guessing" which value is correct can introduce errors in financial data.

C. Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions. Incorrect.

Helpful but insufficient — does not handle **OCR errors or real inconsistencies** in source documents.

D. Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total. Incorrect.

This modifies financial data artificially, which is risky and unacceptable for accounting accuracy.

Question: 41 **Exam Heist**

Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

A. Submit all documents to the Batch API with custom ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

B. Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.

C. Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.

D. Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

Answer: D

Explanation:

A. Submit all documents to the Batch API with custom ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions. Incorrect.

Batch processing introduces delays that can exceed the **30-minute requirement**, making it unsuitable for urgent reports.

B. Submit all documents to the real-time Messages API to ensure consistent processing latency across document types. Incorrect.

This meets latency needs but is unnecessarily expensive for standard reports that don't require real-time processing.

C. Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return. Incorrect.

Hourly batching further increases delay and cannot meet urgent processing requirements.

D. Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API. Correct.

This balances **cost efficiency and latency requirements**, ensuring urgent reports are processed quickly while optimizing cost for non-urgent ones.

Question: 42 **Exam Heist**

The extraction pipeline receives documents of varying types — some are invoices, others are contracts, and some are receipts. You've defined separate extraction tools, each with its own schema tailored to the document type. During testing, you observe that with tool_choice: "auto", Claude sometimes returns conversational text instead of calling an extraction tool, causing downstream parsing failures. You need guaranteed structured output without knowing the document type in advance. What's the most effective approach?

A. Consolidate all document types into a single unified-schema extraction tool and force that tool.

B. Keep tool_choice: "auto" with system prompt instructions requiring tool use.

C. Set tool_choice: "any" with all extraction tools defined.

D. Add a preliminary classification call, then make a second call with tool_choice forced to the identified extraction tool.

Answer: C

Explanation:

A. Consolidate all document types into a single unified-schema extraction tool and force that tool. Incorrect.

This reduces extraction accuracy and creates an overly complex schema that doesn't fit all document types well.

B. Keep tool_choice: "auto" with system prompt instructions requiring tool use. Incorrect.

Instructions alone are not enforceable, which is exactly why the issue occurs.

C. Set tool_choice: "any" with all extraction tools defined. Correct.

This guarantees that **a tool will always be called**, eliminating conversational text responses. While tool selection may not always be perfect, it ensures **structured output every time**, which is the primary requirement.

D. Add a preliminary classification call, then make a second call with tool_choice forced to the identified extraction tool. Incorrect.

While more precise, this adds extra latency and complexity. The question prioritizes **guaranteed structured output**, which C achieves more directly.

Question: 43 **Exam Heist**

Monitoring shows 12% of extractions fall Pydantic validation with specific errors like "expected float for quantity, got '2 to 3". Retrying these requests without modification produces failures. What's the most effective approach to recover from these validation failures?

A. Set temperature to 0 to eliminate output variability and ensure consistent formatting

B. Send a follow-up request including the validation error, asking the model to correct its output

C. Pre-process source documents to standardize problematic formats before sending them for extraction

D. Implement a secondary pipeline using a larger model tier to reprocess documents that fail validation

Answer: B

Explanation:

A. Set temperature to 0 to eliminate output variability and ensure consistent formatting Incorrect.

This reduces randomness but won't fix **systematic extraction errors** like misinterpreting ranges ("2 to 3").

B. Send a follow-up request including the validation error, asking the model to correct its output
Correct.

Providing **specific validation feedback** allows the model to correct the exact issue (e.g., convert "2 to 3" into a valid float), making recovery highly effective.

C. Pre-process source documents to standardize problematic formats before sending them for extraction
Incorrect.

Helpful in some cases, but not scalable or sufficient for diverse real-world variations.

D. Implement a secondary pipeline using a larger model tier to reprocess documents that fail validation
Incorrect.

More expensive and not necessary — targeted correction is more efficient and effective.

Question: 44 **Exam Heist**

After three months of weekly sessions, your conversation history has grown to 85,000 tokens. When users ask "What did we conclude about the theme of isolation?", the assistant provides generic literary analysis rather than referencing the group's specific insights from earlier sessions. Discussions often build on previous meetings' conclusions, so maintaining narrative context is important. What's the most effective approach?

A. Add structured XML tags to mark significant discussion conclusions throughout the conversation history.

B. Use semantic embedding to index the full conversation history and retrieve only relevant past exchanges for each user query, replacing the linear conversation format with retrieved segments.

C. Implement rolling window truncation to keep only the most recent 25,000 tokens.

D. Implement progressive summarization where older conversation blocks are replaced with concise summaries that explicitly extract key conclusions, decisions, and recurring themes, keeping recent exchanges verbatim.

Answer: D

Explanation:

A. Add structured XML tags to mark significant discussion conclusions throughout the conversation history. Incorrect.

Tagging helps organization, but it doesn't solve the **context length limitation** or ensure relevant retrieval at query time.

B. Use semantic embedding to index the full conversation history and retrieve only relevant past exchanges for each user query, replacing the linear conversation format with retrieved segments.
Incorrect.

Powerful for retrieval, but replacing the conversation entirely loses **narrative continuity**, which is important for ongoing discussions.

C. Implement rolling window truncation to keep only the most recent 25,000 tokens. Incorrect.

This discards earlier insights, which are critical for referencing past conclusions.

D. Implement progressive summarization where older conversation blocks are replaced with concise

summaries that explicitly extract key conclusions, decisions, and recurring themes, keeping recent exchanges verbatim. Correct.

This preserves **long-term context and key insights** while staying within token limits, enabling the assistant to reference past conclusions effectively.

Question: 45 **Exam Heist**

After a 40-minute session helping plan a dinner party, the conversation has grown to 78,000 tokens. The history includes: (1) the user mentioning a guest has a severe shellfish allergy, (2) measurements for scaling recipes to 8 servings, (3) the user's clarification that "room temperature butter" means 68°F in their kitchen, and (4) general back-and-forth about meal timing and presentation. You need to implement context management before the window limit is reached. What approach best balances information preservation with token reduction?

A. Summarize the entire conversation history into a concise summary capturing main topics discussed, then append new messages going forward.

B. Implement a sliding window retaining only the most recent 20,000 tokens relying on users to re-state important information when relevant.

C. Store the full conversation externally and use semantic search to retrieve relevant portions for each turn, loading only matching segments into context.

D. Extract critical structured data (allergies, serving counts, user-defined terms) into a compact reference section, summarize general discussion, and retain recent exchanges verbatim.

Answer: D

Explanation:

A. Summarize the entire conversation history into a concise summary capturing main topics discussed, then append new messages going forward. Incorrect.

This risks losing **critical details** like allergies and precise measurements that must remain exact.

B. Implement a sliding window retaining only the most recent 20,000 tokens relying on users to re-state important information when relevant. Incorrect.

This discards essential context (e.g., allergy info), which is unsafe and unreliable.

C. Store the full conversation externally and use semantic search to retrieve relevant portions for each turn, loading only matching segments into context. Incorrect.

Helpful for retrieval, but may miss **important persistent constraints** (like allergies) if not explicitly retrieved every time.

D. Extract critical structured data (allergies, serving counts, user-defined terms) into a compact reference section, summarize general discussion, and retain recent exchanges verbatim. Correct.

This preserves **high-priority facts exactly**, reduces tokens efficiently, and maintains conversational continuity.

Question: 46 **Exam Heist**

You're Implementing a feature where users refine their playlist preferences through multiple conversation turns. After deploying, you notice Claude's responses don't reflect what us earlier in the same conversation — for example, a user says they love jazz, but two messages later Claude asks what genres they enjoy. What is the most

likely cause?

A. The model's context window has been exceeded by the conversation length
B. The Claude API requires a session_id parameter that you haven't configured
C. Claude requires a vector database connection to maintain conversation memory
D. Your application isn't including prior messages in the messages array

Answer: D

Explanation:

A. The model's context window has been exceeded by the conversation length Incorrect.

This would only happen in very long conversations, and the issue appears early within just a few turns.

B. The Claude API requires a session_id parameter that you haven't configured Incorrect.

There is no required session_id — context must be explicitly managed by the application.

C. Claude requires a vector database connection to maintain conversation memory Incorrect.

A vector database is optional for retrieval, not required for basic conversation memory.

D. Your application isn't including prior messages in the messages array Correct.

Claude does not retain memory between calls — if previous messages aren't included, it cannot recall earlier user preferences.

Question: 47 **Exam Heist**

Your fitness coaching assistant uses a system prompt with detailed conditional logic: "If the user mentions being a beginner, provide step-by-step form instructions. If they use term 'progressive overload' or 'superset', respond concisely. If they ask about injury history, always recommend consulting a physician." During evaluation, you find the assistant correct explicit expertise declarations but struggles when users don't clearly state their level-often defaulting to overly detailed responses regardless of contextual cues like technical te
Which change to the system prompt would most directly address this failure to pick up on implicit expertise signals?

A. Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional abou consultations.
B. Add more conditional branches to cover additional expertise signals, such as "If user mentions specific rep ranges or asks about periodization, treat as advanced."
C. Implement a pre-conversation intake that asks users to rate their experience level, then inject that rating into the system prompt as context for all subsequent responses.
D. Add an explicit instruction for the model to ask a clarifying question about experience level whenever the user's expertise isn't immediately clear from their first message.

Answer: A

Explanation:

A. Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional about consultations. Correct.

This shifts from brittle rule-based logic to a **flexible heuristic**, enabling the model to infer expertise from

subtle cues like terminology and tone.

B. Add more conditional branches to cover additional expertise signals, such as "If user mentions specific rep ranges or asks about periodization, treat as advanced." Incorrect.

This increases complexity and brittleness — edge cases will still be missed.

C. Implement a pre-conversation intake that asks users to rate their experience level, then inject that rating into the system prompt as context for all subsequent responses. Incorrect.

Useful, but doesn't solve the issue of **interpreting implicit signals during conversation**.

D. Add an explicit instruction for the model to ask a clarifying question about experience level whenever the user's expertise isn't immediately clear from their first message. Incorrect.

This adds friction and interrupts flow instead of improving implicit understanding.

Question: 48 **Exam Heist**

During initial testing, you notice that Claude doesn't seem to remember vocabulary words from earlier in the conversation. When a student asks "Can you quiz me on those words?", responds as if no words have been discussed. What is the most likely explanation?

A. Your system prompt needs explicit instructions telling Claude to remember information from earlier turns.

B. You're not including prior messages in each API request — the stateless API doesn't retain conversation history.

C. You need to enable conversation persistence by passing a session ID parameter with each API call.

D. The model's context window has filled up, causing earlier conversation content to be dropped.

Answer: B

Explanation:

A. Your system prompt needs explicit instructions telling Claude to remember information from earlier turns. Incorrect.

Instructions alone don't give the model memory — context must be provided with each request.

B. You're not including prior messages in each API request — the stateless API doesn't retain conversation history. Correct.

Claude is stateless. If earlier messages aren't passed in the request, it has **no awareness of prior vocabulary**.

C. You need to enable conversation persistence by passing a session ID parameter with each API call. Incorrect.

There's no required session ID — memory is handled by including past messages.

D. The model's context window has filled up, causing earlier conversation content to be dropped. Incorrect.

This would only happen in very long conversations, not typical early testing scenarios.

Question: 49 **Exam Heist**

Your home renovation planning assistant uses a system prompt defining an expert contractor persona with specific guidelines: always ask about budget, suggest alternatives at multiple price points, and confirm timeline requirements. During testing, responses follow these guidelines for turns 1-4, but by turn 7, the assistant gives generic advice without asking about budget or timeline. The conversation totals only 2,500 tokens. What is the most likely cause?

A. System prompts only establish initial behavior and don't persist across all turns.

B. The system prompt is only sent with the first API request.

C. The assistant's accumulated responses are diluting the system prompt's influence.

D. The model's attention on system prompt instructions naturally weakens as turns accumulate.

Answer: C

Explanation:

A. System prompts only establish initial behavior and don't persist across all turns. Incorrect.

System prompts do persist as long as they are included in each request.

B. The system prompt is only sent with the first API request. Incorrect.

If this were the case, the behavior would degrade immediately after the first turn — not gradually by turn 7.

C. The assistant's accumulated responses are diluting the system prompt's influence. Correct.

As the conversation grows, the model must balance more context. Even within a small token count, **recent conversational patterns can outweigh earlier instructions**, causing drift from the original system prompt.

D. The model's attention on system prompt instructions naturally weakens as turns accumulate. Incorrect.

This is too vague — it's not just "natural weakening," but specifically the **increasing influence of conversational context** relative to the system prompt.

Question: 50 **Exam Heist**

Users report that during extended conversations, the AI loses track of specific topics, examples, and preferences they mentioned earlier in the session. Your current implementation uses a sliding window that keeps only the most recent 25 message pairs to stay within context limits. What's the most effective approach to maintain awareness of earlier conversation content while managing context size?

A. Replace the sliding window with a hybrid approach: summarize older messages while keeping recent messages verbatim.

B. Implement vector similarity search over the full conversation history, retrieving relevant past messages for each user query.

C. Increase the window size to 50 message pairs to retain more conversation history before truncation.

D. Add a separate API call each turn to summarize messages being dropped, prepending this running summary to the conversation.

Answer: A

Explanation:

A. Replace the sliding window with a hybrid approach: summarize older messages while keeping recent

messages verbatim. Correct.

This preserves **long-term context in a compressed form** while keeping recent interactions intact, maintaining continuity without exceeding token limits.

B. Implement vector similarity search over the full conversation history, retrieving relevant past messages for each user query. Incorrect.

While powerful, this is more complex and better suited for **large-scale or cross-session retrieval**, not necessarily the most direct fix for in-session context loss.

C. Increase the window size to 50 message pairs to retain more conversation history before truncation. Incorrect.

This only delays the issue and increases token usage without fundamentally solving it.

D. Add a separate API call each turn to summarize messages being dropped, prepending this running summary to the conversation. Incorrect.

This can lead to **summary drift and compounding errors**, reducing reliability over time.

Question: 51 **Exam Heist**

Users frequently send ambiguous requests like "book a venue for the party" without specifying date, guest count, or budget. Your evaluation shows the assistant asks an average of 4 questions before taking any action, causing 35% of users to abandon mid-conversation. However, when you reduce questions, users sometimes receive recommendations that don't preferences. What's the most effective approach to improve this trade-off?

A. Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendation

B. Configure the assistant to proceed with reasonable defaults (medium sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to corrections if results don't match expectations

C. Instruct the assistant to state explicit assumptions based on conversation status proceed with recommendations while inviting corrections, and reserve clarifying questions only Irreversible actions like confirming bookings.

D. Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total

Answer: C

Explanation:

A. Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendation Incorrect.

This ensures completeness but adds **high friction**, increasing drop-off before any value is delivered.

B. Configure the assistant to proceed with reasonable defaults (medium sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to correct them if results don't match expectations Incorrect.

Hidden assumptions can lead to **misalignment and confusion**, reducing trust.

C. Instruct the assistant to state explicit assumptions based on conversation status, proceed with recommendations while inviting corrections, and reserve clarifying questions only for irreversible actions

like confirming bookings. Correct.

This balances **speed and accuracy** — users get immediate value, assumptions are transparent, and friction is minimized.

D. Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total Incorrect.

This reduces question count but still creates **front-loaded friction** without delivering immediate value.

Question: 52 **Exam Heist**

During QA testing, you notice that Claude follows your system prompt guidelines consistently in the first 10-15 turns, but by turn 25-30, responses begin deviating — using informal tone when formality was specified, occasionally skipping required formatting, or providing information types the guidelines restrict. Conversation length is well within context limits (typically 30,000 tokens out of 200,000 available). What's the most effective approach to maintain consistent behavior throughout extended conversations?

A. Insert user-role messages that reinforce critical guidelines at natural conversation breakpoints, especially before complex requests.

B. Implement post-response validation that regenerates each response until it conforms to the specified guidelines.

C. Automatically start a new conversation after 20 turns, passing a summary of the prior context to maintain continuity.

D. Move behavioral guidelines from the system prompt into the first user message.

Answer: A

Explanation:

A. Insert user-role messages that reinforce critical guidelines at natural conversation breakpoints, especially before complex requests. Correct.

This combats **instruction drift** by periodically refreshing key constraints within the active context, keeping behavior aligned over long conversations.

B. Implement post-response validation that regenerates each response until it conforms to the specified guidelines. Incorrect.

This is costly and reactive, and can create loops without addressing the root cause of drift.

C. Automatically start a new conversation after 20 turns, passing a summary of the prior context to maintain continuity. Incorrect.

Summarization can lose detail and doesn't directly reinforce behavioral rules.

D. Move behavioral guidelines from the system prompt into the first user message. Incorrect.

User messages carry less weight than system prompts and are more likely to be overridden during the conversation.

Question: 53 **Exam Heist**

Performance analysis reveals your context is composed of accumulated RAG results from all previous queries,

which is crowding out conversation history and causing coherence degradation after 15+ turns. Which approach best addresses this issue?

A. Implement semantic deduplication to identify and remove redundant information across the accumulated RAG results and conversation turns

B. Implement a sliding window for RAG results from the last 2-3 queries while preserving conversation history

C. Shift context budget to favor RAG results while reducing conversation history allocation

D. Compress all RAG results into a consolidated summary document that updates incrementally after each retrieval

Answer: B

Explanation:

A. Implement semantic deduplication to identify and remove redundant information across the accumulated RAG results and conversation turns Incorrect.

This reduces redundancy but doesn't solve the core issue of **unbounded accumulation** of RAG context.

B. Implement a sliding window for RAG results from the last 2–3 queries while preserving conversation history Correct.

This directly addresses context crowding by **limiting RAG growth** while keeping conversation continuity intact.

C. Shift context budget to favor RAG results while reducing conversation history allocation Incorrect.

This worsens coherence by sacrificing conversation context.

D. Compress all RAG results into a consolidated summary document that updates incrementally after each retrieval Incorrect.

This can introduce **information loss and summary drift**, especially across many turns.

Question: 54 **Exam Heist**

Your music discovery assistant should consistently maintain an enthusiastic tone, explain its reasoning for each recommendation, and ask clarifying questions to better understand user preferences. You want this behavior to persist reliably across all user interactions. Where should you define these behavioral guidelines?

A. In the first assistant message, instructing Claude to follow these guidelines going forward

B. Prepended to each user message before sending to the API

C. In the system prompt

D. In environmental variables that your application passes to the API client

Answer: C

Explanation:

A. In the first assistant message, instructing Claude to follow these guidelines going forward Incorrect.

Assistant messages don't reliably control future behavior and can be overridden by later context.

B. Prepended to each user message before sending to the API Incorrect.

User messages carry less authority than system-level instructions and are less reliable for enforcing behavior.

C. In the system prompt Correct.

The system prompt is the **highest-priority instruction layer**, making it the most reliable place to enforce consistent tone, reasoning style, and questioning behavior.

D. In environmental variables that your application passes to the API client Incorrect.

Environment variables are not part of the model's context and have no effect on behavior unless explicitly included in the prompt.

Question: 55 **Exam Heist**

During a conversation about order tracking, your external system receives a webhook indicating the user's package has shipped. The user is actively chatting and will likely send a follow-up message soon. You want the assistant to naturally incorporate this status change in its next response. What's the most effective approach?

A. Immediately send an API request with the update as a synthetic user message, generating an unsolicited assistant response.
B. Append the status update as a prefix to the next user message before calling the API.
C. Configure the assistant to call a get_order_status tool at the start of every response.
D. Add the current shipping status to the system prompt before the next API call.

Answer: D

Explanation:

A. Immediately send an API request with the update as a synthetic user message, generating an unsolicited assistant response. Incorrect.

This creates an **unsolicited interruption**, which can feel unnatural and confusing in an active conversation.

B. Append the status update as a prefix to the next user message before calling the API. Incorrect.

This pollutes the user message and mixes **system state with user intent**, which can lead to misinterpretation.

C. Configure the assistant to call a get_order_status tool at the start of every response. Incorrect.

This is inefficient and unnecessary, especially when you already have the update via webhook.

D. Add the current shipping status to the system prompt before the next API call. Correct.

This cleanly injects **up-to-date system state** into context, allowing the assistant to naturally incorporate it into the next response without disrupting the conversation flow.

Question: 56 **Exam Heist**

A new user's first message is "Set up my focus music," This could mean configure preferences, create a playlist, or play music immediately. Your system supports all three actions. What's the effective approach?

A. Create a new "Focus" playlist with curated tracks and notify the user it's ready.
B. Ask one clarifying question about action type: play now or configure for later
C. Play popular focus tracks Immediately and let the user redirect if needed

D. Start preference configuration by asking about genres, temps, and artists they prefer for focus.

Answer: B

Explanation:

A. Create a new "Focus" playlist with curated tracks and notify the user it's ready. Incorrect.

This assumes intent and may do the **wrong action**, frustrating users.

B. Ask one clarifying question about action type: play now or configure for later Correct.

This minimizes friction while resolving ambiguity, enabling the assistant to take the **right action quickly**.

C. Play popular focus tracks immediately and let the user redirect if needed Incorrect.

Acts prematurely and may not match user intent.

D. Start preference configuration by asking about genres, tempo, and artists Incorrect.

Too heavy upfront — adds unnecessary friction before confirming intent.

Question: 57 **Exam Heist**

Users report that responses feel repetitive across turns — each message begins with phrases like "Certainly!" or "I'd be happy to help!" even deep into conversations. You want responses to feel more natural, without these repetitive openers. What's the most effective approach?

A. Implement post-processing to detect and strip common greeting phrases from response beginnings

B. Add system prompt instructions specifying phrases to avoid, such as "Never begin responses with 'Certainly' or similar affirmations"

C. Lower the temperature parameter to make response openings more deterministic and less variable

D. Append a partial assistant message with a direct response opening that the model will continue from

Answer: D

Explanation:

D. Append a partial assistant message with a direct response opening that the model will continue from.

This technique, often referred to as "Pre-filling" or "Response Steering," is a powerful way to control the style and tone of an LLM. By providing the first few words of the assistant's response in the API call, you force the model to continue from that point, bypassing its internal "politeness" training or RLHF-induced habit of starting every turn with a greeting.

Question: 58 **Exam Heist**

Users frequently send ambiguous requests like "book a venue for the party" without specifying date, guest count, or budget. Your evaluation shows the assistant asks an average of 4 questions before taking any action, causing 35% of users to abandon mid-conversation. However, when you reduce questions, users sometimes receive recommendations that don't preferences. What's the most effective approach to improve this trade-off?

A. Implement a structured intake form that collects all required parameters (date, guest count, budget, venue

type) upfront before the assistant begins providing any recommendation

B. Configure the assistant to proceed with reasonable defaults (medium sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to corrections if results don't match expectations

C. Instruct the assistant to state explicit assumptions based on conversation status proceed with recommendations while inviting corrections, and reserve clarifying questions only Irreversible actions like confirming bookings.

D. Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total

Answer: C

Explanation:

A. Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendation Incorrect.

This ensures completeness but adds **high friction**, increasing drop-off before any value is delivered.

B. Configure the assistant to proceed with reasonable defaults (medium sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to correct them if results don't match expectations Incorrect.

Hidden assumptions can lead to **misalignment and confusion**, reducing trust.

C. Instruct the assistant to state explicit assumptions based on conversation status, proceed with recommendations while inviting corrections, and reserve clarifying questions only for irreversible actions like confirming bookings. Correct.

This balances **speed and accuracy** — users get immediate value, assumptions are transparent, and friction is minimized.

D. Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total Incorrect.

This reduces question count but still creates **front-loaded friction** without delivering immediate value.

Question: 59 **Exam Heist**

Your process refund tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (5% of calls), and business errors ("Order exceeds 3 day return window", "item already refunded") that are permanent (12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude.
What's the most effective way to reduce wasted retries while improving customer-facing response quality?

A. Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text.

B. Add a check refund eligibility tool that must be called before process refund to prevent business rule violations.

C. Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries.

D. Return structured error responses with retriable false for business errors and a customer-friendly explanation for Claude to use.

Answer: D

Explanation:

A. Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text. Incorrect.

Parsing plain text error messages is brittle and unreliable compared to explicit structured signals.

B. Add a check refund eligibility tool that must be called before process refund to prevent business rule violations. Incorrect.

This may reduce some failures, but business errors can still occur and it adds extra tool calls and latency.

C. Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries. Incorrect.

This improves handling of transient failures, but Claude may still retry permanent business errors because it lacks explicit retryability information.

D. Return structured error responses with retriable: false for business errors and a customer-friendly explanation for Claude to use. Correct.

Structured error metadata lets the agent reliably distinguish permanent vs. transient failures, preventing wasted retries while improving user-facing responses.

Question: 60 **Exam Heist**

Your agent has called lookup_order multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment inf outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the most effective approach before lookups?

A. Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues

B. Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details

C. Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions

D. Proceed with additional lookups without modifying the existing tool output context

Answer: B

Explanation:

A. Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues Incorrect.

This adds unnecessary infrastructure complexity for a short-lived conversational context problem.

B. Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details Correct.

This preserves the information needed for the current task while significantly reducing context bloat before additional lookups.

C. Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions Incorrect.

Natural language summaries are less compact and may omit structured details needed for accurate return handling.

D. Proceed with additional lookups without modifying the existing tool output context Incorrect.

This worsens context crowding and increases the risk of degraded performance.

Question: 61 **Exam Heist**

During a billing dispute resolution, your agent successfully retrieves customer info via get_customer and order details via lookup_order, but when attempting to call process_refund, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure. What approach best balances first-contact resolution with appropriate error handling?

A. Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically

B. Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later

C. Escalate immediately to a human agent since the refund action cannot be completed

D. Implement automatic retries with exponential backoff for process_refund, keeping the conversation open until the refund is successfully processed

Answer: B

Explanation:

A. Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically Incorrect.

The refund was not actually processed. Confirming success would be inaccurate and misleading.

B. Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later Correct.

This maximizes first-contact value by resolving what can be resolved, while transparently handling the backend failure.

C. Escalate immediately to a human agent since the refund action cannot be completed Incorrect.

Escalation may be unnecessary if the issue is temporary and the agent can still provide useful assistance.

D. Implement automatic retries with exponential backoff for process_refund, keeping the conversation open until the refund is successfully processed Incorrect.

Retries are useful for transient failures, but indefinitely blocking the conversation creates poor user experience and doesn't guarantee success.

Question: 62 **Exam Heist**

Your agent is handling a billing dispute. After calling get_customer and lookup_order, it identifies that the dispute involves a promotional pricing error requiring manager approval — beyond the agent's authorization level. How

should the workflow handle this mid-process escalation?

A. Persist the complete conversation and tool response history to a database, then call escalate_to_human with a reference ID.

B. Call escalate_to_human passing only the customer's original message.

C. Attempt the refund with process_refund anyway, escalating only if the system rejects the transaction.

D. Compile a structured handoff with customer details, order info, and the identified issue before calling escalate_to_human.

Answer: D

Explanation:

A. Persist the complete conversation and tool response history to a database, then call escalate_to_human with a reference ID. Incorrect.

Useful operationally, but the human agent still needs a concise, actionable summary rather than raw logs alone.

B. Call escalate_to_human passing only the customer's original message. Incorrect.

This loses the investigation work already completed and forces the human agent to repeat steps.

C. Attempt the refund with process_refund anyway, escalating only if the system rejects the transaction. Incorrect.

This violates authorization boundaries and business policy.

D. Compile a structured handoff with customer details, order info, and the identified issue before calling escalate_to_human. Correct.

A structured handoff preserves context, reduces repetition, and enables efficient continuation by the human agent.

Question: 63 **Exam Heist**

Production logs reveal inconsistent error handling: when lookup_order fails, the agent sometimes retries 5+ times (wasteful when the order ID doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses: "isError": true, "content": ["type": "text", "text": "Operation failed"] . The agent cannot distinguish between error types. What's the most effective improvement?

A. Enhance error responses with structured metadata: include errorCategory (transient/validation/permission), isRetryable boolean, and a description of what caused the failure.

B. Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.

C. Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.

D. Create an analyze_error MCP tool the agent calls after any failure to determine the error category and recommended action.

Answer: A

Explanation:

A. Enhance error responses with structured metadata: include errorCategory (transient/validation/permission), isRetryable boolean, and a description of what caused the failure. Correct.

Structured error metadata gives the agent clear signals for deciding whether to retry, escalate, or ask the user for clarification, leading to consistent and appropriate handling.

B. Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted. Incorrect.

Retrying all errors wastes resources and still doesn't help the agent distinguish permanent failures like validation or permission errors.

C. Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each. Incorrect.

Parsing generic text errors is brittle and unreliable compared to explicit structured metadata.

D. Create an analyze_error MCP tool the agent calls after any failure to determine the error category and recommended action. Incorrect.

This adds unnecessary complexity and extra tool calls when the information should already be present in the original error response.

Question: 64 **Exam Heist**

A customer raises three separate issues during one session: a refund inquiry (turns 1-15), a subscription question (turns 16-30), and a payment method update (turns 31-45). At turn 48, the customer asks "What happened with my refund?" The conversation is approaching context limits. What strategy best maintains the agent's ability to address all issues throughout the session?

A. Implement sliding window context that retains the most recent 30 turns.
B. Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.
C. Rely on MCP tools to re-fetch relevant information on demand when the customer references earller issues.
D. Summarize earlier turns into a narrative description, preserving full message history only for the active issue.

Answer: B

Explanation:

A. Implement sliding window context that retains the most recent 30 turns. Incorrect.

This would likely drop the earlier refund discussion entirely, preventing accurate follow-up handling.

B. Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer. Correct.

Structured persistence preserves critical state across multiple concurrent issues while minimizing context usage and enabling reliable recall later in the session.

C. Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues. Incorrect.

Tools can retrieve system data, but not the conversational context, decisions, or prior explanations from earlier turns.

D. Summarize earlier turns into a narrative description, preserving full message history only for the active issue. Incorrect.

Narrative summaries may omit important structured details needed to accurately resume earlier issues.

Question: 65 **Exam Heist**

Your update_user_profile tool accepts a user_id (required) and an optional fields_to_update object. In testing, Claude frequently omits user_id or passes incorrectly structured data. What is most critical for helping Claude understand what parameter values to provide?

A. Clear parameter descriptions explaining expected format, such as "user_id : UUID of the user to update (required)"

B. Verbose parameter names encoding format hints, such as user_id_string_uuid_format

C. Strict JSON Schema type constraints marking user_id as required and defining fields_to_update as an object type

D. Detailed error responses explaining why invalid parameter values were rejected

Answer: A

Explanation:

A. Clear parameter descriptions explaining expected format, such as "user_id: UUID of the user to update (required)" Correct.

Clear, human-readable descriptions are the most important guidance for helping Claude understand what values to provide and how parameters should be structured.

B. Verbose parameter names encoding format hints, such as user_id_string_uuid_format Incorrect.

Overly verbose names reduce readability and are less effective than proper descriptions.

C. Strict JSON Schema type constraints marking user_id as required and defining fields_to_update as an object type Incorrect.

Schema constraints help validation, but they don't sufficiently explain semantic expectations like the required UUID format.

D. Detailed error responses explaining why invalid parameter values were rejected Incorrect.

Helpful after failure, but not the most critical factor for preventing mistakes initially.

Question: 66 **Exam Heist**

Production monitoring shows your search_catalog tool fails 12% of the time: 8% are network timeouts that succeed when immediately retried, while 4% are query syntax errors from malformed user-provided filters that never succeed regardless of retry attempts. Currently, both error types are returned to the agent identically, causing it to waste turns retrying syntax errors and telling users to "try again later" for timeouts. How should you modify the tool's error handling?

A. Apply exponential backoff retry logic to all errors uniformly, returning a generic "service temporarily unavailable" message after max retries are exhausted.

B. Return all errors with a retryable boolean flag and error type details.

C. Implement automatic retry with backoff for network timeouts inside the tool; return syntax errors

immediately with parameter validation details.

D. Add few-shot examples to your system prompt demonstrating how to distinguish network errors from syntax errors and handle each case appropriately.

Answer: C

Explanation:

A. Apply exponential backoff retry logic to all errors uniformly, returning a generic "service temporarily unavailable" message after max retries are exhausted. Incorrect.

This wastes retries on **non-recoverable syntax errors** and hides useful diagnostic information from the agent.

B. Return all errors with a retryable boolean flag and error type details. Incorrect.

This improves clarity, but still pushes retry logic to the agent, which leads to inconsistent handling and wasted turns.

C. Implement automatic retry with backoff for network timeouts inside the tool; return syntax errors immediately with parameter validation details. Correct.

This cleanly separates responsibilities:

Tool handles **transient failures (timeouts)**
Tool immediately exposes **non-retryable syntax errors with actionable feedback**
This prevents wasted retries and improves user-facing behavior.

D. Add few-shot examples to your system prompt demonstrating how to distinguish network errors from syntax errors and handle each case appropriately. Incorrect.

Prompt-based reasoning is unreliable for error handling compared to **explicit tool-level logic separation**.

Question: 67 **Exam Heist**

Users frequently send ambiguous requests like "book a venue for the party" without specifying date, guest count, or budget. Your evaluation shows the assistant asks an average of 4.2 clarifying questions before taking any action, causing 35% of users to abandon mid-conversation. However, when you reduce questions, users sometimes receive recommendations that don't match their preferences. What's the most effective approach to improve this trade-off?

A. Instruct the assistant to state explicit assumptions based on conversation history, proceed with recommendations while inviting corrections, and reserve clarifying questions only for irreversible actions like confirming bookings.

B. Configure the assistant to proceed with reasonable defaults (medium-sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to provide corrections if results don't match expectations.

C. Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendations.

D. Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total number of conversational turns.

Answer: D

Explanation:

A. Instruct the assistant to state explicit assumptions based on conversation history, proceed with recommendations while inviting corrections, and reserve clarifying questions only for irreversible actions like confirming bookings. Incorrect.

This improves transparency, but it still relies on assumptions that may not fully align with user intent, leading to partial mismatches.

B. Configure the assistant to proceed with reasonable defaults (medium-sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to provide corrections if results don't match expectations. Incorrect.

Hidden defaults can reduce trust and lead to user confusion when outputs don't match expectations.

C. Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendations. Incorrect.

This ensures accuracy but introduces too much friction early in the flow, increasing drop-off.

D. Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total number of conversational turns. Correct.

This reduces user fatigue while still explicitly capturing required constraints, improving both completion rates and recommendation accuracy without over-questioning.

Question: 68 **Exam Heist**

Your document extraction tool uses ML models to extract invoice fields (vendor, amount, date). The models return confidence scores (0.0-1.0) for each extracted field. In production, you observe:
(1) the agent proceeds with low-confidence extractions that are incorrect 23% of the time, and (2) the agent requests unnecessary human review for 31% of extractions that were actually correct. How should you restructure the tool's output?

A. Return fields with their raw confidence scores and add detailed few-shot examples to your system prompt demonstrating how to interpret different confidence ranges and when to request human review.

B. Compute an aggregate extraction quality score across all fields and return it alongside the extracted values. Include a text summary describing the overall extraction reliability.

C. Return fields with confidence scores, plus a requires_review boolean computed using your tested confidence thresholds, along with a review_reasons array explaining which fields triggered review.

D. Return fields organized into verified and needs_verification objects based on confidence thresholds.

Answer: C

Explanation:

A. Return fields with their raw confidence scores and add detailed few-shot examples to your system prompt demonstrating how to interpret different confidence ranges and when to request human review. Incorrect.

This still leaves the decision logic ambiguous and pushes critical operational behavior into prompt interpretation, which is inconsistent and hard to control in production.

B. Compute an aggregate extraction quality score across all fields and return it alongside the extracted values. Include a text summary describing the overall extraction reliability. Incorrect.

A single aggregate score hides per-field uncertainty, which is exactly what is needed to avoid both false

approvals and unnecessary reviews.

C. Return fields with confidence scores, plus a requires_review boolean computed using your tested confidence thresholds, along with a review_reasons array explaining which fields triggered review. Correct.

This directly encodes decision logic into structured output, ensuring consistent behavior while still preserving explainability at the field level.

D. Return fields organized into verified and needs_verification objects based on confidence thresholds. Incorrect.

This improves structure, but it hides the underlying confidence signals and reduces flexibility for downstream systems that may need finer-grained control or auditability.

Question: 69 **Exam Heist**

Your agent includes an update_game_score tool that accepts game_date (string), home_team (string), and away_team (string) parameters. Production logs reveal recurring issues: the agent uses team nicknames instead of official names, applies inconsistent date formats, and selects the wrong game when teams have rematches in the same season. What tool interface change would effectively prevent these errors?

A. Add a season parameter to disambiguate rematches, and add a confirm_before_update flag that returns the resolved game details for the agent to verify before the score is committed.

B. Add detailed examples to the tool description showing the required date format and complete list of official team names.

C. Add enum constraints listing valid team names for both team parameters, and add a regex pattern enforcing ISO 8601 format for the date parameter.

D. Replace the three parameters with a single game_id parameter and a separate search_games lookup tool that returns matching game IDs.

Answer: D

Explanation:

A. Add a season parameter to disambiguate rematches, and add a confirm_before_update flag that returns the resolved game details for the agent to verify before the score is committed. Incorrect.

This improves safety via confirmation, but it still leaves ambiguity in identifying the correct game as a core issue.

B. Add detailed examples to the tool description showing the required date format and complete list of official team names. Incorrect.

Examples help, but they don't reliably prevent structural errors like wrong team identity or ambiguous match selection.

C. Add enum constraints listing valid team names for both team parameters, and add a regex pattern enforcing ISO 8601 format for the date parameter. Incorrect.

This improves input validation, but it still does not solve the deeper problem of **disambiguating multiple games between the same teams**.

D. Replace the three parameters with a single game_id parameter and a separate search_games lookup tool that returns matching game IDs. Correct.

This removes ambiguity entirely by forcing the agent to resolve the exact entity first, then operate on a unique identifier, eliminating nickname errors, date mismatches, and rematch confusion.

Question: 70 **Exam Heist**

Your resource allocation tool returns a simple acknowledgment message after provisioning is requested. Users frequently approve allocations and immediately ask "how much did that cost?" or "which project was that?" - indicating they confirmed without understanding the request. What tool design change would most effectively address this?

A. Add a user_acknowledged: boolean parameter that must be set true, with instructions for the agent to only set it after the user explicitly confirms they reviewed the details

B. Implement a 60-second hold before execution completes, allowing users time to review pending allocations and cancel if needed

C. Add a detail_level parameter with options "minimal" or "comprehensive" that controls how much context the agent presents in confirmations

D. Return structured data including cost estimate, target project, resource specifications, and impact summary in the tool response

Answer: D

Explanation:

A. Add a user_acknowledged: boolean parameter that must be set true, with instructions for the agent to only set it after the user explicitly confirms they reviewed the details Incorrect.

This relies on agent discipline and is easy to bypass or mis-handle; it doesn't guarantee users actually see or understand key details.

B. Implement a 60-second hold before execution completes, allowing users time to review pending allocations and cancel if needed Incorrect.

Delays alone don't improve understanding; they only slow execution and may frustrate users without ensuring informed approval.

C. Add a detail_level parameter with options "minimal" or "comprehensive" that controls how much context the agent presents in confirmations Incorrect.

This improves formatting flexibility but does not ensure critical cost and impact information is consistently surfaced before approval.

D. Return structured data including cost estimate, target project, resource specifications, and impact summary in the tool response Correct.

This ensures that every allocation includes explicit, structured, reviewable details, reducing uninformed approvals and enabling users to understand cost and impact before proceeding.

Question: 71 **Exam Heist**

Your conversation history includes two types of content: persistent story elements (character backgrounds, plot structure, world rules) that must remain consistent throughout, and extensive brainstorming discussion that's mostly ephemeral. After 40+ turns, you're hitting context limits and users report the assistant "forgets" established character traits, breaking narrative consistency.
Which approach best ensures persistent story elements remain available to the model while reclaiming context

space?

A. Separate persistent story elements into a retained "story bible" section at context start, applying trimming or summarization only to brainstorming discussion.

B. Store all history in a vector database and retrieve semantically similar passages for each new message, replacing conversation history with retrieved chunks.

C. Apply a sliding-window approach keeping only the most recent 25 turns, relying on the model to infer earlier context from recent discussion flow.

D. Summarize the entire conversation history into a condensed synopsis every 20 turns, replacing the full history to free up tokens.

Answer: A

Explanation:

A. Separate persistent story elements into a retained "story bible" section at context start, applying trimming or summarization only to brainstorming discussion. Correct.

This directly solves the problem by preserving **high-value, invariant narrative constraints (characters, rules, canon)** while allowing lower-value conversational content to be compressed. It maintains continuity without losing essential story logic.

B. Store all history in a vector database and retrieve semantically similar passages for each new message, replacing conversation history with retrieved chunks. Incorrect.

This risks missing globally important story rules because retrieval is similarity-based, not constraint-aware. Core canon details may not always be retrieved.

C. Apply a sliding-window approach keeping only the most recent 25 turns, relying on the model to infer earlier context from recent discussion flow. Incorrect.

This will almost certainly break long-range consistency, since key character and world rules will be dropped.

D. Summarize the entire conversation history into a condensed synopsis every 20 turns, replacing the full history to free up tokens. Incorrect.

General summarization tends to lose fine-grained canonical details, which are exactly what must remain stable for narrative consistency.

Question: 72 **Exam Heist**

Your conversational assistant frequently generates multiple clarifying questions when users make ambiguous requests. When a user asks "Can you help me with the report?", the assistant responds: "I'd be happy to help! Could you tell me: 1) Which report? 2) What kind of help—drafting, reviewing, or formatting? 3) What's your deadline?" User analytics show a 40% conversation abandonment rate after these multi-question responses. What's the most effective way to reduce friction while appropriately handling ambiguity?

A. Limit the assistant to one clarifying question per turn, using conversation history to accumulate answers over multiple exchanges rather than requesting everything upfront.

B. Modify the system prompt to instruct the assistant to make reasonable assumptions from available context, state those assumptions explicitly, and offer to adjust if the interpretation is wrong.

C. Add a preprocessing step using a smaller model to classify request ambiguity on a 1-5 scale, routing high-ambiguity requests to a clarification dialog and low-ambiguity requests directly to the assistant.

D. Create a lookup table of common request patterns with predefined default interpretations, having the assistant respond with those defaults without stating the assumptions made.

Answer: B

Explanation:

A. Limit the assistant to one clarifying question per turn, using conversation history to accumulate answers over multiple exchanges rather than requesting everything upfront. Incorrect.

This reduces immediate friction, but it can make resolution slower and still prolong ambiguity across multiple turns, potentially frustrating users who want faster outcomes.

B. Modify the system prompt to instruct the assistant to make reasonable assumptions from available context, state those assumptions explicitly, and offer to adjust if the interpretation is wrong. Correct.

This best balances usability and accuracy: the assistant provides immediate value, avoids overload of questions, and maintains transparency so users can correct assumptions quickly if needed.

C. Add a preprocessing step using a smaller model to classify request ambiguity on a 1-5 scale, routing high-ambiguity requests to a clarification dialog and low-ambiguity requests directly to the assistant. Incorrect.

This adds system complexity and still risks unnecessary gating or misclassification, increasing friction or incorrect routing.

D. Create a lookup table of common request patterns with predefined default interpretations, having the assistant respond with those defaults without stating the assumptions made. Incorrect.

Hidden assumptions reduce trust and can lead to incorrect outputs without user awareness or correction.

Question: 73 **Exam Heist**

Users frequently refine their search criteria mid-conversation. You notice a pattern: when users say things like "Actually, let's raise the budget to $650K" or "I'd prefer a condo now instead of a house," the assistant sometimes continues referencing the original preferences in later responses — even though the updates are clearly present in the conversation history. Context usage is only at 35% capacity. Which solution most reliably ensures the model uses the current preferences?

A. Maintain a structured state object with current preferences, update it on changes, and include it in each request.

B. Implement conversation pruning to remove turns containing outdated preferences, ensuring only current ones remain in context.

C. Include few-shot examples showing the assistant correctly acknowledging and applying preference changes in responses.

D. Add system prompt instructions emphasizing that the model should always prioritize the most recently stated preferences over earlier ones.

Answer: A

Explanation:

A. Maintain a structured state object with current preferences, update it on changes, and include it in each request. Correct.

This is the most reliable solution because it externalizes “truth” into a **single authoritative state**, preventing the model from having to infer which preference is current across many turns.

B. Implement conversation pruning to remove turns containing outdated preferences, ensuring only current

ones remain in context. Incorrect.

Pruning is risky because earlier messages may still contain important constraints or rationale; removing them can break continuity and lead to unintended loss of context.

C. Include few-shot examples showing the assistant correctly acknowledging and applying preference changes in responses. Incorrect.

Examples help behavior, but they do not guarantee consistent override of earlier conflicting context in long conversations.

D. Add system prompt instructions emphasizing that the model should always prioritize the most recently stated preferences over earlier ones. Incorrect.

This is helpful but not reliable enough on its own — models can still misinterpret or fail to consistently resolve conflicts without a structured state.

Question: 74 **Exam Heist**

Your conversational AI tutor has a 2,800-token system prompt containing teaching methodology, persona guidelines, and detailed written instructions for adapting explanations to different proficiency levels. User testing reveals that in conversations exceeding 12 turns (approximately 4,000 tokens of conversation history), the assistant increasingly ignores the proficiency-adaptation guidelines, defaulting to intermediate-level explanations regardless of the learner's stated level. What's the most effective approach to ensure consistent adherence to these guidelines throughout extended conversations?

A. Inject a condensed reminder of the proficiency requirements into the conversation as a system message every 4-5 turns.

B. Replace the verbose proficiency guidelines with few-shot examples demonstrating appropriate responses at each proficiency level, showing concrete differences in vocabulary, complexity, and explanation depth.

C. Restructure the system prompt to place the proficiency-adaptation rules in a clearly-marked final section immediately before the conversation history begins.

D. After each assistant response, make a separate API call to evaluate whether the difficulty level matched the learner's profile, regenerating responses that don't align.

Answer: B

Explanation:

A. Inject a condensed reminder of the proficiency requirements into the conversation as a system message every 4–5 turns. Incorrect.

This is a mitigation strategy, but it’s operationally inefficient and still treats the symptom (drift over long context) rather than improving how the model reliably learns and applies the rule in the first place.

B. Replace the verbose proficiency guidelines with few-shot examples demonstrating appropriate responses at each proficiency level, showing concrete differences in vocabulary, complexity, and explanation depth. Correct.

This is the most effective fix because few-shot examples strongly anchor behavior. Instead of relying on long, abstract instructions that degrade in influence over long contexts, the model gets **clear behavioral patterns it can consistently imitate**, improving adherence across extended conversations.

C. Restructure the system prompt to place the proficiency-adaptation rules in a clearly-marked final section immediately before the conversation history begins. Incorrect.

This can slightly improve recall but does not address the core weakness of long, abstract rule blocks losing effectiveness over time.

D. After each assistant response, make a separate API call to evaluate whether the difficulty level matched the learner's profile, regenerating responses that don't align. Incorrect.

This is expensive, slow, and unnecessary when prompt design improvements can solve the issue.

Question: 75 **Exam Heist**

After deploying an updated system prompt that improves response quality, users with multi-session conversations spanning several weeks report that the assistant now contradicts its earlier statements and has a noticeably different communication style. New users don't experience these issues.
What's the best approach to resolve this?

A. Regenerate summaries of existing conversations using the new prompt and replace the stored histories to align past context with current behavior.

B. Add a transition message when sessions resume explaining that the assistant has been updated and behavior may differ.

C. Version system prompts and associate each conversation with the prompt version under which it started, applying updates only to new conversations.

D. Add instructions to the new system prompt directing the assistant to maintain consistency with any prior statements in the conversation history.

Answer: C

Explanation:

A. Regenerate summaries of existing conversations using the new prompt and replace the stored histories to align past context with current behavior. Incorrect.

This rewrites history and can distort or erase what users actually said, creating continuity risks and potential inaccuracies.

B. Add a transition message when sessions resume explaining that the assistant has been updated and behavior may differ. Incorrect.

This improves transparency but does not solve the core problem of inconsistent behavior across sessions.

C. Version system prompts and associate each conversation with the prompt version under which it started, applying updates only to new conversations. Correct.

This is the most robust solution because it preserves **behavioral consistency within a conversation over time**, avoiding contradictions caused by mid-stream prompt changes.

D. Add instructions to the new system prompt directing the assistant to maintain consistency with any prior statements in the conversation history. Incorrect.

This helps slightly, but it cannot reliably override changes in underlying behavior introduced by prompt updates.

Question: 76 **Exam Heist**

The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate

descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate — it generates messages like "I'll ask the web search agent to find sources on this topic" — but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

A. The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.

B. The coordinator's allowedTools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.

C. Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in ClaudeAgentOptions.

D. The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked.

Answer: B

Explanation:

A. The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified. Incorrect.

If this were the cause, you'd typically see partial tool calls or parsing errors in logs — not silent absence of tool execution.

B. The coordinator's allowedTools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents. Correct.

This exactly matches the symptom: the model plans to delegate (reasoning is fine), but **cannot execute the tool call**, so no subagent runs and no errors appear at the reasoning level.

C. Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in ClaudeAgentOptions. Incorrect.

This would affect what subagents receive, not whether they are spawned at all.

D. The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked. Incorrect.

The model is clearly aware of delegation (it verbalizes it), so discovery is not the issue — execution capability is.

Key idea: reasoning about tools ≠ permission to use tools; missing tool authorization leads to silent non-execution.

Question: 77 **Exam Heist**

Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing nuance), while other times reports over-hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "industry analysts estimate $50B market size (methodology varies)" and the document analysis agent returns "peer-reviewed study estimates $35B (±$7B, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the market may be $35B-$50B depending on factors." What systematic approach best addresses this?

A. Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calibrated confidence.

B. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterizations and methodological context.

C. Configure subagents to only report findings meeting a high-confidence threshold, filtering uncertain information before it reaches the coordinator.

D. Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.

Answer: B

Explanation:

A. Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calibrated confidence. Incorrect.

This forces incompatible uncertainty types (market estimates, confidence intervals, analyst opinions) into a single numeric scale, which can distort meaning and oversimplify fundamentally different methodologies.

B. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterizations and methodological context. Correct.

This directly addresses the root problem: loss of nuance. It preserves **multiple viewpoints, uncertainty types, and methodological differences**, while still producing a coherent, structured report instead of collapsing everything into a misleading single value or vague range.

C. Configure subagents to only report findings meeting a high-confidence threshold, filtering uncertain information before it reaches the coordinator. Incorrect.

This removes the very uncertainty signals needed for proper synthesis and biases outputs toward false certainty.

D. Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources. Incorrect.

This is too restrictive for domains like market sizing where disagreement is meaningful and expected, not a sign of invalid data.

Question: 78 **Exam Heist**

You've configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization — the synthesis agent attempts web searches, and the report generator tries to analyze documents. What is the primary cause of this poor tool selection behavior?

A. Choosing from 18 tools instead of 4-5 relevant ones increases decision complexity beyond reliable selection thresholds.

B. The agents' role descriptions in their system prompts conflict with having access to tools outside that role.

C. The tool definitions consume too much context window space, leaving insufficient room for task content.

D. The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks.

Answer: A

Explanation:

A. Choosing from 18 tools instead of 4–5 relevant ones increases decision complexity beyond reliable selection thresholds. Correct.

This is the core issue: as the number of available tools grows, especially across multiple unrelated domains, the model's **tool selection accuracy degrades due to increased choice entropy and decision load**. Even if each tool is well-described, having too many options in the same context makes it harder for the model to reliably pick the correct one, leading to cross-role tool misuse.

Why the others are not correct:

B. Role-description conflict Incorrect

Role prompts influence behavior, but they don't inherently break tool selection when tools are available.

C. Context window usage Incorrect

There's no indication of truncation or missing tool definitions — this is a selection problem, not a capacity problem.

D. Coordinator tracking issue Incorrect

The failure happens at the subagent decision level, not because the coordinator lacks awareness of tools.

Question: 79 **Exam Heist**

The coordinator provides detailed step-by-step instructions to the web search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports "insufficient results" rather than trying alternative approaches when pre-specified searches fail, (2) research quality drops for emerging topics that don't match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources. What's the most effective way to improve subagent adaptability?

A. Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.

B. Add explicit fallback directives to the detailed instructions: "If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure."

C. Remove procedural details entirely, delegating with simple goals like "research X thoroughly" and relying on the subagent's general capabilities.

D. Implement a topic classification step where the coordinator categorizes requests as "well-defined" or "exploratory" and uses different instruction styles for each category.

Answer: A

Explanation:

A. Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy. Correct.

This is the most effective fix because it shifts from **over-constrained instructions (procedural control)** to **objective-driven autonomy**, allowing the subagent to adapt when initial queries fail and to explore tangential but relevant sources.

B. Add explicit fallback directives to the detailed instructions: "If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure." Incorrect.

This helps slightly, but it still locks behavior into predefined contingencies and doesn't generalize to novel or emerging topics.

C. Remove procedural details entirely, delegating with simple goals like "research X thoroughly" and relying on the subagent's general capabilities. Incorrect.

Too vague — removing structure entirely reduces reliability and makes output quality inconsistent.

D. Implement a topic classification step where the coordinator categorizes requests as "well-defined" or "exploratory" and uses different instruction styles for each category. Incorrect.

This adds complexity and still relies on rigid instruction modes rather than enabling adaptive search behavior.

Question: 80 **Exam Heist**

The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations — the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps. What's the most effective approach to ensure proper source attribution in the final reports?

A. Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).

B. Have the report generator query the web search agent to re-locate sources for claims in the final report.

C. Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator.

D. Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format.

Answer: A

Explanation:

A. Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers). Correct.

This is the most reliable solution because it preserves **provenance as first-class data**, ensuring citations are never lost during summarization. The report generator can then deterministically map claims back to their original sources.

B. Have the report generator query the web search agent to re-locate sources for claims in the final report. Incorrect.

This is inefficient and unreliable because it requires reverse-matching claims to sources, which can be ambiguous or fail entirely.

C. Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator. Incorrect.

This avoids the problem but is not scalable and leads to context bloat and degraded performance.

D. Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format. Incorrect.

This helps, but it still risks losing or breaking attribution during downstream transformations and does not enforce structured traceability.

Question: 81 **Exam Heist**

A user is expanding the research system beyond its single web search agent by adding specialized data sources.

They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

A. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose

B. Standardize all subagent outputs to JSON with fields for claim, evidence, source, and confidence

C. Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation

D. Standardize all subagent outputs to prose summaries with inline citations

Answer: A

Explanation:

A. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose Correct.

This works because it preserves the **native structure of each data type at the point where it matters most (final synthesis)**. Financial data remains comparable in tabular form, news retains narrative coherence, and patents stay structured as categorized lists. The key improvement is that the synthesis layer becomes **format-aware instead of flattening everything into bullets**, which is exactly what was causing quality loss.

Why the others are not correct:

B. Standardize all subagent outputs to JSON with fields for claim, evidence, source, and confidence Incorrect

This forces unrelated data types into a single schema and strips meaningful presentation structure like tables and narratives.

C. Add a format conversion layer between subagents and synthesis Incorrect

This risks over-normalization into a lowest-common-denominator format, reducing expressiveness.

D. Standardize all subagent outputs to prose summaries with inline citations Incorrect

This removes structure from financial and analytical data, reducing precision and comparability.

Question: 82 **Exam Heist**

When the agent calls lookup order and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call process refund escalate to human next?

A. The agent executes the remaining steps in a tool sequence planned at the start of the request.

B. The order details are added to the conversation and the model reasons about which action to take.

C. The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.

D. The orchestration layer automatically routes to the next tool based on the order's status field.

Answer: B

Explanation:

B. The order details are added to the conversation and the model reasons about which action to take. Correct.

In an agentic loop, tool results (like “purchased 45 days ago”) are fed back into the model’s context, and the model **re-evaluates the situation dynamically**. It then decides whether to proceed with process_refund, escalate to a human, or take another action based on policy and the updated information.

Why the others are not correct:

A. Fixed tool sequence planned at the start Incorrect

Agentic systems are not static workflows; they adapt after each observation.

C. Pre-configured decision tree Incorrect

This is rule-based automation, not LLM-driven reasoning.

D. Orchestration layer automatically routes next tool call Incorrect

That removes the model’s reasoning role and turns it into deterministic routing.

Question: 83 **Exam Heist**

The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened. What's the most likely cause of this behavior?

A. Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.
B. The prompt lacks instructions telling Claude to remember information across multiple exchanges.
C. The verification tool is clearing the agent's internal state after each successful validation step.
D. The conversation history isn't being passed in subsequent API requests.

Answer: D

Explanation:

A. Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it. Incorrect.

There is no fixed “two-turn memory limit” like this; context is determined by what the application sends, not a built-in short memory window.

B. The prompt lacks instructions telling Claude to remember information across multiple exchanges. Incorrect.

System prompts don’t control memory persistence. The model does not “forget” within context unless information is missing from input.

C. The verification tool is clearing the agent's internal state after each successful validation step. Incorrect.

Tools do not automatically reset conversational context unless explicitly designed to do so — and that would be an application-level bug, not the most likely general cause.

D. The conversation history isn't being passed in subsequent API requests. Correct.

This is the most likely cause. The model is stateless, so if prior messages aren't included in each request, it behaves as if earlier verification steps never happened — leading to repeated questions like asking for the name again.

Question: 84 **Exam Heist**

When implementing your lookup_order MCP tool, the backend sometimes returns errors (e.g., "Order not found" or temporary database failures). What is the correct pattern for communicating these errors back to the agent?

A. Log the error server-side and return an empty result to avoid confusing the model

B. Throw an exception from the tool handler so the agent framework can catch and log it

C. Return the error message in the tool result content with the isError flag set to true

D. Return a success response with a "status" field indicating the error type

Answer: C

Explanation:

A. Log the error server-side and return an empty result to avoid confusing the model Incorrect.

This hides critical failure information, making it impossible for the agent to distinguish "no data" from "system failure."

B. Throw an exception from the tool handler so the agent framework can catch and log it Incorrect.

Exceptions are useful internally, but the agent still needs a structured tool response; raw exceptions don't reliably propagate useful context to the model.

C. Return the error message in the tool result content with the isError flag set to true Correct.

This is the proper MCP pattern: the tool explicitly signals failure using isError: true, while still providing a readable error message so the agent can decide whether to retry, escalate, or inform the user.

D. Return a success response with a "status" field indicating the error type Incorrect.

This is misleading because it treats failures as successful responses, which confuses downstream reasoning and tool orchestration.

Question: 85 **Exam Heist**

Evaluation shows 94% extraction accuracy on short meeting transcripts (<30 minutes) but only 68% on longer transcripts (>60 minutes) where discussions meander and key information is scattered throughout. Transcripts of both lengths fit within the model's context window. What pattern most effectively improves accuracy on complex, lengthy documents?

A. Add few-shot examples demonstrating correct extraction from lengthy meetings with scattered Information.

B. Split lengthy transcripts Into chunks, extract from each chunk separately, then merge and deduplicate the results.

C. Upgrade to a more capable model tier for the extraction task,

D. Add a pre-extraction step where the model summarizes key discussions and conclusions before performing structured extraction.

Answer: B

Explanation:

A. Add few-shot examples demonstrating correct extraction from lengthy meetings with scattered information. Incorrect.

Examples help somewhat, but they do not fundamentally solve the challenge of maintaining attention and consistency across long, meandering documents.

B. Split lengthy transcripts into chunks, extract from each chunk separately, then merge and deduplicate the results. Correct.

Chunking reduces cognitive load per extraction pass and improves recall of scattered details. Merging and deduplication then reconstruct the final structured output more reliably than processing the entire long transcript in one pass.

C. Upgrade to a more capable model tier for the extraction task. Incorrect.

A stronger model may help marginally, but the core issue is long-document attention and information distribution, which chunking addresses more directly and efficiently.

D. Add a pre-extraction step where the model summarizes key discussions and conclusions before performing structured extraction. Incorrect.

Summarization can omit details needed for extraction, especially when important information is scattered or mentioned briefly.

Question: 86 **Exam Heist**

Your system has been running for 3 weeks and human reviewers have corrected 847 extractions. Analysis reveals a recurring pattern: when recipes use informal measurements like "a handful" or "a splash," the model either invents specific amounts or leaves fields empty — accounting for 23% of all corrections. How should you use this feedback to improve extraction accuracy?

A. Update your JSON schema to add a "measurement_type" enum field (precise/informal).

B. Add few-shot examples to your prompt demonstrating correct handling of informal measurements — extracting them verbatim rather than converting or omitting them.

C. Implement a post-processing layer that uses pattern matching to detect informal measurement phrases in source text and automatically populate values when the extraction is empty.

D. Fine-tune the model on the 847 corrected extractions.

Answer: B

Explanation:

A. Update your JSON schema to add a "measurement_type" enum field (precise/informal). Incorrect.

This may help classify outputs, but it does not teach the model how to correctly extract informal measurements instead of hallucinating or omitting them.

B. Add few-shot examples to your prompt demonstrating correct handling of informal measurements — extracting them verbatim rather than converting or omitting them. Correct.

This directly targets the observed failure mode by showing the model the desired behavior on real edge cases, improving consistency without forcing unsupported normalization.

C. Implement a post-processing layer that uses pattern matching to detect informal measurement phrases in source text and automatically populate values when the extraction is empty. Incorrect.

Pattern matching is brittle and does not address cases where the model invents incorrect quantities instead of leaving fields empty.

D. Fine-tune the model on the 847 corrected extractions. Incorrect.

Fine-tuning is expensive and unnecessary for a narrow, well-defined extraction issue that can be effectively corrected through targeted prompting examples.

Key idea: use real correction data to create **targeted few-shot examples that teach the desired extraction behavior on recurring edge cases.**

Question: 87 **Exam Heist**

Production monitoring shows that follow-up queries like "summarize what we learned about market trends" consistently take 40+ seconds. Investigation reveals the coordinator spawns the synthesis subagent for each summarization request, passing 80K+ tokens of accumulated findings. The coordinator already has these findings in its context from orchestrating the research. What's the most effective way to improve response time for these follow-up summaries?

A. Enable prompt caching on the synthesis subagent to reduce the overhead of repeatedly transferring the same research findings.

B. Have the coordinator handle straightforward summarization requests directly using its existing context, reserving subagent spawning for complex analy

C. Spawn the synthesis subagent with reduced context and have it request specific findings from the coordinator on-demand.

D. Pre-generate and cache summaries at multiple granularities whenever new findings accumulate.

Answer: B

Explanation:

A. Enable prompt caching on the synthesis subagent to reduce the overhead of repeatedly transferring the same research findings. Incorrect.

Caching helps with repeated prompt transfer costs, but it still requires spawning the subagent and processing a huge context for relatively simple follow-up summaries.

B. Have the coordinator handle straightforward summarization requests directly using its existing context, reserving subagent spawning for complex analysis tasks. Correct.

This avoids unnecessary orchestration overhead. Since the coordinator already possesses the accumulated findings in-context, simple summarization can be done immediately without rehydrating 80K+ tokens into another agent call.

C. Spawn the synthesis subagent with reduced context and have it request specific findings from the coordinator on-demand. Incorrect.

This adds coordination complexity and multiple back-and-forth exchanges, which can increase latency rather than reduce it.

D. Pre-generate and cache summaries at multiple granularities whenever new findings accumulate.
Incorrect.

Useful for some systems, but over-engineered here and inefficient if many summaries are never requested.

Question: 88 **Exam Heist**

The document analysis agent has a single analyze_document tool that takes a document and a free-text instruction parameter. During evaluation, requests like "extract the key financial metrics" often return narrative summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports that 35% of analysis results require re-requests with clarified instructions. What's the most effective way to improve reliability?

A. Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats

B. Keep the single tool but add an analysis_type enum parameter requiring explicit selection between extraction, summarization, and verification modes

C. Split the generic tool into purpose-specific tools - extract_data_points, summarize_content, verify_claim_against_source - each with defined input/output contracts

D. Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent

Answer: C

Explanation:

A. Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats Incorrect.

Examples help somewhat, but the core problem is that the tool itself is overloaded with multiple responsibilities and ambiguous behavior.

B. Keep the single tool but add an analysis_type enum parameter requiring explicit selection between extraction, summarization, and verification modes Incorrect.

This improves clarity, but the tool still mixes fundamentally different workflows and output contracts into one interface.

C. Split the generic tool into purpose-specific tools — extract_data_points, summarize_content, verify_claim_against_source — each with defined input/output contracts Correct.

This most effectively improves reliability because each tool has a **clear purpose, constrained behavior, and predictable output format**, dramatically reducing ambiguity and re-requests.

D. Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent Incorrect.

This shifts ambiguity upstream but does not solve the underlying problem of a single overloaded tool with inconsistent outputs.

Question: 89 **Exam Heist**

The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high-confidence extractions (>85%) also contain errors — cases where the model finds plausible-but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time. What approach is most effective?

A. Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score.

B. Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.

C. Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results.

D. Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review.

Answer: B

Explanation:

A. Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score. Incorrect.

This targets only currently known failure patterns and will miss new classes of high-confidence errors over time.

B. Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection. Correct.

This is the most sustainable and statistically sound approach. It both:

measures the true ongoing error rate in “trusted” outputs, and
surfaces emerging failure patterns that confidence scores miss.

It creates a continuous quality-monitoring feedback loop rather than patching isolated issues.

C. Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results. Incorrect.

Useful for detecting instability, but repeated extraction can still confidently reproduce the same wrong answer.

D. Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review. Incorrect.

This increases operational cost dramatically without specifically addressing hidden high-confidence errors.

Question: 90 **Exam Heist**

Your extraction uses tool use with a JSON schema where property_type is defined as an enum: ['house', 'apartment', "condo", "townhouse"]. After deployment, 8% of extractions fall schema validation. Investigation reveals listings mention many uncommon property types — "studio", "loft", "duplex", "mobile home", "tiny house", "converted warehouse" — and new types continue appearing regularly. What's the most effective long-term solution?

A. Add an "other" value to your enum with a separate property_type_detail string field for specifics when "other" is selected.

B. Change property_type from an enum to a free-form string and implement a normalization step in post-

processing.

C. Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value.

D. Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.

Answer: A

Explanation:

A. Add an "other" value to your enum with a separate property_type_detail string field for specifics when "other" is selected. Correct.

This is the most robust long-term solution because it preserves schema validity while still allowing the system to capture new or uncommon property types without breaking validation. It also keeps structured consistency for downstream systems.

B. Change property_type from an enum to a free-form string and implement a normalization step in post-processing. Incorrect.

This sacrifices schema control and pushes complexity downstream, increasing inconsistency and processing burden.

C. Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value. Incorrect.

This leads to forced misclassification and will not scale as new property types continue to appear.

D. Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases. Incorrect.

This is operationally expensive and does not scale in dynamic domains where new categories appear frequently.

Question: 91 **Exam Heist**

Your extraction system parses e-commerce product descriptions to extract specifications like dimensions, weight, and materials into JSON. Despite having a well-defined schema, the model inconsistently extracts the "materials" field—sometimes returning "cotton blend", other times "Cotton/Polyester mix", and occasionally omitting the field when material information is clearly present in the source. What's the most effective way to improve extraction consistency?

A. Set temperature to 0 to eliminate randomness and ensure deterministic outputs

B. Switch to a more capable model tier since inconsistent extraction indicates insufficient model capability

C. Make the "materials" field required instead of optional in the schema to force the model to always extract a value

D. Add few-shot examples showing 2-3 complete Input-output pairs with standardized material description formats

Answer: D

Explanation:

A. Set temperature to 0 to eliminate randomness and ensure deterministic outputs Incorrect.

Lower temperature improves consistency slightly, but it does not solve schema interpretation differences or missing-field behavior when the model is uncertain.

B. Switch to a more capable model tier since inconsistent extraction indicates insufficient model capability Incorrect.

This is not primarily a capability issue; the model already sees the information but is inconsistent in normalization and extraction behavior.

C. Make the "materials" field required instead of optional in the schema to force the model to always extract a value Incorrect.

Forcing requirement does not guarantee correctness; it often increases hallucination when the model tries to fill missing or ambiguous data.

D. Add few-shot examples showing 2–3 complete input-output pairs with standardized material description formats Correct.

This directly teaches the model the expected normalization pattern (e.g., “cotton blend” vs “cotton/polyester mix”) and when to consistently extract rather than omit, which is the core failure mode.

Question: 92 **Exam Heist**

Documents arrive continuously throughout business hours and need structured data extracted. To reduce costs, you want to use the Message Batches API (50% discount, up-to-24-hour processing window). Your SLA specifies that extraction results must be available within 30 hours of document arrival with 99.9% reliability. Which batching strategy is most appropriate?

A. Submit batches every 4 hours containing documents from that window
B. Submit batches every 6 hours containing documents from that window
C. Submit a single batch at end of day containing all documents from that day
D. Use the real-time API for all documents instead of batch processing

Answer: C

Explanation:

A. Submit batches every 4 hours containing documents from that window Incorrect

This is more frequent than necessary for a 30-hour SLA and reduces batching efficiency without improving reliability in a meaningful way.

B. Submit batches every 6 hours containing documents from that window Incorrect

Still unnecessarily frequent; it increases operational overhead without providing measurable SLA benefits compared to less frequent batching.

C. Submit a single batch at end of day containing all documents from that day Correct

This is the most efficient strategy. With a 24-hour batch processing window, end-of-day batching still comfortably meets the 30-hour SLA while maximizing cost savings and operational simplicity.

D. Use the real-time API for all documents instead of batch processing Incorrect

This guarantees speed but significantly increases cost and ignores the requirement to optimize for batch processing efficiency.

Question: 93 **Exam Heist**

Your extraction pipeline processes restaurant menus and must output structured JSON with fields for item names, descriptions, prices, and dietary tags. Some menus use inconsistent formatting — prices as "$12" vs "12.00", dietary info as icons vs text. What's the most reliable approach?

A. Extract data as-is and normalize formats in post-processing code after Claude returns.

B. Define a strict output schema and include format normalization rules in your prompt.

C. Use separate extraction calls for each field to ensure consistent handling of each type.

D. Request multiple extraction attempts per document and select the most common format.

Answer: B

Explanation:

B. Define a strict output schema and include format normalization rules in your prompt. Correct

This is the most reliable approach because it enforces structure through the schema while also guiding the model to normalize inconsistent inputs (like “$12” vs “12.00” and icon-based vs text-based dietary tags) at the point of extraction. That reduces downstream ambiguity and avoids relying on post-processing fixes.

Why the others are not correct:

A. Extract data as-is and normalize formats in post-processing code after Claude returns. Incorrect

This shifts complexity downstream and risks inconsistencies if extraction itself is already noisy or incomplete.

C. Use separate extraction calls for each field to ensure consistent handling of each type. Incorrect

This increases cost and latency and can introduce cross-field inconsistency instead of improving reliability.

D. Request multiple extraction attempts per document and select the most common format. Incorrect

Majority voting does not guarantee correctness and is inefficient for structured extraction tasks.

Question: 94 **Exam Heist**

Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article-for example, outputting "500" for attendee_count when the source contains no attendance information. What's the most effective way to reduce these false extractions?

A. Make all schema fleids required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.

B. Add prompt instructions to return null for any field where information is not directly stated in the source.

C. Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.

D. Add a post-processing step using a second LLM call to verify each extracted value exists in the source document.

Answer: B

Explanation:

A. Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data. Incorrect

Making fields required doesn't prevent hallucination; it often makes it worse because the model is forced to invent values instead of leaving them blank.

B. Add prompt instructions to return null for any field where information is not directly stated in the source. Correct

This directly targets the root failure mode (hallucinating absent fields). It enforces a clear extraction rule: only extract explicitly supported data, otherwise return null, which is the correct behavior for structured information extraction.

C. Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies. Incorrect

A better model may reduce errors slightly, but the fundamental issue is missing extraction constraints, not model capability.

D. Add a post-processing step using a second LLM call to verify each extracted value exists in the source document. Incorrect

This adds cost and latency and still relies on probabilistic verification rather than preventing hallucination at the source.

Question: 95 **Exam Heist**

Your research assistant helps users analyze academic papers over extended conversations. User testing reveals a recurring issue: after conversations exceed 60K tokens, users ask follow-up questions requiring precise numerical details from papers discussed earlier — sample sizes, exact p-values, specific inclusion criteria. Your current approach summarizes paper discussions after 8 turns to stay within context limits. Users report that responses to these precision-dependent questions are often hedged or inaccurate. What's the most effective architectural change?

A. Implement retrieval that re-injects relevant paper sections when the user's question suggests they need specific numerical details.

B. Maintain a structured database of key facts extracted from each paper (sample sizes, statistics, methods) and retrieve relevant entries into context when precision-dependent questions are detected.

C. Keep source text from methodology and results sections in context permanently, while summarizing only the conversational discussion and interpretation portions.

D. Use a separate Claude call with explicit instructions to generate higher-fidelity summaries that preserve all numerical details and statistical values.

Answer: B

Explanation:

A. Implement retrieval that re-injects relevant paper sections when the user's question suggests they need specific numerical details. Incorrect

This helps, but it is reactive and depends on correct detection at query time; it doesn't ensure consistent availability of critical structured facts across the session.

B. Maintain a structured database of key facts extracted from each paper (sample sizes, statistics,

methods) and retrieve relevant entries into context when precision-dependent questions are detected. Correct

This is the most effective architectural change. It separates **stable factual data (structured facts store)** from conversational history, ensuring that precise values like p-values and sample sizes remain accessible even after summarization or long-context drift.

C. Keep source text from methodology and results sections in context permanently, while summarizing only the conversational discussion and interpretation portions. Incorrect

This is inefficient and quickly exhausts context window limits, especially across many papers.

D. Use a separate Claude call with explicit instructions to generate higher-fidelity summaries that preserve all numerical details and statistical values. Incorrect

Even high-fidelity summaries can introduce omission or distortion; it is not as reliable as structured fact extraction and storage.

Question: 96 **Exam Heist**

Your fitness coaching assistant uses a system prompt with detailed conditional logic: "If the user mentions being a beginner, provide step-by-step form instructions. If they use terms like 'progressive overload' or 'superset', respond concisely. If they ask about injury history, always recommend consulting a physician." During evaluation, you find the assistant correctly adapts to explicit expertise declarations but struggles when users don't clearly state their level — often defaulting to overly detailed responses regardless of contextual cues like technical terminology. Which change to the system prompt would most directly address this failure to pick up on implicit expertise signals?

A. Add an explicit instruction for the model to ask a clarifying question about experience level whenever the user's expertise isn't immediately clear from their first message.

B. Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional about injury consultations.

C. Implement a pre-conversation intake that asks users to rate their experience level, then inject that rating into the system prompt as context for all subsequent responses.

D. Add more conditional branches to cover additional expertise signals, such as "If user mentions specific rep ranges or asks about periodization, treat as advanced."

Answer: B

Explanation:

A. Add an explicit instruction for the model to ask a clarifying question about experience level whenever the user's expertise isn't immediately clear from their first message. Incorrect

This reduces responsiveness and forces unnecessary clarification questions, even when the model could reasonably infer expertise from context.

B. Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional about injury consultations. Correct

This directly fixes the core issue: over-reliance on brittle rules. A general adaptive principle allows the model to infer expertise from implicit signals (terminology, phrasing, specificity) instead of only explicit declarations.

C. Implement a pre-conversation intake that asks users to rate their experience level, then inject that rating into the system prompt as context for all subsequent responses. Incorrect

This improves clarity but adds friction and fails when users provide inaccurate or inconsistent self-assessments.

D. Add more conditional branches to cover additional expertise signals,such as "If user mentions specific rep ranges or asks about periodization, treat as advanced." **Incorrect**

This increases prompt complexity and still won't scale to the full range of implicit cues users naturally express.

Question: 97 **Exam Heist**

You're building a security scanning workflow. When engineers need to locate all occurrences of a dangerous function like eval() across a large codebase, which tool should your agent use for content search?

A. Use Grep to search for the pattern "eval(" across all files in the codebase.
B. Use Glob with a pattern like **/*eval to find files, then Read each matching file.
C. Use Bash to run ls -R | grep eval to recursively list files containing eval.
D. Read the project's main entry file and follow import statements to trace where eval might be used.

Answer: A

Explanation:

A. Use Grep to search for the pattern "eval(" across all files in the codebase. (Correct)

Grep is specifically designed for content searching within files. It can efficiently scan a large codebase and locate every occurrence of a pattern such as eval(, making it the most appropriate tool for security analysis and code auditing.

B. Use Glob with a pattern like **/*eval to find files, then Read each matching file. (Incorrect)

Glob searches file names and paths, not file contents. It would miss occurrences of eval(inside files whose names do not contain "eval".

C. Use Bash to run ls -R | grep eval to recursively list files containing eval. (Incorrect)

This searches file names returned by ls, not the contents of files.

D. Read the project's main entry file and follow import statements to trace where eval might be used. (Incorrect)

This is inefficient and unreliable for finding all occurrences across a large codebase. It may miss many uses of eval().

Question: 98 **Exam Heist**

agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total). What's next step for building understanding while managing context constraints?

A. Use Glob to find files matching common caching patterns (cache.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps.
B. Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.

C. Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context.

D. Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.

Answer: D

Explanation:

D. Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations. (Correct)

When the logic spans many files (~8,000 lines), the goal is to build understanding efficiently while respecting context limits. Starting with the core abstraction (the base cache class or interface) provides a high-level understanding of how the caching system is structured. From there, you can trace the specific implementations and invalidation paths that matter, rather than reading thousands of lines indiscriminately.

A. Use Glob to find files matching common caching patterns, prioritize the largest files by reading them first, then check smaller files for gaps. (Incorrect)

File size is not a reliable indicator of importance. Large files may contain unrelated logic.

B. Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation. (Incorrect)

This is inefficient and likely to exceed context constraints. Understanding should be built incrementally.

C. Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context. (Incorrect)

While useful for targeted investigation, this can miss the broader architecture and relationships needed to understand the caching system correctly.

Question: 99 **Exam Heist**

Today she wants to continue that specific investigation. She's worked on three other codebases since then and knows the session was dive". How should she resume?

A. Start fresh and re-read the same files

B. Use --resume auth-deep-dive to load that specific session by name

C. Use --continue to pick up where the most recent conversation left off

D. Use --session-id with the UUID from yesterday's session transcript file

Answer: B

Explanation:

B. Use --resume auth-deep-dive to load that specific session by name (Correct)

When returning to a specific investigation after working on other projects, using **--resume with the session name** is the most direct way to continue that exact context. It restores the relevant conversation and state associated with the named session.

A. Start fresh and re-read the same files (Incorrect)

This discards existing context and wastes time rebuilding understanding.

C. Use --continue to pick up where the most recent conversation left off (Incorrect)

--continue resumes the most recent session, which may be one of the other codebases worked on since then.

D. Use --session-id with the UUID from yesterday's session transcript file (Incorrect)

While a session ID could identify a session, the question specifically indicates the session is already known by name, making --resume auth-deep-dive the appropriate approach.

Question: 100 **Exam Heist**

"You're the Lead Data Scientist/Engineer on a critical project. Something is not completely right with technical details or the data, and it negative impacts expected outputs. They need to independently develop both approaches to evaluation data... How do you manage this scenario?"

A. Start two fresh sessions, having each re-read the relevant source files before beginning.
B. Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.
C. Resume the analysis session with fork_session enabled, creating a separate branch for each testing strategy.
D. Export the analysis session's key findings to a file, then create two new sessions that reference this file.

Answer: C

Explanation:

C. Resume the analysis session with fork_session enabled, creating a separate branch for each testing strategy. (Correct)

Forking the existing session allows both testing strategies to start from the same investigation context and understanding of the codebase. Each branch can then evolve independently, making it easy to compare approaches without losing prior analysis or mixing the reasoning paths.

A. Start two fresh sessions, having each re-read the relevant source files before beginning. (Incorrect)

This duplicates work and loses the benefit of the existing analysis context.

B. Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially. (Incorrect)

Sequential development makes comparison harder and mixes the reasoning for the two approaches.

D. Export the analysis session's key findings to a file, then create two new sessions that reference this file. (Incorrect)

While possible, it is less efficient than directly forking the existing session and maintaining full contextual continuity.

Question: 101 **Exam Heist**

A security audit requires updating your authentication library from v2 to v3. The migration guide documents breaking changes: authenticate() now returns a Promise instead of accepting a callback, the User type has restructured fields, and three deprecated methods were removed. Grep shows the library is imported in 45 files across several modules. What's the most effective approach?

A. Paste the migration guide's breaking changes into your prompt and use direct execution to update all usages across the 45 files.

B. Enter plan mode to explore library usage across modules, map affected code paths, then create a migration strategy before implementing.

C. Update the dependency version, run the test suite, and use Claude Code to fix each failure as it appears.

D. Create a custom slash command encapsulating the migration transformations, then execute it against each file without prior codebase exploration.

Answer: B

Explanation:

B. Enter plan mode to explore library usage across modules, map affected code paths, then create a migration strategy before implementing. (Correct)

A major library upgrade with breaking changes affecting 45 files should be approached systematically. Plan mode helps analyze how the authentication library is used across the codebase, identify callback-based implementations that must be converted to Promises, understand dependencies on the restructured User type, and locate uses of deprecated methods. Creating a migration strategy before making changes reduces risk and ensures consistency across all modules.

A. Paste the migration guide's breaking changes into your prompt and use direct execution to update all usages across the 45 files. (Incorrect)

Directly modifying dozens of files without first understanding usage patterns can introduce widespread errors and inconsistent implementations.

C. Update the dependency version, run the test suite, and use Claude Code to fix each failure as it appears. (Incorrect)

This is a reactive approach that depends on test coverage and may miss affected code paths that are not exercised by tests.

D. Create a custom slash command encapsulating the migration transformations, then execute it against each file without prior codebase exploration. (Incorrect)

Automating changes without understanding the codebase can apply incorrect transformations and create additional issues.

Question: 102 **Exam Heist**

During testing, you observe that in extended exploration sessions (30+ minutes), the agent starts giving inconsistent answers about context discussed earlier. Engineers report having to repeat context about modules they've already explored. What's the most effective approach to address this?

A. Create summaries of all source files before exploration begins, loading only these compressed representations into context.

B. Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data.

C. Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context.

D. Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions.

Answer: D

Explanation:

D. Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions. (Correct)

In long exploration sessions, context window limitations can cause earlier discoveries to be forgotten or summarized away. Maintaining a persistent scratchpad of important findings, module relationships, decisions, and observations allows the agent to consistently reference prior work without requiring engineers to repeatedly provide the same context.

A. Create summaries of all source files before exploration begins, loading only these compressed representations into context. (Incorrect)

Summaries can omit important implementation details and are not a complete substitute for maintaining evolving investigation knowledge.

B. Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data. (Incorrect)

A larger context window may help temporarily, but it does not solve the underlying problem of managing knowledge across lengthy investigations.

C. Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context. (Incorrect)

Clearing context would worsen the problem by causing the agent to lose even more information from the ongoing investigation.

Question: 103 **Exam Heist**

old_string parameter cannot find unique text to match — the file has repetitive docstrings, variable names, and structural patterns. What is a reliable way to complete this insertion?

A. Use Edit with an extremely long old_string capturing 30+ lines of context to guarantee uniqueness

B. Use Read to load the file, add the function at the appropriate location, then Write the updated file

C. Use Edit's replace_all parameter to target a common pattern and embed the new function in the replacement text

D. Use Bash to append the function definition to the end of the file using heredoc syntax

Answer: B

Explanation:

B. Use Read to load the file, add the function at the appropriate location, then Write the updated file (Correct)

When old_string cannot uniquely identify a match due to repetitive patterns, the most reliable approach is to first **read the full file**, understand its structure, and then **rewrite or update the file explicitly**. This avoids ambiguity and ensures the insertion happens in the correct location without relying on fragile string matching.

A. Use Edit with an extremely long old_string capturing 30+ lines of context to guarantee uniqueness (Incorrect)

This is brittle and error-prone; large context strings still may not guarantee uniqueness in repetitive files.

C. Use Edit's replace_all parameter to target a common pattern and embed the new function in the replacement text (Incorrect)

Using replace_all on common patterns risks unintended modifications across multiple unrelated locations.

D. Use Bash to append the function definition to the end of the file using heredoc syntax (Incorrect)

Blindly appending code can place the function in an incorrect location and break file structure or dependencies.

Question: 104 **Exam Heist**

You need to add a date validation check ensuring event dates are in the future. This requires adding a conditional statement to one existing function in a single file. What is the most appropriate approach?

A. Enter plan mode to analyze how the validation might impact other parts of the reservation flow
B. Start with extended thinking mode enabled to ensure thorough reasoning about the validation logic
C. Use direct execution to make the change
D. Enter plan mode first to create a detailed implementation strategy before making the change

Answer: C

Explanation:

C. Use direct execution to make the change (Correct)

Since this is a **small, localized change** (adding a single validation check inside one existing function in one file), it does not require broader architectural analysis or a multi-step plan. Direct execution is appropriate for quick, low-risk modifications.

A. Enter plan mode to analyze how the validation might impact other parts of the reservation flow (Incorrect)

This is unnecessary overhead for a simple, isolated validation change.

B. Start with extended thinking mode enabled to ensure thorough reasoning about the validation logic (Incorrect)

While reasoning is useful, extended planning modes are not needed for a straightforward conditional check.

D. Enter plan mode first to create a detailed implementation strategy before making the change (Incorrect)

Planning is better suited for multi-file or system-wide changes, not a single-function update.

Question: 105 **Exam Heist**

Your team's CLAUDE.md includes a rule: "Use 4-space indentation and always run Prettier formatting." Despite this, code reviews reveal that roughly 30% of files Claude Code generates use inconsistent formatting — sometimes 2-space indentation, sometimes missing trailing commas. Adding emphasis ("IMPORTANT: You MUST use Prettier formatting") reduces violations to about 15%, but doesn't eliminate them. What is the most effective way to ensure all generated code is consistently formatted?

A. Extract the formatting rules into a dedicated skill that Claude loads automatically when generating code, with more detailed examples of correct formatting.

B. Add a Stop hook with a prompt-based check that evaluates whether generated code follows formatting standards and prompts Claude to fix violations.

C. Configure a Post ToolUse hook with an Edit|Write matcher that automatically runs Prettier on each file Claude modifies.

D. Split the formatting rules into path-scoped .claude/rules/ files that load when Claude works on matching file types.

Answer: C

Explanation:

C. Configure a Post ToolUse hook with an Edit|Write matcher that automatically runs Prettier on each file Claude modifies. (Correct)

The most reliable way to enforce consistent formatting is to **remove reliance on the model remembering rules** and instead enforce formatting through an **automatic post-processing step**. A Post ToolUse hook that runs Prettier guarantees every modified file is formatted correctly, regardless of how the code was generated.

A. Extract the formatting rules into a dedicated skill that Claude loads automatically when generating code, with more detailed examples of correct formatting. (Incorrect)

More instructions and examples improve compliance slightly but do not guarantee consistent formatting.

B. Add a Stop hook with a prompt-based check that evaluates whether generated code follows formatting standards and prompts Claude to fix violations. (Incorrect)

This adds extra reasoning cycles but is still LLM-dependent and not deterministic.

D. Split the formatting rules into path-scoped .claude/rules/ files that load when Claude works on matching file types. (Incorrect)

This improves organization but does not enforce formatting automatically or reliably.

Question: 106 **Exam Heist**

You've asked Claude Code to build a PDF report generation feature. The initial implementation queries the database correctly, but the output has formatting issues: table columns are too narrow causing content truncation, dates display without proper formatting, and page break handling is incorrect. You've noticed these issues interact — changing column widths affects how dates render, and page breaks depend on content height. What's the most effective approach for iterating toward a working solution?

A. Address the column width issue first with specific measurements, verify it works, then fix date formatting within the corrected columns, then adjust page breaks— testing after each change.

B. Provide all three issues in a single detailed message with exact specifications for each, allowing Claude to address them together in one update.

C. Show Claude an example of a correctly formatted report and ask it to match that output, rather than listing the specific technical issues.

D. Start fresh with a detailed prompt specifying all formatting requirements upfront.

Answer: A

Explanation:

A. Address the column width issue first with specific measurements, verify it works, then fix date formatting within the corrected columns, then adjust page breaks — testing after each change. (Correct)

When issues are **interdependent (layout, formatting, and pagination affecting each other)**, the most effective strategy is **iterative isolation**. Fixing one constraint at a time (starting with structural layout like column widths) prevents cascading side effects and makes it easier to understand what each change impacts before moving to the next issue.

B. Provide all three issues in a single detailed message with exact specifications for each, allowing Claude to address them together in one update. (Incorrect)

This can lead to unstable results because multiple interacting constraints are modified at once, making debugging harder.

C. Show Claude an example of a correctly formatted report and ask it to match that output, rather than listing the specific technical issues. (Incorrect)

Example-based prompting helps, but it does not resolve underlying structural dependencies in layout logic.

D. Start fresh with a detailed prompt specifying all formatting requirements upfront. (Incorrect)

Restarting loses iteration context and still doesn't address the need for step-by-step validation of interdependent issues.

Question: 107 **Exam Heist**

You're implementing a caching layer for API responses to speed up the /products endpoint. You have a rough idea — Redis with a 5-minute TTL — but you're new to production caching and aren't sure what other considerations a robust implementation requires. What's the most effective way to start your iterative workflow?

A. Ask Claude to interview you about the caching requirements before implementing, surfacing considerations like invalidation strategies, cache layers, consistency guarantees, and failure modes.

B. Use plan mode to analyze the current/products endpoint implementation, then provide your caching requirements once Claude explains how the existing code is structured.

C. Start with a minimal request: "Add Redis caching to/products with 5-minute TTL." Add features and fix issues through follow-up prompts as problems surface during testing.

D. Write a specification with your known requirements and "TBD" markers for uncertain areas, having Claude propose solutions for each TBD as it implements.

Answer: A

Explanation:

A. Ask Claude to interview you about the caching requirements before implementing, surfacing considerations like invalidation strategies, cache layers, consistency guarantees, and failure modes. (Correct)

When you're unsure about production-grade caching design, the most effective starting point is to first **surface missing requirements and constraints**. An interactive clarification step helps uncover critical considerations like cache invalidation, stale data handling, consistency models, key design, TTL trade-offs, and failure modes before any implementation begins. This leads to a more robust and less error-prone design.

B. Use plan mode to analyze the current /products endpoint implementation, then provide your caching

requirements once Claude explains how the existing code is structured. (Incorrect)

Understanding the endpoint is useful, but jumping into code analysis before clarifying caching requirements can miss key architectural decisions.

C. Start with a minimal request: "Add Redis caching to /products with 5-minute TTL." Add features and fix issues through follow-up prompts as problems surface during testing. (Incorrect)

This reactive approach often leads to brittle caching logic and repeated rework.

D. Write a specification with your known requirements and "TBD" markers for uncertain areas, having Claude propose solutions for each TBD as it implements. (Incorrect)

While structured, it still assumes you already know what to specify, which is not true in this scenario.

Question: 108 **Exam Heist**

You're implementing a complex graph traversal algorithm with specific performance requirements and edge cases to handle (disconnected nodes, cycles, weighted edges).
You want to structure your workflow for efficient iterative refinement with Claude. What approach will most effectively enable progressive improvement across multiple iterations?

A. Write a test suite covering expected behavior, edge cases, and performance requirements before implementation. Ask Claude to write code that passes the tests, then iterate by sharing test failures with each refinement request.

B. Provide Claude with a detailed natural language specification of the algorithm, including all requirements and edge cases. Review each output manually and provide descriptive feedback on what behavior needs to change.

C. Provide Claude with a reference implementation from documentation, then ask it to rewrite the code to match your codebase style and add the required edge case handling, comparing outputs against the reference.

D. Have Claude extensively research the algorithm and create a detailed implementation plan using extended thinking, then implement the complete solution based on that plan.

Answer: A

Explanation:

A. Write a test suite covering expected behavior, edge cases, and performance requirements before implementation. Ask Claude to write code that passes the tests, then iterate by sharing test failures with each refinement request. (Correct)

For a complex graph traversal problem with edge cases like cycles, disconnected nodes, and weighted edges, the most effective workflow is **test-driven iteration**. Defining a test suite upfront creates a precise correctness target, ensures all edge cases are explicitly covered, and allows each iteration to be validated objectively. This reduces ambiguity and makes progressive refinement efficient and measurable.

B. Provide Claude with a detailed natural language specification of the algorithm, including all requirements and edge cases. Review each output manually and provide descriptive feedback on what behavior needs to change. (Incorrect)

This relies too heavily on subjective interpretation and manual review, making iteration slower and less precise than test-driven validation.

C. Provide Claude with a reference implementation from documentation, then ask it to rewrite the code to match your codebase style and add the required edge case handling, comparing outputs against the reference. (Incorrect)

A reference implementation helps, but it does not guarantee correctness for custom edge cases or performance constraints.

D. Have Claude extensively research the algorithm and create a detailed implementation plan using extended thinking, then implement the complete solution based on that plan. (Incorrect)

Planning is useful, but without executable tests, correctness across iterations is harder to validate and refine.

Question: 109 **Exam Heist**

Your team is configuring MCP servers in Claude Code. You want to add a shared venue lookup server that all team members should have access to, and you personally want to add an experimental music playlist server that only you are testing. Which configuration approach correctly applies MCP server scopes?

A. Add venue server to .mcp.json and playlist server to ~/.claude.json
B. Add both servers to the project-level .mcp.json file
C. Add venue server to ~/.claude.json and playlist server to .mcp.json
D. Add both servers to your local ~/.claude.json

Answer: A

Explanation:

A. Add venue server to .mcp.json and playlist server to ~/.claude.json (Correct)

Project-level configuration (.mcp.json) is used for **shared team servers**, ensuring everyone working in the repository has access to the same MCP tools (like the venue lookup server). User-level configuration (~/.claude.json) is used for **personal or experimental servers**, such as a music playlist server that only you want to test locally.

B. Add both servers to the project-level .mcp.json file (Incorrect)

This would expose the experimental playlist server to the entire team, which is not intended.

C. Add venue server to ~/.claude.json and playlist server to .mcp.json (Incorrect)

This reverses the correct scoping: shared tools should not be user-only, and experimental tools should not be project-wide.

D. Add both servers to your local ~/.claude.json (Incorrect)

This would prevent the team from accessing the shared venue server, breaking collaboration.

Question: 110 **Exam Heist**

Your infrastructure-as-code repository includes Terraform modules (/terraform/), Kubernetes manifests (/kubernetes/), and CI/CD pipeline scripts (/pipelines/). Each requires different conventions, but your single root CLAUDE.md has grown to 500+ lines. When developers work on Kubernetes files, Terraform-specific rules load into context unnecessarily, consuming tokens. What is the best approach to reorganize so only relevant guidance loads when editing specific file types?

A. Restructure the root CLAUDE.md into clearly labeled sections with headers (e.g., "## Terraform Conventions"), improving organization and readability.

B. Split content into subdirectory CLAUDE.md files (/terraform/CLAUDE.md, /kubernetes/CLAUDE.md), so Claude loads directory-specific guidance.

C. Keep the root CLAUDE.md and use @path/to/import syntax to modularly include tool-specific guidance files from separate documents.

D. Create files in .claude/rules/ with YAML frontmatter path-scoping (e.g., paths: ["terraform/**/*"]), loading rules only when editing matching files.

Answer: D

Explanation:

D. Create files in .claude/rules/ with YAML frontmatter path-scoping (e.g., paths: ["terraform//*"]), loading rules only when editing matching files.** (Correct)

This approach enables **context-aware rule loading**, ensuring that only relevant guidance is injected based on the files being edited. Terraform rules apply only in /terraform, Kubernetes rules only in /kubernetes, and CI/CD rules only in /pipelines, which significantly reduces unnecessary token usage while keeping governance precise and scalable.

A. Restructure the root CLAUDE.md into clearly labeled sections with headers (e.g., "## Terraform Conventions"), improving organization and readability. (Incorrect)

This improves readability but does not solve the core issue of **unnecessary context loading**.

B. Split content into subdirectory CLAUDE.md files (/terraform/CLAUDE.md, /kubernetes/CLAUDE.md), so Claude loads directory-specific guidance. (Incorrect)

While modular, it does not guarantee selective loading based on file type or path matching.

C. Keep the root CLAUDE.md and use @path/to/import syntax to modularly include tool-specific guidance files from separate documents. (Incorrect)

This still risks pulling unrelated context into memory even when it is not needed.

Question: 111 **Exam Heist**

Your team frequently migrates React components to Vue. You've written a step-by-step workflow for Claude Code to follow during each migration, and you want every developer on the team to invoke it by typing /migrate-component. The workflow should stay in sync as the team iterates on it. Where should you place the skill file?

A. In ~/.claude/skills/migrate-component/SKILL.md on each developer's machine

B. In the project's .claude/settings.json using a skillOverrides entry to register and define the workflow

C. In .claude/skills/migrate-component/SKILL.md at the project root, committed to version control

D. As a detailed instruction block in the project's root CLAUDE.md file

Answer: C

Explanation:

C. In .claude/skills/migrate-component/SKILL.md at the project root, committed to version control (Correct)

A shared team workflow like /migrate-component should live in the **project-level skills directory under version control**. This ensures:

Every developer automatically gets the same workflow
Updates are tracked via git
The skill stays consistent and evolves with the codebase
The /migrate-component command is available project-wide

A. In ~/.claude/skills/migrate-component/SKILL.md on each developer's machine (Incorrect)

This makes the workflow **local and inconsistent across the team**, and it won't stay in sync.

B. In .claude/settings.json using a skillOverrides entry to register and define the workflow (Incorrect)

Settings files are for configuration, not defining reusable workflow skills.

D. As a detailed instruction block in the project's root CLAUDE.md file (Incorrect)

CLAUDE.md is for general guidance, not structured, reusable slash-command skills.

Question: 112 **Exam Heist**

A critical bug is affecting production users. Error logs show exceptions in the OrderProcessing module with a clear stack trace pointing to a specific function. You haven't worked with this module before. What's the most effective approach?

A. Start with direct execution to gather initial information, then switch to plan mode to design a comprehensive solution before implementing any changes.

B. Use plan mode to analyze the error in context of the module's design, enumerate potential root causes, and prioritize fixes systematically.

C. Enter plan mode to explore the module's architecture and dependencies before attempting any fixes.

D. Use direct execution to examine the stack trace, read the relevant code, and implement a fix once you identify the root cause.

Answer: D

Explanation:

D. Use direct execution to examine the stack trace, read the relevant code, and implement a fix once you identify the root cause. (Correct)

Since the issue already has a **clear stack trace pointing to a specific function**, the fastest and most effective approach is to directly investigate the referenced code path, understand the failure, and apply a targeted fix. This minimizes overhead and is appropriate when the failure location is already well-identified.

A. Start with direct execution to gather initial information, then switch to plan mode to design a comprehensive solution before implementing any changes. (Incorrect)

This adds unnecessary planning overhead for a bug that already has a clearly identified source.

B. Use plan mode to analyze the error in context of the module's design, enumerate potential root causes, and prioritize fixes systematically. (Incorrect)

Over-analysis is unnecessary because the stack trace already localizes the issue.

C. Enter plan mode to explore the module's architecture and dependencies before attempting any fixes. (Incorrect)

Architecture-level exploration is too broad for a clearly localized production bug.

Question: 113 **Exam Heist**

You're implementing a new payment processing module that must follow your project's established patterns for database transactions, error handling, and audit logging.
You've identified three existing modules that exemplify these patterns: db_utils.py, error_handlers.py, and audit_logger.py. This is a one-off integration task — these patterns are well-documented in your team wiki and don't need additional project-level documentation. What's the most effective approach?

A. Describe the patterns from the three modules in natural language in your prompt, explaining the transaction handling approach, error format, and logging conventions Claude should follow.

B. Use @references to include the three modules directly in your prompt, giving Claude concrete code examples of the patterns to follow.

C. Ask Claude to explore your codebase to find and understand the transaction, error handling, and logging patterns before generating the new module.

D. Add documentation of each pattern to your CLAUDE.md file, establishing them as project conventions that Claude will apply automatically.

Answer: B

Explanation:

B. Use @references to include the three modules directly in your prompt, giving Claude concrete code examples of the patterns to follow. (Correct)

For a one-off implementation that must strictly follow existing patterns, the most effective approach is to provide **direct, concrete code references**. Including db_utils.py, error_handlers.py, and audit_logger.py ensures Claude can accurately replicate transaction handling, error formats, and logging conventions without relying on abstraction or interpretation.

A. Describe the patterns from the three modules in natural language in your prompt, explaining the transaction handling approach, error format, and logging conventions Claude should follow. (Incorrect)

Natural language descriptions are prone to ambiguity and may miss subtle implementation details present in the actual code.

C. Ask Claude to explore your codebase to find and understand the transaction, error handling, and logging patterns before generating the new module. (Incorrect)

While exploration helps, it is less precise and less reliable than explicitly providing the canonical examples.

D. Add documentation of each pattern to your CLAUDE.md file, establishing them as project conventions that Claude will apply automatically. (Incorrect)

This is unnecessary overhead for a one-off task and does not guarantee accurate replication of nuanced implementation details.

Question: 114 **Exam Heist**

You've asked Claude to write a data migration script, but the initial output doesn't correctly handle records with null values in required fields. What's the most effective way to iterate toward a working solution?

A. Provide a test case with example input containing null values and the expected output, then ask Claude to fix it.

B. Manually edit the generated code to fix the null handling, then continue working with Claude on other parts.

C. Describe the null value problem in detail and ask Claude to regenerate the entire script with improved edge case handling.

D. Add "think harder about edge cases" to your prompt and request a complete rewrite of the migration logic.

Answer: A

Explanation:

A. Provide a test case with example input containing null values and the expected output, then ask Claude to fix it. (Correct)

The most effective iterative approach is to make the failure **explicit and verifiable**. By providing concrete input/output examples for null-handling edge cases, you give a precise contract for the expected behavior. This allows Claude to adjust the implementation accurately and testably, rather than relying on vague descriptions or full rewrites.

B. Manually edit the generated code to fix the null handling, then continue working with Claude on other parts. (Incorrect)

Manual fixes break the iterative feedback loop and make it harder for Claude to learn from structured correction signals.

C. Describe the null value problem in detail and ask Claude to regenerate the entire script with improved edge case handling. (Incorrect)

Regenerating the full script is inefficient and risks introducing regressions in already correct parts of the logic.

D. Add "think harder about edge cases" to your prompt and request a complete rewrite of the migration logic. (Incorrect)

This is too vague and does not provide actionable guidance or measurable correctness criteria.

Question: 115 **Exam Heist**

You've documented API error handling conventions in a CLAUDE.md file at your project root, specifying that endpoint handlers should use a custom ApiError class. After several sessions, you notice Claude Code sometimes follows these conventions and sometimes uses generic try/catch blocks with string messages. The inconsistency appears random across different coding sessions. What's the most efficient first diagnostic step?

A. Add more detailed code examples to your CLAUDE.md showing the exact ApiError usage pattern for different endpoint types.

B. Run /memory to check which memory files are loaded and verify your CLAUDE.md is included.

C. Search for conflicting instructions in ~/.claude/CLAUDE.md or ~/.claude/rules/ that might override your project conventions.

D. Create path-specific rules in claude/rules/handlers.md with YAML frontmatter scoping the error handling instructions to your API handler files.

Answer: B

Explanation:

B. Run /memory to check which memory files are loaded and verify your CLAUDE.md is included.

(Correct)

The first and most efficient diagnostic step is to confirm whether the expected **project-level context is actually being loaded**. Since behavior is inconsistent across sessions, the most likely issue is not the rule content itself but **context loading or memory inclusion**. Checking /memory verifies whether CLAUDE.md and other relevant rules are being applied in the current session.

A. Add more detailed code examples to your CLAUDE.md showing the exact ApiError usage pattern for different endpoint types. (Incorrect)

This addresses rule clarity, but the issue is not necessarily lack of detail — it may be missing or conflicting context.

C. Search for conflicting instructions in ~/.claude/CLAUDE.md or ~/.claude/rules/ that might override your project conventions. (Incorrect)

This is a valid next step, but premature before confirming whether the project rules are even being loaded.

D. Create path-specific rules in claude/rules/handlers.md with YAML frontmatter scoping the error handling instructions to your API handler files. (Incorrect)

This is a structural fix, not a diagnostic step. You should first identify why the inconsistency is happening.

Question: 116 **Exam Heist**

Your monorepo contains shared coding standards in /docs/standards/security-rules.md (for services handling user data), testing-patterns.md (for all packages), and api-conventions.md (for API-facing services). Your 15 packages are organized by feature domain (/packages/auth/, /packages/billing/, /packages/notifications/, etc.) without naming conventions indicating which handle user data or expose APIs. Package maintainers are expected to configure their own local development settings, as they understand their package's domain requirements. Currently, all package CLAUDE.md files duplicate all three standards, applying irrelevant guidance. What's the most effective approach?

A. Create claude/rules/ files for each standard with YAML frontmatter paths listing every package directory where that standard should apply.

B. Create a shared-standards.nd that uses @imports to combine all three standards, then have each package's CLAUDE.md import that combined file.

C. Put all standards in the root CLAUDE.md with override instructions like "ignore security-rules.md when working in packages that don't handle user data."

D. Use @imports in each package's CLAUDE.md to reference only the specific standard files relevant to that package, based on the maintainer's domain knowledge.

Answer: D

Explanation:

D. Use @imports in each package's CLAUDE.md to reference only the specific standard files relevant to that package, based on the maintainer's domain knowledge. (Correct)

Since the package structure does not inherently indicate which standards apply, and maintainers are expected to understand their package's requirements, the most effective approach is for each package's CLAUDE.md to import only the relevant standards. This eliminates duplication, keeps guidance targeted, and avoids loading irrelevant instructions while leveraging the maintainers' domain expertise.

A. Create .claude/rules/ files for each standard with YAML frontmatter paths listing every package directory where that standard should apply. (Incorrect)

This requires centrally maintaining package-to-standard mappings, which is difficult when applicability depends on domain knowledge rather than directory structure.

B. Create a shared-standards.md that uses @imports to combine all three standards, then have each package's CLAUDE.md import that combined file. (Incorrect)

This continues loading irrelevant standards into every package, which is the problem being solved.

C. Put all standards in the root CLAUDE.md with override instructions like "ignore security-rules.md when working in packages that don't handle user data." (Incorrect)

This increases context size and complexity, forcing Claude to process and ignore unnecessary guidance rather than loading only what is needed.

Question: 117 **Exam Heist**

The system processes product reviews using tool use with a defined schema: rating (integer 1-5), pros (string array), cons (string array), and overall_sentiment (enum: positive, negative, mixed). Testing reveals two issues with brief or ambiguous reviews (~20% of the dataset): (1) for reviews like "Great product!", Claude fabricates specific pros and cons rather than indicating this information isn't explicitly stated, and (2) for sarcastic reviews like "Well that was... interesting", Claude picks sentiment arbitrarily since there's no option for ambiguous cases. What schema modification best addresses both issues?

A. Add an extraction_confidence field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.

B. Allow null values for pros/cons, and add "unclear" to the sentiment enum.

C. Allow empty arrays for pros/cons as valid output, and add "unclear" to the sentiment enum.

D. Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum.

Answer: C

Explanation:

C. Allow empty arrays for pros/cons as valid output, and add "unclear" to the sentiment enum. (Correct)

This directly addresses both problems:

For reviews like **"Great product!"**, there may be no explicitly stated pros or cons. Allowing **empty arrays** lets the model accurately represent that no specific pros or cons were mentioned, instead of hallucinating details.

For ambiguous or sarcastic reviews like **"Well that was... interesting"**, adding **"unclear"** to the sentiment enum provides a valid option when sentiment cannot be determined reliably from the text.

A. Add an extraction_confidence field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold. (Incorrect)

Confidence scores do not solve the underlying schema problem. The model may still fabricate pros/cons or guess sentiment.

B. Allow null values for pros/cons, and add "unclear" to the sentiment enum. (Incorrect)

Null suggests the fields are missing rather than that no pros/cons were explicitly stated. Empty arrays more accurately represent "none identified."

D. Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum. (Incorrect)

Optional fields can lead to inconsistent outputs. "Neutral" is different from "unclear" and does not address ambiguity in determining sentiment.

Question: 118 **Exam Heist**

After your daily batch of 10,000 documents completes, 300 documents (3%) failed with "context_length_exceeded" errors. The results file identifies each failure by custom_id. What's the most cost-effective approach to process these failures?

A. Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts

B. Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions

C. Increase the max_tokens parameter for the 300 failed documents and resubmit them in a new batch

D. Resubmit the entire 10,000 document batch using a model tier with a larger context window

Answer: B

Explanation:

B. Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions. (Correct)

The failures are specifically due to **context_length_exceeded**, meaning the documents are too large for the model's context window. The most cost-effective solution is to **retry only the failed 300 documents**, split them into smaller chunks that fit within context limits, and then merge the results. This avoids reprocessing the 9,700 documents that already succeeded.

A. Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts (Incorrect)

Prompt caching may reduce costs, but it does not solve the context-length issue and unnecessarily reprocesses successful documents.

C. Increase the max_tokens parameter for the 300 failed documents and resubmit them in a new batch (Incorrect)

max_tokens controls output length, not input context capacity. It will not resolve context_length_exceeded errors.

D. Resubmit the entire 10,000 document batch using a model tier with a larger context window (Incorrect)

This is significantly more expensive and unnecessary when only 3% of documents failed.

Question: 119 **Exam Heist**

The system needs to extract candidate information (name, contact details, skills, work experience, education) from uploaded resumes. The extracted data must strictly conform to a predefined JSON schema, as missing required fields or incorrect data types will cause downstream validation failures. What is the most reliable approach to ensure Claude's output consistently matches the schema?

A. Define a tool with an input schema matching your required JSON structure and extract the data from

Claude's tool_use response.

B. Include detailed JSON formatting instructions and a template example in the system prompt, asking Claude to output only valid JSON.

C. Parse Claude's text response with regex patterns to extract JSON objects, using retry logic for malformed responses.

D. Make two separate API calls — first extracting information as text, then asking Claude to format that text as JSON.

Answer: A

Explanation:

A. Define a tool with an input schema matching your required JSON structure and extract the data from Claude's tool_use response. (Correct)

Using **tool use with a defined schema** is the most reliable way to ensure the extracted candidate information conforms to the required JSON structure. The schema enforces field names, required properties, data types, and allowed values, significantly reducing validation failures and eliminating the need to parse free-form text.

B. Include detailed JSON formatting instructions and a template example in the system prompt, asking Claude to output only valid JSON. (Incorrect)

Prompt instructions improve consistency but cannot guarantee strict schema compliance.

C. Parse Claude's text response with regex patterns to extract JSON objects, using retry logic for malformed responses. (Incorrect)

Regex-based parsing is brittle and does not ensure required fields or correct data types.

D. Make two separate API calls — first extracting information as text, then asking Claude to format that text as JSON. (Incorrect)

This adds complexity and still does not provide the schema enforcement that tool use offers.\

Question: 120 **Exam Heist**

After deploying automated code review, developers report that approximately 35% of flagged findings are false positives falling into consistent patterns: style suggestion contradicting team conventions, security warnings for patterns safe in your deployment context, and performance suggestions that would degrade your specific use ca. You want to reduce false positives while maintaining the ability to catch genuine issues. Which approach best enables the model to generalize its judgment to novel co patterns it hasn't seen before?

A. Include few-shot examples in your prompt showing annotated code snippets that distinguish acceptable patterns from genuine issues in each category.

B. Implement post-processing that uses keyword matching to filter out findings containing terms like "convention," "context-dependent," or "trade-off."

C. Add instructions to your system prompt to "be conservative," "only flag definite issues," and "consider that some patterns may be intentional."

D. Create a comprehensive written specification of all patterns that should not be flagged, then include this full documentation in the system prompt.

Answer: A

Explanation:

A. Include few-shot examples in your prompt showing annotated code snippets that distinguish acceptable patterns from genuine issues in each category. (Correct)

Few-shot examples teach the model the underlying distinctions between acceptable and problematic patterns, allowing it to generalize to new, unseen code scenarios. By demonstrating how team conventions, deployment-specific security considerations, and performance trade-offs should be evaluated, the model learns the decision-making criteria rather than memorizing specific cases. This is the most effective approach for reducing systematic false positives while preserving the ability to identify genuine issues.

B. Implement post-processing that uses keyword matching to filter out findings containing terms like "convention," "context-dependent," or "trade-off." (Incorrect)

Keyword filtering is brittle and may suppress legitimate findings that happen to contain those terms while failing to address the root cause of incorrect model reasoning.

C. Add instructions to your system prompt to "be conservative," "only flag definite issues," and "consider that some patterns may be intentional." (Incorrect)

General instructions may reduce the number of findings but do not provide enough contextual guidance for distinguishing acceptable patterns from genuine issues, especially in novel situations.

D. Create a comprehensive written specification of all patterns that should not be flagged, then include this full documentation in the system prompt. (Incorrect)

A long list of rules is difficult for the model to apply consistently and may not help it generalize to new patterns. Examples are typically more effective at teaching nuanced judgment than exhaustive documentation alone.

Question: 121 **Exam Heist**

After expanding the agent's MCP tools with delivery-specific capabilities (check_delivery_status, contact_driver, issue_credit, apply_promo_code, update_delivery_address, reschedule_delivery), the total tool count has grown from 4 to 10. Your evaluation suite shows tool selection accuracy has dropped to 71%. Log analysis reveals the majority of errors involve the agent selecting between semantically overlapping tools-calling issue_credit when process_refund is correct, and calling check_delivery_status when lookup_order already returns the needed data. Which approach structurally eliminates the semantic overlaps that are being logged as the error source?

A. Split the tools across two sub-agents - a "financial resolution" agent with process_refund, issue_credit, and apply_promo_code, and a "delivery" agent with the remaining delivery tools - with a coordinator routing between them.

B. Add few-shot examples to the system prompt demonstrating correct selection for each ambiguous tool pair, such as showing when issue_credit or process_refund is appropriate.

C. Consolidate semantically overlapping tools-merge issue_credit and process_refund into a single resolve_compensation tool with an optional include_tracking flag.

D. Enable the tool search tool with defer_loading on the six new tools, keeping the original four always loaded, so the agent dynamically calls it when needed.

Answer: C

Explanation:

C. Consolidate semantically overlapping tools — merge issue_credit and process_refund into a single resolve_compensation tool with an optional include_tracking flag. (Correct)

The logged errors stem from **semantic overlap between tools**, where multiple tools can satisfy similar intents. Consolidating overlapping tools removes the ambiguity entirely, eliminating the need for the model to choose

between near-equivalent actions. By designing tools around distinct user intents rather than overlapping implementations, tool selection becomes simpler and more reliable.

A. Split the tools across two sub-agents - a "financial resolution" agent with process_refund, issue_credit, and apply_promo_code, and a "delivery" agent with the remaining delivery tools - with a coordinator routing between them. (Incorrect)

This reduces the number of tools visible to each agent but does not eliminate overlap within a domain. The financial agent would still need to choose between issue_credit and process_refund.

B. Add few-shot examples to the system prompt demonstrating correct selection for each ambiguous tool pair, such as showing when issue_credit or process_refund is appropriate. (Incorrect)

Few-shot examples can improve accuracy but do not remove the underlying ambiguity. The model must still make a difficult distinction between overlapping tools.

D. Enable the tool search tool with defer_loading on the six new tools, keeping the original four always loaded, so the agent dynamically calls it when needed. (Incorrect)

Deferred loading reduces the number of immediately available tools but does not address the root cause of errors: overlapping tool semantics. Once loaded, the same ambiguity remains.

Question: 122 **Exam Heist**

Anthropic's tool use documentation states: "Write instructive error messages. Instead of generic errors like 'failed', include what went wrong and what Claude should try next." A billing dispute agent uses lookup_order, which catches all exceptions and returns a tool_result with is_error: true and the message "execution failed". Monitoring shows two failure modes: the agent retries the identical call until hitting the turn limit, or it immediately calls escalate_to_human without trying alternative tools. Which change follows the documented recommendation and gives Claude the information it needs to select the correct recovery action for each error type?

A. Implement retry logic with exponential backoff inside each tool implementation so transient errors are resolved transparently within the tool before any failure result is surfaced to Claude in the agentic loop.

B. Return error-type-specific messages with is_error: true, e.g., "order not found-try get_customer to search by phone" for data errors and "Database timeout (transient)-retry should succeed" for infrastructure errors.

C. Remove is_error: true and return the error details as normal tool content, so Claude reasons about the response as data rather than treating it as a flagged failure condition that biases retry behavior.

D. Add an error classification step in the agentic loop that intercepts tool errors before Claude sees them, tags each as "retry" "try_alternative," or "escalate," and adds that recommendation to the tool result.

Answer: B

Explanation:

B. Return error-type-specific messages with is_error: true, e.g., "order not found — try get_customer to search by phone" for data errors and "Database timeout (transient) — retry should succeed" for infrastructure errors. (Correct)

Anthropic's recommendation is to provide **instructive error messages** that explain both **what went wrong** and **what action Claude should take next**. Error-specific messages give the model the context needed to distinguish between failures that warrant a retry and failures that require trying a different tool. This directly addresses the observed behaviors of repeated identical retries and premature escalation.

A. Implement retry logic with exponential backoff inside each tool implementation so transient errors are resolved transparently within the tool before any failure result is surfaced to Claude in the agentic loop.

(Incorrect)

Internal retries can help with transient failures, but they do not solve the problem of providing actionable guidance when an error is ultimately returned to Claude.

C. Remove is_error: true and return the error details as normal tool content, so Claude reasons about the response as data rather than treating it as a flagged failure condition that biases retry behavior. (Incorrect)

Errors should still be clearly identified as errors. Removing is_error: true obscures failure states and can lead to incorrect reasoning about tool outputs.

D. Add an error classification step in the agentic loop that intercepts tool errors before Claude sees them, tags each as "retry," "try_alternative," or "escalate," and adds that recommendation to the tool result. (Incorrect)

While potentially helpful, this adds an external classification layer rather than following the documented guidance. The recommendation is for tools themselves to return informative, actionable error messages that explain the failure and next step.

Question: 123 **Exam Heist**

Your code review prompts include both implementation changes and the corresponding test file, but the LLM's review comments fail to point out untested code paths.
Analysis reveals the model correctly flags functions that have no tests at all, but fails to identify when conditional branches or error-handling paths within tested functions that have no tests at all, but fails to identify when conditional branches or error-handling paths within tested functio
lack coverage.
What's the most effective way to improve detection of branch-level coverage gaps without overcomplicating the pipeline?

A. Implement a multi-pass pipeline where separate LLM calls first extract all conditional branches, then cross-reference each against test assertions in a second pass.

B. Include few-shot examples showing code with an uncovered branch paired with the review comment identifying the specific missing test case.

C. Add explicit instructions directing the model to enumerate each conditional branch and exception path, then verify each has a corresponding test assertion.

D. Restructure the prompt to interleave implementation and tests, presenting each function followed immediately by its test cases

Answer: B

Explanation:

B. Include few-shot examples showing code with an uncovered branch paired with the review comment identifying the specific missing test case. (Correct)

Few-shot examples are the most effective way to improve the model's ability to recognize **branch-level test coverage gaps** and generalize that reasoning to new code. By demonstrating examples where conditional branches or error-handling paths are untested — and showing the expected review comments — the model learns the pattern it should look for during review without adding significant pipeline complexity.

A. Implement a multi-pass pipeline where separate LLM calls first extract all conditional branches, then cross-reference each against test assertions in a second pass. (Incorrect)

This may improve accuracy but substantially increases system complexity, latency, and cost. The question asks for improvement **without overcomplicating the pipeline**.

C. Add explicit instructions directing the model to enumerate each conditional branch and exception path, then verify each has a corresponding test assertion. (Incorrect)

Instructions can help, but they are generally less effective than examples at teaching nuanced reasoning patterns such as identifying branch-level coverage gaps.

D. Restructure the prompt to interleave implementation and tests, presenting each function followed immediately by its test cases. (Incorrect)

Improving code-test proximity may aid review, but it does not directly teach the model how to identify missing coverage for specific branches or error paths.

Question: 124 **Exam Heist**

After deploying the automated review, you notice high precision but low recall - real bugs are slipping through undetected. Investigation reveals your review prompt instructs Claude to "only report high-confidence issues you are certain about" and "err on the side of not commenting." Developers appreciate the low noise, but a race condition that caused a production outage was visible in a reviewed PR and went unreported. You need to substantially improve bug detection while keeping false positive rates manageable for your team. What is the most effective approach?

A. Expand the context window by including related test files, recent git history, and the module's dependency graph alongside the diff, giving Claude richer signals to assess issue severity.

B. Remove the conservative filtering instructions and prompt Claude to report all potential issues, then apply a programmatic filter to deduplicate and suppress categories that historically generate false positives.

C. Split the review into a finding stage where Claude's goal is coverage - flagging every potential issue with confidence and severity metadata - and a separate thresholds those findings.

D. Add detailed few-shot examples demonstrating bug categories Claude should flag - race conditions, null dereferences, error handling gaps - while keeping confidence filtering instruction to maintain current precision levels.

Answer: C

Explanation:

C. Split the review into a finding stage where Claude's goal is coverage — flagging every potential issue with confidence and severity metadata — and a separate filtering stage that applies team-specific thresholds to those findings. (Correct)

The core problem is that the prompt's **conservative filtering instructions are suppressing recall**, causing genuine bugs (such as the race condition) to go unreported. Separating **issue detection** from **issue filtering** is the most effective solution:

Stage 1 (Detection): Maximize recall by identifying all plausible issues and assigning confidence/severity scores.
Stage 2 (Filtering): Apply team-specific thresholds to control noise and maintain manageable false-positive rates.

This architecture avoids forcing the model to simultaneously optimize for both recall and precision, which often leads to missed bugs.

A. Expand the context window by including related test files, recent git history, and the module's dependency graph alongside the diff, giving Claude richer signals to assess issue severity. (Incorrect)

More context can help in some cases, but it does not address the primary issue: the prompt explicitly tells

Claude to suppress anything it is not certain about.

B. Remove the conservative filtering instructions and prompt Claude to report all potential issues, then apply a programmatic filter to deduplicate and suppress categories that historically generate false positives. (Incorrect)

This improves recall but relies on coarse category-level filtering rather than evaluating each finding's confidence and severity. It is less flexible and effective than a dedicated detection-and-filtering pipeline.

D. Add detailed few-shot examples demonstrating bug categories Claude should flag — race conditions, null dereferences, error handling gaps — while keeping confidence filtering instructions to maintain current precision levels. (Incorrect)

Examples may help identify more bug patterns, but the conservative instructions would still suppress many findings, leaving the recall problem largely unresolved.

Question: 125 **Exam Heist**

Your automated reviewer uses a single prompt covering security issues, API design, and business logic correctness. Your evaluation suite shows strong recall findings (82%) but poor recall for business logic edge cases in quiz scoring (34%). When you add few-shot examples of logic bugs to the prompt, logic recall is 41% but API design recall drops to 68%. How should you address this trade-off to improve detection across both categories?

A. Split the review into separate focused prompts - one for security and API design, another for business logic - each with dedicated examples, then combine findings before posting.

B. Upgrade to a more capable model tier, since its stronger reasoning will handle both concern types in a single prompt and eliminate the recall trade-off.

C. Provide the full repository as context instead of just the changed files and surrounding code, giving the model deeper visibility into business logic.

D. Replace the few-shot examples with a detailed checklist of specific logic edge cases to verify, such as division-by-zero in score calculation or grading thresholds.

Answer: A

Explanation:

A. Split the review into separate focused prompts - one for security and API design, another for business logic - each with dedicated examples, then combine findings before posting. (Correct)

The results show a classic prompt-capacity trade-off: adding business-logic examples improves logic recall but reduces API design recall. By separating concerns into specialized review passes, each prompt can focus on its domain with targeted instructions and examples without competing for attention or context. The findings can then be merged into a single review output, improving overall recall across categories.

B. Upgrade to a more capable model tier, since its stronger reasoning will handle both concern types in a single prompt and eliminate the recall trade-off. (Incorrect)

A stronger model may help, but it does not fundamentally solve the prompt specialization problem. The trade-off can still persist when multiple review objectives compete within one prompt.

C. Provide the full repository as context instead of just the changed files and surrounding code, giving the model deeper visibility into business logic. (Incorrect)

More context increases cost and complexity and may even dilute attention. The observed issue stems from balancing multiple review goals, not a lack of repository-wide context.

D. Replace the few-shot examples with a detailed checklist of specific logic edge cases to verify, such as division-by-zero in score calculation or grading thresholds. (Incorrect)

Checklists can help for known cases but are less effective than dedicated, specialized review passes. They also do not address the underlying competition between business-logic and API-design review objectives within a single prompt.

Question: 126 **Exam Heist**

Your agent is handling a billing dispute. After calling get_customer and lookup_order, it identifies that the dispute involves a promotional pricing ero
approval — beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

A. Persist the complete conversation and tool response history to a database, then call escalate_to_human with a reference ID.

B. Compile a structured handoff with customer details, order info, and the identified issue before calling escalate_to_human

C. Attempt the refund with process refund anyway, escalating only if the system rejects the transaction.

D. Call escalate_to_human passing only the customer's original message.

Answer: B

Explanation:

B. Compile a structured handoff with customer details, order info, and the identified issue before calling escalate_to_human. (Correct)

When an agent determines that a case requires human approval or exceeds its authorization level, it should create a structured handoff containing all relevant context gathered during the investigation. This enables the human agent to continue from the current state without repeating work and ensures a smooth transition.

A. Persist the complete conversation and tool response history to a database, then call escalate_to_human with a reference ID. (Incorrect)

While storing history may be useful, providing only a reference ID does not ensure the human agent receives a clear summary of the issue or the investigation already performed.

C. Attempt the refund with process_refund anyway, escalating only if the system rejects the transaction. (Incorrect)

The scenario states that the refund amount exceeds the agent's authorization level. The agent should respect authorization boundaries and escalate rather than attempting the action.

D. Call escalate_to_human passing only the customer's original message. (Incorrect)

Passing only the original message omits the customer information, order details, and findings already collected, forcing the human agent to repeat the investigation.

Question: 127 **Exam Heist**

Your agent has called lookup_order multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment info, status history). Tool outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the effective

approach before making additional lookups?97

A. Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details

B. Proceed with additional lookups without modifying the existing tool output context

C. Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues

D. Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions

Answer: A

Explanation:

A. Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details. (Correct)

As tool outputs begin to dominate the context window, the best approach is to **compress and retain only the information relevant to the current task**. Since the conversation is focused on return requests, fields such as items, purchase date, return eligibility, and order status should be preserved, while unrelated details like payment information, full shipping records, and extensive status history can be removed. This maintains the necessary context while conserving tokens for future interactions and tool calls.

B. Proceed with additional lookups without modifying the existing tool output context. (Incorrect)

Continuing to accumulate large tool responses wastes context window space and increases the risk of losing important information later in the conversation.

C. Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues. (Incorrect)

While retrieval systems can be useful for long-term memory, this is a more complex solution than necessary. The immediate issue is excessive tool output, which can be addressed more effectively through targeted context reduction.

D. Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions. (Incorrect)

Natural language summaries may omit important fields and are generally less reliable than retaining a compact, structured representation of the relevant data.

Question: 128 **Exam Heist**

Production logs show that when the agent handles complex billing disputes requiring 6+ tool calls, it sometimes exhausts its max_turns limit after gathering data and before completing resolution or escalating. The team's goal is to guarantee that every customer interaction ends with either a completed resolution or a human escalation, regardless of how the agent loop terminates. Which approach achieves this guarantee?

A. Add orchestration-layer code that checks the agent's outcome after each loop termination - if the loop ended without a completed resolution or escalation, programmatically call escalate_to_human with the accumulated conversation context and tool results.

B. Implement a pre-tool-use hook that counts tool invocations and terminates the loop with an automatic escalation once the agent reaches 80% of its remaining actions.

C. Add system prompt instructions telling the agent to call escalate_to_human with a summary of its findings whenever it determines it cannot resolve the dispute.

D. Split the workflow into two sequential agent invocations — a first agent gathers information via get_customer and lookup_order, then the second agent uses that data and handles process_refund or escalate_to_human, each with separate turn budgets.

Answer: A

Explanation:

A. Add orchestration-layer code that checks the agent's outcome after each loop termination - if the loop ended without a completed resolution or escalation, programmatically call escalate_to_human with the accumulated conversation context and tool results. (Correct)

This is the only option that **guarantees** every interaction ends in either a successful resolution or a human escalation. By enforcing the check outside the agent loop, the system remains robust even if the agent exhausts its max_turns, encounters unexpected behavior, or fails to explicitly escalate. The orchestration layer acts as a reliable fallback that ensures no case is left unresolved.

B. Implement a pre-tool-use hook that counts tool invocations and terminates the loop with an automatic escalation once the agent reaches 80% of its remaining actions. (Incorrect)

This may reduce the likelihood of hitting the turn limit, but it does not guarantee resolution or escalation in all termination scenarios.

C. Add system prompt instructions telling the agent to call escalate_to_human with a summary of its findings whenever it determines it cannot resolve the dispute. (Incorrect)

Prompt instructions improve behavior but cannot provide a hard guarantee, especially when the agent reaches max_turns before making that decision.

D. Split the workflow into two sequential agent invocations — a first agent gathers information via get_customer and lookup_order, then the second agent uses that data and handles process_refund or escalate_to_human, each with separate turn budgets. (Incorrect)

This may reduce turn-limit issues but still does not guarantee that one of the agents will successfully resolve or escalate every case. An external enforcement mechanism is still required.

Question: 129 **Exam Heist**

The document analysis agent has a single analyze_document tool that takes a document and a free-text instruction parameter. During evaluation, requests like "key financial metrics" often return narrative summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports analysis results require re-requests with clarified instructions. What's the most effective way to improve reliability?

A. Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats

B. Keep the single tool but add an analysis type enum parameter requiring explicit selection between extraction, summarization, and verification

C. Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent

D. Split the generic tool into purpose specific tools — extract data points, summarize content, verify claim against source — each with

Answer: D

Explanation:

D. Split the generic tool into purpose-specific tools — extract data points, summarize content, verify claims against source — each with clearly defined inputs and outputs. (Correct)

The root cause is that a single tool with a free-text instruction parameter has **ambiguous semantics**, leading to inconsistent outputs. Splitting it into purpose-specific tools removes this ambiguity by giving each tool a clear responsibility and expected output format. This makes tool selection more reliable and reduces the need for re-requests or clarification by downstream agents.

A. Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats. (Incorrect)

Examples may improve performance, but the tool still relies on interpreting free-text instructions, leaving ambiguity and inconsistency in place.

B. Keep the single tool but add an analysis type enum parameter requiring explicit selection between extraction, summarization, and verification. (Incorrect)

An enum helps reduce ambiguity, but the tool still combines multiple responsibilities. Purpose-specific tools generally provide clearer semantics and more reliable behavior.

C. Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent. (Incorrect)

This shifts the classification burden elsewhere but does not solve the underlying issue that the tool itself performs multiple distinct tasks with overlapping behavior.

Question: 130 **Exam Heist**

In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What's the most effective approach to optimize for varying query complexity?

A. Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.

B. Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.

C. Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.

D. Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined combination.

Answer: A

Explanation:

A. Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.

Why this is the correct approach

This scenario highlights the classic trade-off between standard, rigid routing pipelines and **agentic dynamic routing**. Because your system encounters a highly diverse and evolving distribution of queries, you cannot rely on fixed patterns or frozen models.

Dynamic Assessment: Leveraging an LLM coordinator to analyze incoming intent allows the system to scale its compute dynamically. Simple factual queries are answered immediately or routed only to a single retrieval agent (taking ~1–2 seconds), while complex analytical prompts naturally branch into multi-agent execution paths.
Adaptability to Evolving Usage: As users find new applications for your system, an LLM-based coordinator handles novel query structures inherently by reasoning about the task goals, avoiding the bottleneck of continuous manual updates or retraining cycles.

Why the other choices are Incorrect

Option B introduces heavy operational friction: Training and maintaining a standalone classification model requires a constant supply of clean, manually labeled historical datasets. In an environment where user patterns are rapidly evolving, the classifier will frequently experience data drift and misclassify new variations of complex queries.
Option C creates an inefficient binary bottleneck: Forcing a rigid "all-or-nothing" pipeline means that any query not caught by the narrow factual fast-path — such as a moderately complex query requiring only two subagents — will still be forced to traverse all four, continuing to waste time and tokens.
Option D fails on real-world semantic variability: Pattern-based routing (like Regex or simple keyword matching) looks at the structure of a sentence rather than its semantic meaning. A query like "Compare the signing years of the Paris Agreement and the Kyoto Protocol" looks structurally identical to an intensive comparative research paper, yet it only requires two quick factual lookups.

Question: 131 **Exam Heist**

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach best balances information fidelity with context efficiency when restoring agent state?

A. Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings.

B. Have each agent maintain its own persistent state file and reload it independently at the start of each session.

C. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.

D. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.

Answer: C

Explanation:

C. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts. (Correct)

This approach preserves **structured, deterministic intermediate state** from each agent, which is critical for resuming a partially completed multi-agent pipeline without redoing work. The coordinator can reconstruct the workflow precisely by loading a **manifest of completed and partial results**, then selectively passing only the required context back into each agent.

It avoids redundant computation, prevents loss of fidelity, and keeps context usage efficient by not reintroducing full logs or relying on probabilistic retrieval mechanisms.

A. Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings. (Incorrect)

Vector search introduces **non-deterministic retrieval**, which can miss exact intermediate structured outputs needed for correct continuation of a pipeline.

B. Have each agent maintain its own persistent state file and reload it independently at the start of each session. (Incorrect)

This leads to **fragmented state ownership**, making cross-agent coordination harder and increasing the risk of inconsistencies in shared workflow context.

D. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming. (Incorrect)

Raw logs are **unstructured and inefficient**, and may include irrelevant context that wastes tokens and reduces clarity when resuming execution.

Question: 132 **Exam Heist**

Your agent has analyzed a complex service module — reading 23 source files, tracing request flows, and identifying error handling patterns. A developer needs to develop two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. The developer needs to independently develop both approaches to evaluate trade-offs. How should you manage the sessions?

A. Resume the analysis session with fork_session enabled, creating a separate branch for each testing strategy.
B. Export the analysis session's key findings to a file, then create two new sessions that reference this file.
C. Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.
D. Start two fresh sessions, having each re-read the relevant source files before beginning.

Answer: A

Explanation:

A. Resume the analysis session with fork_session enabled, creating a separate branch for each testing strategy. (Correct)

Forking the session is the most effective way to support **parallel, independent exploration while preserving the full analysis context in-memory**. Since the agent has already completed a heavy analysis phase (23 source files, request flows, error handling), that state should not be recomputed or exported — it should be reused directly.

With fork_session, each branch inherits the same authoritative context and can independently explore end-to-end tests vs snapshot tests without interference, ordering bias, or redundant reprocessing. This makes it ideal for evaluating trade-offs under identical starting conditions.

B. Export the analysis session's key findings to a file, then create two new sessions that reference this file. (Incorrect)

This introduces unnecessary serialization and reconstruction overhead. It also risks losing subtle contextual relationships present in the live session state.

C. Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially. (Incorrect)

Sequential execution prevents independent evaluation and introduces bias from the order of exploration.

D. Start two fresh sessions, having each re-read the relevant source files before beginning. (Incorrect)

This wastes computation by repeating the expensive 23-file analysis instead of reusing already established context.

Question: 133 **Exam Heist**

Production monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, then invokes the document analysis subagent and waits again. These tasks are independent — neither requires the other's output. What is the most effective way to run these subagents concurrently?

A. Switch both subagents to use a Haiku-tier model instead of Sonnet to reduce their individual execution time.

B. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator.

C. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message.

D. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution at the same time.

Answer: C

Explanation:

C. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message. (Correct)

This enables true **concurrent execution at the tool-calling layer**. Since the web search and document analysis subagents are independent, issuing both tool calls in the same coordinator response allows the runtime to execute them in parallel instead of waiting for one to finish before starting the other. This directly removes the sequential bottleneck without changing model quality or adding unnecessary system complexity.

A. Switch both subagents to use a Haiku-tier model instead of Sonnet to reduce their individual execution time. (Incorrect)

This reduces per-task latency but does not address the core inefficiency, which is **sequential execution**, not model speed.

B. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator. (Incorrect)

This introduces unnecessary architectural complexity and duplicates coordination logic when parallelism can already be achieved within a single coordinator response.

D. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution at the same time. (Incorrect)

Prompt instructions alone cannot guarantee parallel execution; only structured multi-tool invocation can enforce concurrency.

Question: 134 **Exam Heist**

An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth — having the agent propose specific code changes for each — before deciding which to implement. What's the most effective way to structure this exploration?

A. Use fork_session to create two branches from yesterday's analysis, exploring one approach in each fork.

B. Resume yesterday's session and explore both approaches sequentially within the same conversation thread.

C. Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.

D. Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.

Answer: A

Explanation:

A. Use fork_session to create two branches from yesterday's analysis, exploring one approach in each fork.

Why this is the correct approach

When conducting deep structural comparisons (like architecture or code refactoring variations), the **forking pattern** is the cleanest way to manage agent sessions.

Isolating Context Contamination: Exploring complex code refactoring generation produces a massive trail of text, code blocks, and intermediate tool executions. If you do this sequentially in a single thread (Option B), the code proposals for the microservice approach will bleed into the context window for the in-place approach. This causes the agent to get confused, mix up patterns, and experience severe attention dilution.
Preserving Baseline Context without Overhead: Forking allows both branches to cleanly inherit all the heavy codebase discoveries, directory maps, and legacy module analyses built up yesterday. You skip the manual work and token waste of re-summarizing or re-reading the code (Options C and D), while giving each approach a clean, dedicated playground.

Why the other choices are Incorrect:

Option B leads to context pollution and limits: Forcing both deep code explorations into one thread will quickly bloat the context window. The agent will likely hit maximum token limits or begin hallucinating by mixing architectural constraints from both paradigms.
Option C and D introduce unnecessary human overhead: Manually copy-pasting summaries or trying to replicate an extensive exploration state across fresh sessions is inefficient. It risks missing subtle code dependencies that the agent found during its native session yesterday.

Question: 135 **Exam Heist**

An engineer asks your agent to add comprehensive tests to a legacy codebase with 200 files and minimal existing test coverage. The engineer hasn't spent much time on the codebase, so they ask for help identifying which modules to prioritize. How should the agent decompose this open-ended task?

A. Use Glob and Grep to map codebase structure, identify heavily-coupled modules, create a prioritized plan for high-impact areas, and revise as dependencies are discovered.

B. Create a fixed testing schedule upfront based on directory structure, allocating equal effort to each top-level directory regardless of code complexity or importance.

C. Systematically read all 200 files to create a complete function inventory before writing any tests, ensuring the testing plan accounts for everything from the beginning.

D. Start writing tests for the first module alphabetically, using test failures and imports to discover related files

organically.

Answer: A

Explanation:

A. Use Glob and Grep to map codebase structure, identify heavily-coupled modules, create a prioritized plan for high-impact areas, and revise as dependencies are discovered. (Correct)

This is the most effective decomposition strategy because it uses lightweight exploration tools first to understand structure and coupling, then prioritizes testing based on impact and dependency density. It is iterative and adapts as new relationships between modules are discovered, which is essential in large, poorly tested codebases.

B. Create a fixed testing schedule upfront based on directory structure, allocating equal effort to each top-level directory regardless of code complexity or importance. (Incorrect)

This ignores differences in module complexity, risk, and coupling. Treating all directories equally leads to inefficient allocation of testing effort and misses high-risk areas that need deeper coverage.

C. Systematically read all 200 files to create a complete function inventory before writing any tests, ensuring the testing plan accounts for everything from the beginning. (Incorrect)

While thorough, this is not scalable and delays actual testing work. It front-loads too much analysis and defeats the purpose of incremental discovery, especially in large legacy systems.

D. Start writing tests for the first module alphabetically, using test failures and imports to discover related files organically. (Incorrect)

This is an unstructured and arbitrary approach. Alphabetical ordering has no relation to system importance or dependency structure, so it risks focusing effort on low-impact areas first and missing critical modules early.

Question: 136 **Exam Heist**

Your automated review calls the Claude API for each PR, using tool_use with a report_findings tool that returns a JSON array of finding objects (each with file_path, line_number, severity, category, and description). During testing on a large PR touching 30+ files, the response hits the max_tokens limit and the output is truncated mid-JSON, causing your pipeline's parser to fail. What is the most effective way to handle this?

A. Increase max_tokens to the model's maximum and instruct Claude to keep finding descriptions under 50 words each.

B. Switch from tool_use to prompting Claude to return findings as a markdown list.

C. Split the review into multiple API calls that each analyze a subset of the changed files, then merge the resulting findings arrays.

D. Add retry logic that detects truncated JSON and re-sends the request with instructions to report only critical and high severity findings.

Answer: C

Explanation:

C. Split the review into multiple API calls that each analyze a subset of the changed files, then merge the resulting findings arrays. **Correct**

Why C is correct:

Breaking the PR into smaller batches keeps each JSON response within token limits, preventing truncation and ensuring valid, parseable output.

A. Increase max_tokens to the model's maximum and instruct Claude to keep finding descriptions under 50 words each Incorrect

May reduce truncation but does not guarantee the response will fit.

B. Switch from tool_use to prompting Claude to return findings as a markdown list Incorrect

Loses structured output and still does not solve token limit issues.

D. Add retry logic that detects truncated JSON and re-sends the request with instructions to report only critical and high severity findings Incorrect

Produces incomplete reviews by omitting lower-severity findings.

Question: 137 **Exam Heist**

You are setting up a non-interactive automated code review pipeline using Claude Code. You want Claude to analyze a pulled Git diff (git diff) against the main branch and apply a custom set of code review instructions. However, you notice that when you run the pipeline, Claude only looks at the raw diff text itself and completely stops using its file-reading or code navigation tools. As a result, it fails to inspect the broader codebase repository context, which is critical because the diff modifies a core function called by many other external modules.
Which change to the CLI invocation will cause Claude to read related files in the repository while still successfully applying your custom review instructions?

A. Replace --system-prompt with --append-system-prompt so your review instructions are added to Claude Code's default prompt instead of overwriting the built-in guidance for using file-reading and code navigation tools.

B. Keep --system-prompt and add --allowedTools "Read, Glob, Grep" so that the non-interactive mode permits file system tools that it otherwise disables.

C. Stop piping the diff via stdin and instead embed the diff contents inside the prompt string, so Claude Code treats the invocation as an agentic session rather than a stream-processing one.

D. Remove --system-prompt entirely and place the review instructions in a CLAUDE.md file at the repo root, since --system-prompt is incompatible with tool use under -p.

Answer: A

Explanation:

A. Replace --system-prompt with --append-system-prompt so your review instructions are added to Claude Code's default prompt instead of overwriting the built-in guidance for using file-reading and code navigation tools. Correct

Why A is correct:

--system-prompt replaces Claude Code's default system instructions, which include guidance for using file-reading and code navigation tools. --append-system-prompt preserves those built-in instructions while adding your custom review rules, allowing Claude to inspect related repository files.

B. Keep --system-prompt and add --allowedTools "Read, Glob, Grep" Incorrect

Tools may be permitted, but replacing the default system prompt can remove the instructions that tell Claude to use them.

C. Stop piping the diff via stdin and instead embed the diff contents inside the prompt string **Incorrect**

Changing how the diff is provided does not restore the missing tool-use guidance.

D. Remove --system-prompt entirely and place the review instructions in a CLAUDE.md file at the repo root **Incorrect**

--system-prompt is not inherently incompatible with tool use; the issue is overwriting the default prompt.

Question: 138 **Exam Heist**

Your development team is using Claude Code to automate test generation across a large codebase. However, developers are frequently rejecting the generated test suites because Claude creates a high volume of trivial assertions or tests that merely maximize line coverage without validating meaningful behavioral logic or edge cases.
You want to guide Claude to generate high-quality, production-ready tests directly without introducing high latency or modifying the core pipeline script.
Which strategy best ensures that high-quality, meaningful tests are generated in the first place?

A. Restrict test generation to directories where historical quality metrics show higher acceptance rates, disabling it for areas where generated tests consistently require heavy editing.

B. Add post-generation coverage analysis that automatically filters out any generated test that doesn't increase line coverage beyond what existing tests provide.

C. Document testing standards in CLAUDE.md including valuable test criteria, available fixtures with intended use cases, and examples distinguishing meaningful behavioral tests from trivial assertions.

D. Implement a two-phase generation where a second Claude call scores each test against quality criteria, filtering out low-scoring tests before presenting results to developers.

Answer: C

Explanation:

C. Document testing standards in CLAUDE.md including valuable test criteria, available fixtures with intended use cases, and examples distinguishing meaningful behavioral tests from trivial assertions. **Correct**

Why C is correct:

CLAUDE.md provides persistent guidance that shapes test generation from the start. By documenting quality standards, fixtures, and examples of meaningful behavioral tests, Claude is more likely to generate production-ready tests that validate logic and edge cases instead of chasing line coverage.

A. Restrict test generation to certain directories **Incorrect**

Limits where tests are generated but does not improve test quality.

B. Add post-generation coverage analysis **Incorrect**

Coverage increases do not guarantee meaningful or high-quality tests.

D. Implement a second scoring call **Incorrect**

Can filter poor tests but adds latency and modifies the pipeline, which the requirements explicitly avoid.

Question: 139 **Exam Heist**

A developer uses Claude Code to refactor a function during their development session. Before committing, they ask the same Claude session to review the code for issues. Later, a separate automated CI review catches several bugs that the same-session review missed. What best explains this discrepancy?

A. The CI environment has access to the full codebase context while the local session only sees the current file

B. The CI review uses a more specific prompt tailored for catching bugs, while the developer's request was too general

C. Claude retains context about its prior reasoning in the session, making it less likely to question its own decisions

D. The extended session length caused the context window to fill with conversation history, leaving less room for thorough analysis

Answer: C

Explanation:

C. Claude retains context about its prior reasoning in the session, making it less likely to question its own decisions. Correct

Why C is correct:

When Claude reviews code in the same session where it created or refactored the code, it retains its earlier reasoning and assumptions. This can lead to confirmation bias, making it less likely to critically re-evaluate its own changes. A fresh CI review starts with no prior attachment to those decisions and may identify issues that the same-session review misses.

A. The CI environment has access to the full codebase context while the local session only sees the current file Incorrect

Local sessions can also access the codebase using file-reading and navigation tools; this is not the primary reason for the discrepancy.

B. The CI review uses a more specific prompt tailored for catching bugs, while the developer's request was too general Incorrect

Prompt quality can affect results, but the key issue described is reviewing code within the same session that generated it.

D. The extended session length caused the context window to fill with conversation history, leaving less room for thorough analysis Incorrect

Long conversations may affect context management, but this is not the main explanation for missing bugs that a fresh review catches.

Question: 140 **Exam Heist**

Your automated reviewer uses a single prompt covering security issues, API design, and business logic correctness. Your evaluation suite shows strong recall for API design findings (82%) but poor recall for business logic edge cases in quiz scoring (34%). When you add few-shot examples of logic bugs to the prompt, logic recall improves to 41% but API design recall drops to 68%. How should you address this trade-off to improve detection across both categories?

A. Split the review into separate focused prompts — one for security and API design, another for business logic — each with dedicated examples, then consolidate findings before posting.

B. Replace the few-shot examples with a detailed checklist of specific logic edge cases to verify, such as division-by-zero In score calculations and boundary conditions in grading thresholds.

C. Upgrade to a more capable model tier, since its stronger reasoning will handle both concern types in a single prompt and eliminate the recall trade-off.

Answer: A

Explanation:

A. Split the review into separate focused prompts — one for security and API design, another for business logic — each with dedicated examples, then consolidate findings before posting. Correct

Why A is correct:

Combining multiple review objectives in a single prompt creates context competition, where adding examples for one category can reduce performance in another. Separate, focused prompts allow each review type to have its own instructions and examples, improving recall across categories without sacrificing performance.

B. Replace the few-shot examples with a detailed checklist of specific logic edge cases Incorrect

A checklist may help business logic detection but still forces multiple concerns into one prompt and does not eliminate the trade-off.

C. Upgrade to a more capable model tier Incorrect

A stronger model may improve overall performance, but it does not guarantee removal of prompt interference between different review objectives.

Question: 141 **Exam Heist**

Your codebase exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge. What approach best balances efficiency and accuracy?

A. Resume the session and immediately have the agent re-read all 12 previously analyzed files to ensure complete alignment.

B. Resume the session without informing the agent about the changed files, allowing it to naturally detect discrepancies during execution.

C. Resume the session and inform the agent which specific files changed for targeted re-analysis while retaining context on the unchanged files.

D. Start a fresh session to ensure the agent works with current codebase state without stale assumptions from the previous day.

Answer: C

Explanation:

C. Resume the session and inform the agent which specific files changed for targeted re-analysis while retaining context on the unchanged files. Correct

Why C is correct:

This approach preserves the valuable context built during the previous session while ensuring accuracy by re-analyzing only the files that changed. It balances efficiency and correctness by avoiding unnecessary re-reading of unchanged files and preventing stale assumptions.

A. Resume the session and immediately have the agent re-read all 12 previously analyzed files to ensure complete alignment. Incorrect

Ensures freshness but is inefficient because nine files remain unchanged.

B. Resume the session without informing the agent about the changed files, allowing it to naturally detect discrepancies during execution. Incorrect

Risks relying on outdated assumptions and missing the impact of recent changes.

D. Start a fresh session to ensure the agent works with current codebase state without stale assumptions from the previous day. Incorrect

Guarantees fresh context but discards an hour of useful analysis and is unnecessarily inefficient.

Question: 142 **Exam Heist**

An engineer sees an unfamiliar error message "SYNC_CONFLICT: entity version mismatch detected" in production logs but doesn't know which of the 12 services in the codebase generates it. They ask the agent to help locate the source code. What exploration approach will most efficiently find the responsible code?

A. Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.

B. Use Glob to find files in directories commonly associated with error handling (such as errors/, exceptions/, or handlers/) across services, then Read each matching file.

C. Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.

D. Read the project's README and service configuration files to understand the architecture, then systematically Read source files in service directory.

Answer: A

Explanation:

A. Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context. Correct

Why A is correct:

Searching for unique strings in the error message is the fastest and most direct way to locate the source code that generates the error. Grep quickly identifies matching files, and Read provides the surrounding context to determine which service is responsible.

B. Use Glob to find files in error-related directories Incorrect

Error messages may not be defined in dedicated error directories, making this approach inefficient and incomplete.

C. Use Grep to find all files that import the project's error handling module Incorrect

Many files may import the error module, creating numerous irrelevant matches and increasing investigation time.

D. Read the project's README and service configuration files **Incorrect**

Provides architectural context but is a slow, indirect approach for locating a specific error message.

Question: 143 **Exam Heist**

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search. When a user asks the agent to "create a branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

A. The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.

B. Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.

C. You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.

D. The agent automatically selects the most relevant server based on the request and loads only that server's tools.

Answer: B

Explanation:

B. Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent. **Correct**

Why B is correct:

When multiple MCP servers are configured, the agent discovers and loads tools from all connected servers during initialization. It can then use tools across servers in a single request — for example, using the Git server to create a branch, the Jira server to update the ticket, and the Documentation server to retrieve links.

A. The agent queries each server sequentially **Incorrect**

The agent does not probe servers one by one for every request or rely on tool name prefixes.

C. You must specify which MCP server to use for each turn **Incorrect**

The agent can access tools from multiple servers simultaneously and does not require manual server selection per turn.

D. The agent automatically selects one server and loads only that server's tools **Incorrect**

Tools from all configured servers remain available; the agent is not limited to a single server per request.

Question: 144 **Exam Heist**

Compliance requires that refunds exceeding $500 must automatically escalate to a human agent-this rule cannot be left to model discretio Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate) How should you achieve guaranteed compliance?

A. Modify the refund tool to return an error with message "Amount exceeds policy limit please escalate" when threshold is exceeded.

B. Implement a hook to intercept tool calls when the refund process amount exceeds $500, block it and invoke

human escalation.

C. Strengthen the system prompt with emphatic language: "CRITICAL POLICY! Refunds over $500 MUST trigger human escalation. NEVER process these directly!"

D. Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts ($40, $500, $600).

Answer: B

Explanation:

B. Implement a hook to intercept tool calls when the refund process amount exceeds $500, block it and invoke human escalation. Correct

Why B is correct:

Compliance requirements need deterministic enforcement, not model judgment. A hook that intercepts refund tool calls and blocks any refund over $500 guarantees that prohibited actions cannot occur and automatically triggers human escalation.

A. Modify the refund tool to return an error message Incorrect

An error message depends on the model interpreting and handling it correctly and does not provide guaranteed enforcement.

C. Strengthen the system prompt Incorrect

Prompts improve behavior but cannot guarantee 100% compliance, as shown by the existing 3% failure rate.

D. Add few-shot examples Incorrect

Examples may improve adherence but remain probabilistic and cannot enforce a hard compliance rule.

Question: 145 **Exam Heist**

Production logs reveal inconsistent error handling: when tool_code fails, the agent sometimes retries 5 times (even if the tool_id doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes adds user-friendly explanation (inappropriate when the issue is a backend permission error). Investigation shows four MCP tool returns uniform error responses: "status": "error", "content": " "type": "Error", "message": "Operation failed." " . The agent learns different types.
What's the most effective improvement?

A. Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.

B. Create an analyze_error MCP tool the agent calls after any failure to determine the error category and recommended action.

C. Add a few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.

D. Enhance error responses with structured metadata. Include error_category (transient/retriable/permission), reason, and a description of what caused the failure.

Answer: D

Explanation:

D. Enhance error responses with structured metadata. Include error_category

(transient/retriable/permission), reason, and a description of what caused the failure. **Correct**

Why D is correct:

The current error responses are generic ("Operation failed."), giving the agent no information to distinguish between transient network issues, invalid tool IDs, or permission errors. Structured metadata enables the agent to take the appropriate action — retry, escalate, or inform the user — consistently and correctly.

A. Implement retry logic with exponential backoff for all errors **Incorrect**

Not all errors are retriable. Retrying permission or invalid-tool errors wastes time and can worsen behavior.

B. Create an analyze_error MCP tool **Incorrect**

Adds extra complexity and latency. The root problem is that the original errors lack sufficient information.

C. Add few-shot examples to the system prompt **Incorrect**

Examples cannot reliably infer error types from identical, generic error messages.

Question: 146 **Exam Heist**

Production logs reveal inconsistent error handling. When looking up facts, the agent sometimes retries 5+ times (sadly, when the outer ID doesn't exist), sometimes escalates immediately (prematurely for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses ("isError": true, "context": "type": "text", "text": "Operation failed."). The agent cannot distinguish between error types.
What's the most effective improvement?

A. Implement retry logic with exponential backoff in your MCP service for all errors returning to the agent only after retries are exhausted.

B. Create an analyze_error MCP tool that the agent calls after any failure to determine the error category and recommended action.

C. Add ten shot examples to the system's conduct_demo training to interpret error message context and select appropriate responses for each.

D. Return structured error metadata in MCP responses.

Answer: D

Explanation:

D. Return structured error metadata in MCP responses. **Correct**

Why D is correct:

The agent receives the same generic error ("Operation failed.") for every failure, so it cannot determine whether it should retry, escalate, or inform the user. Returning structured metadata (for example, error_category, reason, retriable, and permission_denied) enables consistent and appropriate error handling.

A. Implement retry logic with exponential backoff for all errors **Incorrect**

Not all errors are retriable. Retrying invalid IDs or permission errors wastes time and may cause repeated failures.

B. Create an analyze_error MCP tool **Incorrect**

Adds extra complexity and latency. The root issue is insufficient information in the original error response.

C. Add ten-shot examples to training **Incorrect**

Examples cannot reliably infer different error types from identical, generic error messages.

Question: 147 **Exam Heist**

After expanding the agent's MCP tools with delivery-specific capabilities [apply_promo_code, update_delivery_address, reconcile_delivery], the total tool count has grown from 1 to 7. We have observed that the agent now shows tool selection accuracy has dropped from 86% to 71%. Log analysis reveals the majority of errors involve the agent selecting between semantically overlapping tools — calling issue_credit when process_refund was correct, and calling check_delivery_status when looking order_allready_returns_the_needed_data. Which approach structurally eliminate the semantic overlap identified in the error source?

A. Consolidate semantically overlapping tools – merge issue_credit and process_refund into a single handle_promotions tool with an action parameter and fold check_delivery_status into lookup_order with an optional include_tracking flag.

B. Enable the tool search tool with defer_loading on the six new tools, keeping the original two always loaded, so the agent dynamically discovers specialized tools only when needed.

C. Add few-shot examples to the system prompt demonstrating correct selection for each ambiguous tool, such as showing when issue_credit applies versus when process_refund is appropriate.

D. Split the tools across two sub-agents — a 'financial resolution' agent with issue_credit, process_refund, return_order, and apply_promo_code and a 'delivery operations' agent with the remaining delivery tools — with a coordinating routing between them.

Answer: A

Explanation:

A. Consolidate semantically overlapping tools – merge issue_credit and process_refund into a single handle_promotions tool with an action parameter and fold check_delivery_status into lookup_order with an optional include_tracking flag. **Correct**

Why A is correct:

The primary issue is **semantic overlap** between tools. When multiple tools perform similar functions or return overlapping information, the agent struggles to choose correctly. Consolidating overlapping tools into unified interfaces removes ambiguity and structurally improves tool selection accuracy.

B. Enable tool search with defer_loading **Incorrect**

Reduces the number of simultaneously loaded tools but does not eliminate the underlying semantic overlap.

C. Add few-shot examples **Incorrect**

Examples may improve selection temporarily, but they do not address the root cause of overlapping tool capabilities.

D. Split tools across two sub-agents **Incorrect**

Domain separation can help organization, but overlap still exists within domains (for example, issue_credit vs. process_refund), so ambiguity remains.

Question: 148 **Exam Heist**

You're implementing the escalation logic for when the agent should call a human. Your team processes for different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

A. Instruct the agent to escalate when the customer requests a human. When the issue requires policy exceptions, or the agent can not make meaningful progress.

B. Configure the agent to escalate after three consecutive tools fails to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.

C. Build a rules engine that maps specific issue types, customer segments and product categories to escalation decisions, removing the need for model judgment calls.

D. Implement sentiment analysis that monitors for frustration indicators (e.g., all-caps, repeated questions, exclamations) and trigger escalation when the frustration score exceeds a configured threshold.

Answer: A

Explanation:

A. Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress. Correct

Why A is correct:

This approach directly identifies situations that genuinely require human intervention. It covers customer preference, requests that need policy exceptions or discretionary decisions, and cases where the agent is unable to make meaningful progress. It provides a balanced and reliable escalation strategy.

B. Configure the agent to escalate after three consecutive tool failures Incorrect

Tool failures do not always mean a human is needed, and some cases require immediate escalation without waiting for multiple failures.

C. Build a rules engine that maps issue types, customer segments, and product categories to escalation decisions Incorrect

A rigid rules engine is difficult to maintain and cannot capture all situations that require human judgment or customer preference.

D. Implement sentiment analysis that triggers escalation based on frustration indicators Incorrect

Frustration may indicate dissatisfaction, but it is not a reliable indicator that human intervention is necessary.

Question: 149 **Exam Heist**

Your code-review prompts include both implementation changes and the corresponding test file, but the review comments fail to identify untested code paths. The model correctly flags functions that have no tests at all, but it fails to recognize when conditional branches or error-handling paths within tested functions lack coverage. What is the most effective way to improve branch-level gap detection without overcomplicating the pipeline?

A. Interleave the implementation and tests in the prompt, presenting each function immediately before its test cases.

B. Add explicit instructions requiring Claude to enumerate every conditional branch and exception path, then verify that each path has a corresponding test assertion.

C. Implement a two-pass pipeline in which one model call extracts all conditional branches and another cross-references them against test assertions.

D. Include few-shot examples showing code with an uncovered branch and the corresponding review comment identifying the missing test case.

Answer: B

Explanation:

A. Incorrect

Interleaving each function with its tests may improve readability, but it does not specifically teach the model to check whether every conditional branch or exception path is covered by tests.

B. Correct

Adding explicit instructions to enumerate every conditional branch and exception path, and then verify that each has a corresponding test assertion, directly addresses the branch-level coverage problem. This improves detection of missing tests for if/else branches, early returns, and error-handling paths without adding major pipeline complexity.

C. Incorrect

A two-pass pipeline can improve accuracy, but it introduces additional orchestration, latency, and cost. The question asks for the most effective improvement without overcomplicating the pipeline.

D. Incorrect

Few-shot examples can help the model learn the desired behavior, but they are less reliable than giving an explicit branch-checking instruction. The model may still miss branches in unfamiliar code patterns.

Question: 150 **Exam Heist**

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
Your automated review jobs take 18 seconds to initialize before Claude begins analyzing code. Profiling reveals that the delay results from automatically discovering hooks, MCP servers, plugins, skills, and multiple nested CLAUDE.md files throughout the monorepo.
You need to reduce startup time while ensuring reviews still enforce the coding standards documented in the root-level CLAUDE.md file.
What is the most effective approach?

A. Replace the default prompt using --system-prompt-file ./CLAUDE.md, which bypasses default prompt assembly and loads only the project rules.

B. Run in --bare mode and pass --append-system-prompt-file ./CLAUDE.md to load the required project standards explicitly while skipping automatic discovery.

C. Run in --bare mode and repeat all review criteria directly in the -p prompt for every invocation.

D. Keep the default initialization and add --exclude-dynamic-system-prompt-sections to improve prompt-cache reuse across CI runners.

Answer: B

Explanation:

A. Incorrect — Replace the default prompt entirely

Using --system-prompt-file replaces Claude Code's default system prompt and built-in review behavior. This may remove important coding-assistant instructions and reduce review quality. The goal is to keep the standard behavior while avoiding expensive automatic discovery.

B. Correct — Bare mode + append the root CLAUDE.md

--bare skips automatic discovery of hooks, MCP servers, plugins, skills, and nested CLAUDE.md files, which is the main source of the 18-second startup delay. Adding --append-system-prompt-file ./CLAUDE.md explicitly loads the root-level coding standards you still want enforced. This gives the best balance of fast startup, preserved default behavior, and consistent project standards.

C. Incorrect — Repeat rules in every prompt

This avoids discovery overhead, but it duplicates policy text in every invocation, increases maintenance burden, and makes prompts harder to keep consistent with the canonical CLAUDE.md.

D. Incorrect — Exclude dynamic sections only

This may help prompt-cache reuse, but it does not eliminate the expensive discovery of hooks, MCP servers, plugins, skills, and nested CLAUDE.md files. Startup time would improve far less than with --bare.

Final Answer

B. Run in --bare mode and pass --append-system-prompt-file ./CLAUDE.md to load the required project standards explicitly while skipping automatic discovery.

Question: 151 **Exam Heist**

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.
An engineer submits two requests:
Request A: “Rename the getUserData function to fetchUserProfile everywhere it’s used.”
Request B: “Improve error handling throughout the data processing module — add try/catch blocks, meaningful error messages, and ensure failures don’t silently corrupt data.”
For which request does specifying an explicit multi-phase workflow (such as analyze → propose → implement with review) most improve outcome quality?

A. Neither request benefits significantly
B. Request A, the function rename task
C. Both requests benefit equally
D. Request B, the error handling task

Answer: D

Explanation:

A. Incorrect Neither request benefits significantly

Request B is a complex, judgment-heavy change that benefits substantially from a structured workflow.

B. Incorrect Request A, the function rename task

A global function rename is usually a relatively mechanical refactor. Tools like Grep, language-aware rename features, and tests often make this straightforward.

C. Incorrect Both requests benefit equally

While both can benefit from planning, the improvement in quality is much greater for Request B.

D. Correct Request B, the error handling task

Improving error handling requires understanding control flow, identifying failure points, preserving data integrity, avoiding unintended side effects, and designing consistent error messages and recovery behavior. A multi-phase workflow such as analyze → propose → implement → review helps the agent reason about the module before making broad changes, leading to higher-quality and safer results.

Final Answer: D

Question: 152 **Exam Heist**

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
A developer uses Claude Code to refactor a function during a development session. Before committing, the developer asks the same Claude session to review the code for issues. Later, a separate automated CI review catches several bugs that the same-session review missed.
What best explains this discrepancy?

A. Claude retains the implementation context and prior decisions in the session, making it less likely to challenge assumptions underlying its own changes.

B. The session's context window necessarily became full, leaving insufficient capacity for meaningful review.

C. The CI review must have used a more specific prompt, while the developer's review request was too general.

D. The CI environment can access the full repository, while a local Claude Code session can access only the current file.

Answer: A

Explanation:

A. Correct Claude retains the implementation context and prior decisions in the session, making it less likely to challenge assumptions underlying its own changes.

When the same Claude session performs both the refactor and the review, it carries forward the reasoning, assumptions, and implementation choices that led to the changes. This can reduce the likelihood of critically questioning those decisions, making the review less independent.

B. Incorrect The session's context window necessarily became full, leaving insufficient capacity for meaningful review.

A full context window is not the most likely explanation, and the question does not provide evidence that the context limit was reached.

C. Incorrect The CI review must have used a more specific prompt, while the developer's review request was too general.

Prompt quality can affect results, but the key distinction in the scenario is the difference between a same-session self-review and a separate automated review.

D. Incorrect The CI environment can access the full repository, while a local Claude Code session can access only the current file.

Claude Code can access multiple files and the repository context when used locally; it is not limited to the current file.

Final Answer: A

Question: 153 **Exam Heist**

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
Your pipeline includes a release-notes generation step that classifies and summarizes approximately 200 commits at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API request using a Sonnet-tier Claude model. The release notes are not needed until the following morning, providing approximately 12 hours of acceptable latency.
Your team must reduce the per-token API cost while retaining the same model, prompts, and output quality.
Which approach satisfies all these constraints?

A. Issue the 200 Messages API requests concurrently because parallel execution reduces the per-token price.

B. Submit the 200 requests through the Message Batches API with unique custom_id values and retrieve the results after the batch finishes.

C. Concatenate all 200 commit messages into one Messages API request because reducing the number of requests always reduces token costs.

D. Replace the Sonnet-tier model with a Haiku-tier model to obtain a lower per-token price.

Answer: B

Explanation:

A. Incorrect Issue the 200 Messages API requests concurrently because parallel execution reduces the per-token price.

Concurrency may reduce total wall-clock time, but it does not reduce the per-token API cost.

B. Correct Submit the 200 requests through the Message Batches API with unique custom_id values and retrieve the results after the batch finishes.

The Message Batches API is designed for large numbers of independent requests where latency is not critical. It allows you to keep the same model, prompts, and output quality while reducing cost for high-volume asynchronous workloads such as release-note generation.

C. Incorrect Concatenate all 200 commit messages into one Messages API request because reducing the number of requests always reduces token costs.

Combining all commits into one request changes the task structure, can affect output quality, may increase context complexity, and does not guarantee lower token usage.

D. Incorrect Replace the Sonnet-tier model with a Haiku-tier model to obtain a lower per-token price.

This violates the requirement to retain the same model and preserve output quality.

Final Answer: B

Question: 154 **Exam Heist**

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.
Your extraction system processes two document types: standard monthly reports, which are archived after processing, and urgent exception reports, which must trigger business alerts within 30 minutes of receipt. Both

use the same JSON schema. You want to minimize API costs while meeting the latency requirements.
How should you architect the processing pipeline?

A. Submit all documents to the Message Batches API with custom_id values for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

B. Route standard reports to the Message Batches API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

C. Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.

D. Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.

Answer: B

Explanation:

A. Incorrect Submit all documents to the Message Batches API with custom_id values for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

The Message Batches API is asynchronous and does not guarantee the sub-30-minute latency required for urgent exception reports.

B. Correct Route standard reports to the Message Batches API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

This approach satisfies both requirements:

Standard monthly reports: low urgency, so use the lower-cost batch processing path.
Urgent exception reports: require alerts within 30 minutes, so use the real-time API.

The same JSON schema can be applied in both paths, preserving consistency while optimizing cost.

C. Incorrect Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.

Hourly batching can easily violate the 30-minute alert requirement.

D. Incorrect Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.

This meets the latency requirement but does not minimize API costs, which is a stated goal.

Final Answer: B

Question: 155 **Exam Heist**

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.
Testing reveals that when source documents are missing certain specifications, the model fabricates plausible-sounding values to satisfy your schema's required fields. For example, a document mentioning only dimensions receives a fabricated "weight: 2.3 kg" in the extraction output.
What schema design change most effectively addresses this hallucination behavior?

A. Add explicit instructions to the prompt stating "only extract information explicitly stated in the document; use placeholder text for missing values."

B. Change fields that may not exist in source documents from required to optional, allowing the model to omit

them.

C. Add a “confidence” field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions.

D. Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text.

Answer: B

Explanation:

A. Incorrect Add explicit instructions to the prompt stating “only extract information explicitly stated in the document; use placeholder text for missing values.”

Prompt instructions help, but they do not remove the structural pressure created by a schema that requires values for fields that may not exist.

B. Correct Change fields that may not exist in source documents from required to optional, allowing the model to omit them.

The hallucination occurs because the schema requires a value for every field. Making potentially absent fields optional removes the incentive to fabricate plausible values and allows the extraction to accurately represent missing information.

C. Incorrect Add a “confidence” field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions.

Confidence scores are not a reliable mechanism for preventing hallucinations, and fabricated values may still be reported with high confidence.

D. Incorrect Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text.

This is a useful downstream safeguard, but it is not the most effective schema design change to prevent the behavior at its source.

Final Answer: B

Question: 156 **Exam Heist**

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.
Your extraction pipeline occasionally receives responses that cannot be parsed as valid JSON, causing downstream processing failures. The current implementation prompts Claude to return JSON in the response text and then parses it.
What is the most reliable approach to ensure Claude returns valid, schema-compliant structured data?

A. Add explicit formatting instructions to the prompt with JSON examples, emphasizing that Claude must return only valid JSON with no surrounding text.

B. Use regular expressions to locate and extract JSON from the response text, handling cases where Claude includes explanatory text around the JSON block.

C. Define a tool with a JSON schema specifying the expected structure, using tool use to constrain Claude’s output to schema-compliant JSON.

D. Implement a retry loop that catches JSON parsing errors and re-prompts Claude with the error details, asking it to correct the malformed output.

Answer: C

Explanation:

A. Incorrect Add explicit formatting instructions to the prompt with JSON examples, emphasizing that Claude must return only valid JSON with no surrounding text.

This improves reliability, but the model can still occasionally produce malformed or non-compliant JSON.

B. Incorrect Use regular expressions to locate and extract JSON from the response text, handling cases where Claude includes explanatory text around the JSON block.

Regex extraction is brittle and does not guarantee schema compliance.

C. Correct Define a tool with a JSON schema specifying the expected structure, using tool use to constrain Claude's output to schema-compliant JSON.

Tool use with a JSON schema is the most reliable approach because it constrains the model to return structured arguments that match the defined schema, greatly reducing parsing failures and improving downstream integration.

D. Incorrect Implement a retry loop that catches JSON parsing errors and re-prompts Claude with the error details, asking it to correct the malformed output.

Retries can recover from some failures, but they add latency, cost, and complexity and are less reliable than schema-constrained tool output.

Final Answer: C

Question: 157 **Exam Heist**

You are building developer-productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools — Read, Write, Bash, Grep, and Glob — and integrates with Model Context Protocol (MCP) servers.
Your productivity agent connects to three MCP servers: an issue tracker with search_issues , get_issue , and create_comment ; a documentation wiki with search_docs , get_page , and list_spaces ; and a database explorer with run_query , get_schema , and list_databases . For cross-system questions such as, "Which database tables are affected by the authentication refactor in PROJ-1234?", the agent makes eight to ten sequential exploratory calls, lacks visibility into each server's available content, and exhausts context before completing complex investigations.
What architectural change best leverages MCP capabilities to address these problems?

A. Add a prepare_investigation tool to each server that accepts a natural-language question and returns relevant summaries.

B. Consolidate all three systems into one unified MCP server with cross-referencing capabilities.

C. Expose each server's content catalog as MCP resources, including issue summaries, documentation hierarchies, and database schemas.

D. Add an orchestrator that routes each question to one server based on keywords.

Answer: C

Explanation:

A. Incorrect Add a prepare_investigation tool to each server that accepts a natural-language question and returns relevant summaries.

This may reduce some calls, but it does not solve the core problem of discoverability and context-efficient exploration across systems.

B. Incorrect Consolidate all three systems into one unified MCP server with cross-referencing capabilities.

This is a heavy operational change and is not necessary to take advantage of MCP's native strengths.

C. Correct Expose each server's content catalog as MCP resources, including issue summaries, documentation hierarchies, and database schemas.

MCP resources are designed for discoverable, structured context. Exposing issue summaries, wiki structure, and database schemas allows the agent to understand what content exists before making targeted tool calls, reducing exploratory requests and preserving context for complex investigations.

D. Incorrect Add an orchestrator that routes each question to one server based on keywords.

Cross-system questions inherently require information from multiple servers, so single-server routing would miss important relationships.

Final Answer: C

Question: 158 **Exam Heist**

The coordinator provides detailed step-by-step instructions to the web-search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports "insufficient results" instead of trying alternative approaches when the specified searches fail, (2) research quality drops for emerging topics that do not match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources. What is the most effective way to improve subagent adaptability?

A. Specify research objectives and quality criteria — such as coverage breadth, source diversity, and recency — rather than prescribing procedural steps, allowing the subagent to determine its search strategy.

B. Remove procedural details entirely and delegate using simple goals such as "research this topic thoroughly," relying on the subagent's general capabilities.

C. Add fallback directives requiring alternative query formulations whenever the specified searches produce fewer than a predetermined number of results.

D. Classify each topic as either "well-defined" or "exploratory" and use a different instruction style for each category.

Answer: A

Explanation:

A. Correct Specify research objectives and quality criteria — such as coverage breadth, source diversity, and recency — rather than prescribing procedural steps, allowing the subagent to determine its search strategy.

This directly addresses all three problems:

The subagent can adapt when initial searches fail.
It can explore emerging topics that do not fit predefined query patterns.
It is encouraged to surface valuable tangential sources through broader, goal-oriented exploration.

B. Incorrect Remove procedural details entirely and delegate using simple goals such as "research this topic thoroughly," relying on the subagent's general capabilities.

This gives too little guidance and may reduce consistency and quality control.

C. Incorrect Add fallback directives requiring alternative query formulations whenever the specified searches produce fewer than a predetermined number of results.

This improves robustness for failed searches but does not solve the broader adaptability and source-discovery issues.

D. Incorrect Classify each topic as either "well-defined" or "exploratory" and use a different instruction style for each category.

This can help in some cases, but it is less effective than shifting from procedural control to objective-based delegation across all research tasks.

Final Answer: A

Question: 159 **Exam Heist**

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web-search and document-analysis agents did not find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

A. Have the coordinator evaluate the synthesis output for gaps, then redelegate targeted queries to the web-search and document-analysis agents before invoking synthesis again.

B. Have the report-generation agent identify unanswered research questions so users understand the limitations of the final output.

C. Increase the initial breadth of queries sent to the web-search and document-analysis agents to reduce the probability of missing relevant information.

D. Give the synthesis agent direct access to web-search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

Answer: A

Explanation:

A. Correct Have the coordinator evaluate the synthesis output for gaps, then redelegate targeted queries to the web-search and document-analysis agents before invoking synthesis again.

This creates an iterative research loop: synthesize → identify gaps → perform targeted follow-up research → resynthesize. It directly improves completeness while preserving clear separation of responsibilities between coordinator, retrieval agents, and synthesis.

B. Incorrect Have the report-generation agent identify unanswered research questions so users understand the limitations of the final output.

This improves transparency but does not improve research completeness.

C. Incorrect Increase the initial breadth of queries sent to the web-search and document-analysis agents to reduce the probability of missing relevant information.

Broader initial searches may help somewhat, but they increase cost and noise and still cannot guarantee coverage of specific missing subtopics.

D. Incorrect Give the synthesis agent direct access to web-search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

This blurs responsibilities and reduces orchestration control. Coordinators are better suited to managing iterative retrieval and synthesis workflows.

Final Answer: **A**

Question: 160 **Exam Heist**

In addition to your CI pipeline, your organization has enabled Claude's managed Code Review through the Claude GitHub App on this repository, and reviews run automatically on every pull request. Reviews average 18 findings per pull request. Developer feedback reveals three categories of unwanted noise: (1) style and formatting issues already enforced by your CI linter, (2) findings on automatically generated template code under src/gen/*, and (3) rendering-helper patterns that are intentional project conventions but are flagged because they resemble common anti-patterns. Only approximately four findings per pull request are genuine logic bugs.
What is the most effective way to reduce this noise while preserving the detection of real issues?

A. Create a REVIEW.md file at the repository root containing skip rules for CI-enforced checks and generated files, together with a verification requirement that rendering-related findings cite a specific line demonstrating incorrect behavior.

B. Configure separate GitHub Actions workflow files for each code area: one for generated code with findings suppressed, one for rendering code with custom instructions, and one general workflow for everything else.

C. Add custom review instructions to a GitHub Actions workflow file, using the action's prompt parameter to suppress duplicate lint findings, ignore generated template code, and impose stricter evidence requirements on rendering-related issues.

D. Add detailed explanations to the project's CLAUDE.md describing intentional patterns, stating that CI handles linting, and identifying src/gen/ as automatically generated code.

Answer: A

Explanation:

A. Correct Create a REVIEW.md file at the repository root containing skip rules for CI-enforced checks and generated files, together with a verification requirement that rendering-related findings cite a specific line demonstrating incorrect behavior.

This is the most effective option because it gives the managed review system review-specific instructions:

ignore issues already enforced by CI,
skip generated code under src/gen/*,
and require stronger evidence before flagging intentional rendering patterns.

These targeted review constraints reduce noise while preserving detection of real logic bugs.

B. Incorrect Configure separate GitHub Actions workflow files for each code area: one for generated code with findings suppressed, one for rendering code with custom instructions, and one general workflow for everything else.

This adds workflow complexity and does not directly control the behavior of the managed code review system.

C. Incorrect Add custom review instructions to a GitHub Actions workflow file, using the action's prompt parameter to suppress duplicate lint findings, ignore generated template code, and impose stricter evidence requirements on rendering-related issues.

This applies to a workflow-based review integration, not the managed review configuration described in the question.

D. Incorrect Add detailed explanations to the project's CLAUDE.md describing intentional patterns, stating that CI handles linting, and identifying src/gen/ as automatically generated code.

CLAUDE.md provides general repository guidance, but it is less precise for controlling review-time suppression and evidence requirements than a dedicated review-configuration file.

Final Answer: A

Question: 161 **Exam Heist**

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.
Your team has connected a custom MCP server that provides DevOps workflow templates. The server exposes several MCP prompts (such as deploy_checklist and incident_response) in addition to tools.
How do these MCP prompts become accessible within Claude Code?

A. They are automatically prepended to every conversation as additional system-level context, influencing Claude's behavior throughout the session.

B. They are added to Claude Code's tool registry alongside the server's tools, invoked automatically by the model when relevant to the task.

C. They are surfaced as @ -mentionable resources alongside files, fetched and attached to your message when referenced.

D. They appear as slash commands (e.g., /mcp__servername__deploy_checklist) that you can invoke, with arguments passed after the command name.

Answer: D

Explanation:

A. Incorrect They are automatically prepended to every conversation as additional system-level context, influencing Claude's behavior throughout the session.

MCP prompts are not injected automatically into every conversation.

B. Incorrect They are added to Claude Code's tool registry alongside the server's tools, invoked automatically by the model when relevant to the task.

MCP tools can be invoked by the model, but MCP prompts are not tools.

C. Incorrect They are surfaced as @-mentionable resources alongside files, fetched and attached to your message when referenced.

This describes resource attachment behavior, not MCP prompts.

D. Correct They appear as slash commands (e.g., /mcp__servername__deploy_checklist) that you can invoke, with arguments passed after the command name.

In Claude Code, MCP prompts are exposed as slash commands. You invoke them explicitly, and any arguments are provided after the command name.

Question: 162 **Exam Heist**

Your pipeline includes a release-notes generation step that classifies and summarizes approximately 200 commits

at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API call using a Sonnet-tier Claude model. The release notes are not needed until the following morning, so results have approximately 12 hours of acceptable latency. Your team needs to reduce per-token API cost for this step while keeping the same model and prompts, with no change to the model tier or output quality.
Which approach satisfies all these constraints?

A. Concatenate all 200 commit messages into a single Messages API request and have the model return all summaries in one response, because fewer requests always reduce total token cost.

B. Issue the 200 Messages API requests in parallel using concurrent connections, because concurrency lowers the per-token price charged by the API.

C. Submit the 200 requests to the Message Batches API with unique custom_id values and retrieve the results after the batch finishes, which applies a 50% discount to all input and output tokens.

D. Switch the summarization calls from the Sonnet-tier model to a Haiku-tier model to take advantage of Haiku's lower per-token rates.

Answer: C

Explanation:

A. Incorrect Concatenate all 200 commit messages into a single Messages API request and have the model return all summaries in one response, because fewer requests always reduce total token cost.

Fewer requests do not guarantee lower token usage. Combining all commits can increase context complexity, reduce output quality, and may use a similar or larger number of tokens overall.

B. Incorrect Issue the 200 Messages API requests in parallel using concurrent connections, because concurrency lowers the per-token price charged by the API.

Concurrency can reduce total processing time, but it does not change the per-token pricing.

C. Correct Submit the 200 requests to the Message Batches API with unique custom_id values and retrieve the results after the batch finishes, which applies a 50% discount to all input and output tokens.

The Message Batches API is designed for large asynchronous workloads where latency is not critical. It preserves the same model, prompts, and output quality while reducing cost through the batch pricing discount.

D. Incorrect Switch the summarization calls from the Sonnet-tier model to a Haiku-tier model to take advantage of Haiku's lower per-token rates.

This violates the requirement to keep the same model tier and maintain the same output quality.

Question: 163 **Exam Heist**

You are building developer-productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools — Read, Write, Bash, Grep, and Glob — and integrates with Model Context Protocol (MCP) servers.
Engineers frequently ask the agent to cross-reference code changes with Jira tickets during reviews — checking ticket descriptions, acceptance criteria, and recent comments. This currently requires manually copying and pasting content into conversations. The team wants the agent to access this standard Jira ticket data directly.
What is the most effective approach?

A. Use the Bash tool with curl to call Jira's REST API, including authentication headers and parsing JSON responses inline.

B. Build a custom MCP server wrapping Jira's API with tools designed specifically for this team's code-review workflow.

C. Export Jira tickets to Markdown files in the repository that the agent accesses using the Read tool.

D. Integrate an existing Jira MCP server that exposes tickets, comments, and metadata through discoverable tool interfaces.

Answer: D

Explanation:

A. Incorrect Use the Bash tool with curl to call Jira's REST API, including authentication headers and parsing JSON responses inline.

This works, but it is brittle, requires manual authentication handling, and does not provide discoverable, reusable interfaces for the team.

B. Incorrect Build a custom MCP server wrapping Jira's API with tools designed specifically for this team's code-review workflow.

A custom server is appropriate only if existing solutions do not meet the need. The question asks for the most effective approach for standard Jira ticket access.

C. Incorrect Export Jira tickets to Markdown files in the repository that the agent accesses using the Read tool.

This creates synchronization overhead and loses live ticket updates, comments, and metadata.

D. Correct Integrate an existing Jira MCP server that exposes tickets, comments, and metadata through discoverable tool interfaces.

This is the best fit for the Agent SDK and MCP architecture. It provides structured, discoverable access to Jira data, avoids manual copy/paste, and enables the agent to retrieve ticket descriptions, acceptance criteria, comments, and metadata directly during code reviews.

Question: 164 **Exam Heist**

A user expands the research system beyond its original web-search agent by adding specialized data sources. A financial API agent returns structured JSON containing revenue, margins, and growth rates. A news-monitoring agent returns prose summaries of recent developments. A patent-analysis agent returns structured lists of technology areas.
The synthesis agent combines these results into executive briefings. Currently, it converts everything into bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose their narrative flow. What change would most improve briefing quality?

A. Standardize all subagent outputs as prose summaries with inline citations.

B. Standardize all subagent outputs as JSON containing fields for claim, evidence, source, and confidence.

C. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose, and technology areas as structured lists.

D. Add a format-conversion layer that transforms every subagent result into a common intermediate representation before synthesis.

Answer: C

Explanation:

A. Incorrect Standardize all subagent outputs as prose summaries with inline citations.

This would reduce the clarity of structured financial data and technology lists.

B. Incorrect Standardize all subagent outputs as JSON containing fields for claim, evidence, source, and confidence.

Useful for validation and traceability, but it does not solve the presentation-quality problem in executive briefings.

C. Correct Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose, and technology areas as structured lists.

Different information types have different optimal representations:

Financial data: tables for comparison
News: narrative prose for flow and context
Technology areas: structured lists for scanability

Adapting rendering to content type preserves the strengths of each source and produces higher-quality executive briefings.

D. Incorrect Add a format-conversion layer that transforms every subagent result into a common intermediate representation before synthesis.

A common internal representation may help orchestration, but it does not inherently improve the final presentation quality.

Question: 165 **Exam Heist**

Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document-analysis agent frequently identifies evidence gaps — for example, noting that "the retrieved sources discuss API authentication but lack details about token-refresh patterns."
Under the current strict pipeline, this insight is not actionable because searching has already finished.
What is the most effective architectural change?

A. Add a research-planning agent before the initial search phase to decompose every topic into detailed subquestions.

B. Have the synthesis agent assign confidence scores to each report section and flag insufficiently supported sections for manual review.

C. Require the analysis agent to return specific evidence gaps to the coordinator, which launches targeted searches and invokes analysis again until the defined coverage criteria are satisfied

D. Have the coordinator look for general gap indicators in the analysis output and run additional searches without repeating the analysis stage.

Answer: C

Explanation:

A. Incorrect Add a research-planning agent before the initial search phase to decompose every topic into detailed subquestions.

This improves initial coverage but cannot address new evidence gaps discovered during analysis.

B. Incorrect Have the synthesis agent assign confidence scores to each report section and flag insufficiently supported sections for manual review.

This increases transparency but does not automatically improve report depth.

C. Correct Require the analysis agent to return specific evidence gaps to the coordinator, which launches targeted searches and invokes analysis again until the defined coverage criteria are satisfied.

This creates a closed-loop iterative research architecture:

analyze retrieved evidence,
identify precise missing information,
perform targeted follow-up searches,
reanalyze,
repeat until coverage requirements are met.

It directly converts analysis insights into actionable retrieval.

D. Incorrect Have the coordinator look for general gap indicators in the analysis output and run additional searches without repeating the analysis stage.

Without reanalysis, the system cannot determine whether the new evidence actually resolves the identified gaps.

Question: 166 **Exam Heist**

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer , lookup_order , process_refund , escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.
Your process_refund tool returns two types of errors: technical errors (“503 Service Unavailable”, “Connection timeout”) that are transient (~5% of calls), and business errors (“Order exceeds 30-day return window”, “Item already refunded”) that are permanent (~12% of calls). Monitoring shows the agent wastes 3–4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude.
What’s the most effective way to reduce wasted retries while improving customer-facing response quality?

A. Implement automatic retry logic at the tool layer for technical errors only, passing business errors to Claude without retries.

B. Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text.

C. Add a check_refund_eligibility tool that must be called before process_refund to prevent business rule violations.

D. Return structured error responses with " retriable " : false for business errors and a customer-friendly explanation for Claude to use.

Answer: D

Explanation:

A. Incorrect Implement automatic retry logic at the tool layer for technical errors only, passing business errors to Claude without retries.

This helps with transient failures, but Claude still receives unstructured business-error text and may continue retrying unnecessarily.

B. Incorrect Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text.

Parsing free-form error text is brittle and less reliable than explicit machine-readable metadata.

C. Incorrect Add a check_refund_eligibility tool that must be called before process_refund to prevent

business rule violations.

This may reduce some business errors, but it adds an extra call to every refund request and does not solve the general problem of distinguishing retriable and non-retriable failures.

D. Correct Return structured error responses with "retriable": false for business errors and a customer-friendly explanation for Claude to use.

Structured errors give the agent explicit control-flow information:

Technical errors: retriable: true
Business errors: retriable: false

Including a customer-friendly explanation allows Claude to respond clearly and empathetically without wasting turns on impossible retries.

Question: 167 **Exam Heist**

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
During initial testing of the automated review pipeline, you notice that reviews on large pull requests containing more than 50 changed files sometimes take over 20 minutes and cost $8–$12 per run because of extensive agentic loops. Claude reads files, runs analysis tools, and iterates many times. Your team needs each invocation to abort once it reaches both a fixed iteration count and a fixed dollar amount, enforced by Claude Code itself rather than by the surrounding job runner.
Which configuration change directly enforces both per-invocation caps?

A. Switch the --model flag to a smaller, less expensive model so each iteration uses fewer tokens and has a lower per-call cost.

B. Set timeout-minutes: 5 on the GitHub Actions job step and monitor per-run costs through the Anthropic Console usage dashboard.

C. Add --max-turns 10 --max-budget-usd 2.00 to the claude -p invocation to cap iterations and spending.

D. Set --permission-mode dontAsk to automatically deny tool-permission requests not included in the explicitly allowed set.

Answer: C

Explanation:

A. Incorrect Switch the --model flag to a smaller, less expensive model so each iteration uses fewer tokens and has a lower per-call cost.

This may reduce the average cost per run, but it does not enforce hard limits on either the number of iterations or the total amount spent.

B. Incorrect Set timeout-minutes: 5 on the GitHub Actions job step and monitor per-run costs through the Anthropic Console usage dashboard.

A GitHub Actions timeout stops the workflow externally. The question specifically requires the limits to be enforced by Claude Code itself.

C. Correct Add --max-turns 10 --max-budget-usd 2.00 to the claude -p invocation to cap iterations and spending.

These flags directly enforce a maximum number of agentic turns and a maximum dollar budget for that

specific invocation, which satisfies both requirements.

D. Incorrect Set --permission-mode dontAsk to automatically deny tool-permission requests not included in the explicitly allowed set.

This controls tool-permission behavior, not the number of iterations or the spending limit.

Question: 168 **Exam Heist**

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers. Your code review assistant needs to analyze pull requests and provide feedback on three aspects: code style compliance, potential security issues, and documentation completeness. Each aspect requires reading files, running analysis tools, and generating a report section. The review process follows the same three-step workflow for every PR.
Which task decomposition pattern is most appropriate for this workflow?

A. Single comprehensive prompt — include all three instructions in one prompt and let the model handle all three aspects simultaneously.

B. Orchestrator-workers — have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask.

C. Prompt chaining — break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.

D. Routing — classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category.

Answer: C

Explanation:

A. Incorrect Single comprehensive prompt — include all three instructions in one prompt and let the model handle all three aspects simultaneously.

This is simpler, but it reduces structure and makes it harder to manage tool usage, intermediate results, and consistent output formatting across repeated reviews.

B. Incorrect Orchestrator-workers — have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask.

This pattern is useful when the required subtasks vary from one request to another. Here, the workflow is fixed and always includes the same three analyses.

C. Correct Prompt chaining — break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.

Prompt chaining is the best fit for a deterministic, repeatable multi-step workflow. Each step can read files, run the relevant tools, and produce a focused result, followed by a final synthesis that combines the three sections into one review report.

D. Incorrect Routing — classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category.

Routing is useful when different request types require different workflows, but the question states that every PR follows the same three-step review process.

Question: 169 **Exam Heist**

An AI agent running in Claude Code (or similar agentic harness with tools like Read, Write, Bash, Grep, Glob, and Model Context Protocol servers) is investigating a large payment processing module spanning 45 files. After reading the first 8 source files, the agent's responses are becoming noticeably less accurate — it is forgetting previously discussed code patterns and hasn't yet located all test files or traced critical payment flows.
What is the most effective approach to complete this investigation without exceeding context limits or causing context degradation?

A. Clear context with /clear, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets.

B. Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation.

C. Spawn subagents (or sub-tasks) to investigate specific questions (e.g., "find all test files for payment processing," "trace refund flow dependencies") while the main agent coordinates findings and preserves high-level context.

D. Continue reading the remaining files in the main session, relying on Claude's extended context window to auto-truncate earlier messages as new files are loaded.

Answer: C

Explanation:

A. Incorrect Clear context with /clear, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets.

This can help in some situations, but it interrupts the investigation and requires repeated re-reading of files, which is inefficient for a large, multi-question codebase analysis.

B. Incorrect Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation.

A summary alone is usually too lossy for complex investigations involving control flow, dependencies, and tests spread across many files.

C. Correct Spawn subagents (or sub-tasks) to investigate specific questions (e.g., "find all test files for payment processing," "trace refund flow dependencies") while the main agent coordinates findings and preserves high-level context.

This is the most effective approach. Specialized subagents can explore different parts of the codebase independently, keeping each context focused and small. The main agent maintains the overall investigation state and integrates the findings without overloading a single context window.

D. Incorrect Continue reading the remaining files in the main session, relying on Claude's extended context window to auto-truncate earlier messages as new files are loaded.

Allowing earlier information to be truncated is exactly what causes the observed degradation in accuracy and loss of important patterns.

Question: 170 **Exam Heist**

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks using built-in tools (Read, Write, Bash, Grep, Glob) and Model Context Protocol (MCP) servers.
Your agent has spent significant context exploring the rendering subsystem — reading shader code, buffer

management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific VulkanPipeline and FrameGraph classes it discovered earlier.
What's the most effective approach to handle this transition while preserving accuracy?

A. Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.

B. Use /clear to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.

C. Continue in the current context with more targeted prompts referencing the specific classes by name.

D. Summarize key rendering findings (including specific class names and architectural patterns), clear or reset the main context window (or delegate to a sub-agent seeded with that summary), then proceed with the physics integration exploration.

Answer: D

Explanation:

A. Incorrect Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.

This helps with separation of concerns, but it does not address the fact that the main conversation is already suffering from context degradation.

B. Incorrect Use /clear to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.

Resetting without preserving the important rendering discoveries risks losing the specific VulkanPipeline and FrameGraph knowledge needed for accurate integration analysis.

C. Incorrect Continue in the current context with more targeted prompts referencing the specific classes by name.

The symptoms indicate that the current context is already becoming degraded. Adding more targeted prompts may not reliably restore the lost precision.

D. Correct Summarize key rendering findings (including specific class names and architectural patterns), clear or reset the main context window (or delegate to a sub-agent seeded with that summary), then proceed with the physics integration exploration.

This preserves the high-value, project-specific rendering knowledge while freeing context capacity for the new investigation. Seeding the next exploration with a concise architectural summary maintains accuracy across the subsystem transition.

Question: 171 **Exam Heist**

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks using built-in tools (Read, Write, Bash, Grep, Glob) and Model Context Protocol (MCP) servers.
Your agent has spent significant context exploring the rendering subsystem — reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific VulkanPipeline and FrameGraph classes it discovered earlier.
What's the most effective approach to handle this transition while preserving accuracy?

A. Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.

B. Use /clear to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.

C. Continue in the current context with more targeted prompts referencing the specific classes by name.

D. Summarize key rendering findings (including specific class names and architectural patterns), clear or reset the main context window (or delegate to a sub-agent seeded with that summary), then proceed with the physics integration exploration.

Answer: D

Explanation:

A. Incorrect Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.

This helps divide the work, but it does not solve the underlying problem that the main context is already becoming degraded and losing project-specific accuracy.

B. Incorrect Use /clear to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.

Resetting without preserving the important rendering discoveries risks losing the specific knowledge about VulkanPipeline, FrameGraph, and the architectural relationships already uncovered.

C. Incorrect Continue in the current context with more targeted prompts referencing the specific classes by name.

The observed behavior indicates that the context is already overloaded. Simply adding more targeted prompts is unlikely to reliably restore the lost precision.

D. Correct Summarize key rendering findings (including specific class names and architectural patterns), clear or reset the main context window (or delegate to a sub-agent seeded with that summary), then proceed with the physics integration exploration.

This is the most effective approach because it preserves the high-value, project-specific rendering knowledge while freeing context capacity for the new investigation. A concise architectural summary acts as a durable bridge between the rendering and physics explorations and helps maintain accuracy during the transition.

Question: 172 **Exam Heist**

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration. What is the most effective approach to handle this situation?

A. Launch a fresh subagent and include the prior transcript in the initial prompt for context.

B. Resume the subagent from its previous transcript without mentioning the changes — the architecture understanding remains valid.

C. Launch a fresh subagent with a summary of prior findings.

D. Resume the subagent from its previous transcript and inform it about the renamed functions.

Answer: D

Explanation:

A. Incorrect Launch a fresh subagent and include the prior transcript in the initial prompt for context.

Reinserting the entire transcript is inefficient and may consume a large amount of context with redundant details.

B. Incorrect Resume the subagent from its previous transcript without mentioning the changes — the architecture understanding remains valid.

The codebase changed while the engineer was away. Omitting the renamed functions risks inconsistent references and incorrect conclusions.

C. Incorrect Launch a fresh subagent with a summary of prior findings.

A summary can preserve high-level understanding, but it loses the detailed state accumulated during the 30-minute investigation across 47 files.

D. Correct Resume the subagent from its previous transcript and inform it about the renamed functions.

This preserves the rich investigation context already built by the subagent while updating it with the relevant code changes introduced by the merged PR. The architecture and data-flow analysis remain valuable, and the renamed utilities can be reconciled with the existing exploration.

Question: 173 **Exam Heist**

Your schema includes a 'skills: string[] field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned."
What's the most effective improvement?

A. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

B. Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."

C. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.

D. Enrich the schema to skill: string, confidence: float, source_quote: string to capture extraction confidence.

Answer: A

Explanation:

A. Correct Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

Few-shot examples are the most effective improvement because they teach the model the desired extraction behavior directly. They show how to split compound phrases consistently, include only explicitly mentioned skills, and maintain a consistent level of granularity across documents. This addresses all three observed inconsistencies without imposing arbitrary limits.

B. Incorrect Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."

These constraints reduce variability, but a fixed maximum can truncate legitimate documents that contain many skills and may reduce extraction completeness.

C. Incorrect Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.

Normalization helps downstream consistency, but it does not teach the model how to decide what counts as a skill during extraction.

D. Incorrect Enrich the schema to skill: string, confidence: float, source quote: string to capture extraction provenance.

This improves traceability and auditing, but it does not directly resolve the inconsistent extraction behavior.

Question: 174 **Exam Heist**

During testing, you find that when a customer says "I need a refund for my recent purchase," the agent calls process_refund immediately – but populates the required 'order_id' parameter with a plausible-looking but fabricated value instead of first calling lookup_order to retrieve the actual order ID. The refund call fails because the fabricated ID doesn't exist.
Which change directly addresses the root cause of the agent fabricating the 'order_id' value?

A. Switch 'tool_choice' from "auto" to "any" to force the agent to make a tool call on every turn.

B. Pre-parse incoming customer messages to extract any order IDs mentioned, and inject them into the conversation context before passing to Claude.

C. Update the 'process_refund' tool description to explicitly state that 'order_id' must be obtained from a prior 'lookup_order' call and must never be assumed or invented.

D. Add server-side validation that checks whether the 'order_id' exists in your database before executing or to the agent if not.

Answer: C

Explanation:

A. Incorrect Switch 'tool_choice' from "auto" to "any" to force the agent to make a tool call on every turn.

This would ensure a tool is called, but it would not prevent the agent from fabricating an order_id when calling process_refund.

B. Incorrect Pre-parse incoming customer messages to extract any order IDs mentioned, and inject them into the conversation context before passing to Claude.

This helps only when the customer actually provides an order ID. In the example, no order ID was mentioned.

C. Correct Update the 'process_refund' tool description to explicitly state that 'order_id' must be obtained from a prior 'lookup_order' call and must never be assumed or invented.

The root cause is that the model is treating order_id as a required parameter that it can plausibly infer. Making the dependency explicit in the tool contract teaches the agent the correct workflow: first retrieve the order through lookup_order, then pass the real identifier to process_refund.

D. Incorrect Add server-side validation that checks whether the 'order_id' exists in your database before executing the refund and returns an error to the agent if not.

Validation is important for safety, but it only catches the fabricated ID after the incorrect behavior has already occurred. It does not address the root cause of the fabrication.

Question: 175 **Exam Heist**

A customer returns 4 hours after their initiar session about the same billing dispute. The previous 32-turn session contains lookup_order results showing "Status: PENDING, Expected resolution: 24-48 hours." In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., "I see your refund is still being processed") even after subsequent fresh tool calls return different information.
What approach most reliably handles returning customers?

A. Resume with full history but filter out previous tool_result messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data.

B. Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to thesame tool exist in context.

C. Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.

D. Resume with full history and configure the agent to automatically re-call all previously used tools at session start to ensure data freshness.

Answer: C

Explanation:

A. Incorrect Resume with full history but filter out previous tool_result messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data.

This removes stale data, but it also removes important structured information and may leave the conversation without a reliable record of what actions were previously taken.

B. Incorrect Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to the same tool exist in context.

This helps somewhat, but long histories still create confusion and increase the risk that outdated information influences the response.

C. Correct Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.

This is the most reliable approach because it preserves the important business context while ensuring that all operational data comes from fresh tool calls. It minimizes stale-state contamination and reduces context-window clutter.

D. Incorrect Resume with full history and configure the agent to automatically re-call all previously used tools at session start to ensure data freshness.

Fresh data helps, but the old tool results remain in context and can still bias the model toward outdated conclusions.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers



Future is Secured

100% Pass Guarantee



24/7 Customer Support

Mail us - support@examheist.com



Verified Updates

Lifetime Updates!

Total: **175 Questions**

Link: https://examheist.com/papers/anthropic/claude-certified-architect